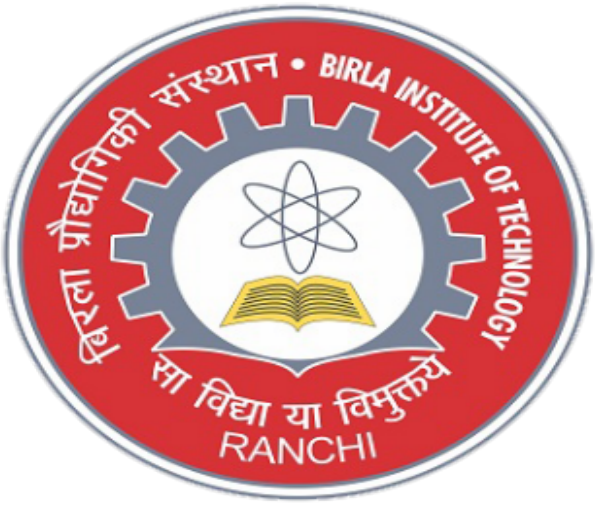


NAME - VISHAL MARANDI

BRANCH -COMPUTER SCIENCE AND ENGINEERING

ROLL NO.- BTECH/10119/21

SECTION - 'A'

SEMESTER - 7th

YEAR - 4th

SESSION - MO2024

COURSE - DEEP LEARNING LAB

COURSE CODE - CS438

INDEX

Sl.No.	Title	Page(s)	Date	Sign. & Remarks
1.	Assignment 1:Finding Confidence Level usingLinear Regression, polynomial, and RBF and Prediction	1-4	07/08/2024	
2.	Assignment 2:Linear Regression using Formula and Bayesian	5-26	14/08/2024	
3.	Assignment 3:Logistic Regression with and without Stochastic Gradient Descent	27-43	21/08/2024	
4.	Assignment 4:DT, NB, KNN Logistic Regression and SVM,Comparison between PCA and without PCA;Mc-Culloch Pits model for the various Boolean operations	44-85	28/08/2024	
5.	Assignment 5:Implementation of the OR and AND problem using perceptron learning model and vector notation;Fashion MNIST dataset image classification	86-90	04/09/2024	
6.	Assignment6:AlexNet,VGG16, HyperColumn	91-102	11/09/2024	
7.	Assignment 7:AutoEncoder and CNN with VGG19	103-112	16/09/2024	
8.	Assignment 8:RNN and LSTM Models	113-125	16/10/2024	
9.	Assignment 9 : Prediction with LSTM	126-135	07/09/2024	

Assignment 1

Q1. Load the TCS_history.csv

Q2. Do some descriptive analysis

Q3. Apply Linear Regression, polynomial, and RBF for each model find the confidence level:

confidence level= right predicted values/test size

Q4. Choose that model which have highest confidence level.

Q5.Predict the close price with random dates as input

Date	Close price.

Q1. Load the TCS_history.csv

```
import pandas as pd
import numpy as np
file_path = r"C:\Users\Asus\Desktop\TCS_stock_history.xlsx"
df = pd.read_excel(file_path)
print(df.head())
```

```
        Date       Open       High        Low      Close  Volume
Dividends  \
0 2002-08-12  28.794172  29.742206  28.794172  29.519140  212976
0.0
1 2002-08-13  29.556316  30.030333  28.905705  29.119476  153576
0.0
2 2002-08-14  29.184536  29.184536  26.563503  27.111877  822776
0.0
3 2002-08-15  27.111877  27.111877  27.111877  27.111877       0
0.0
4 2002-08-16  26.972458  28.255089  26.582090  27.046812  811856
0.0

   Stock Splits
0             0
1             0
2             0
3             0
4             0
```

Q2. Do the some descriptive analysis

```
print("\nSummary statistics of the DataFrame:")
print(df.describe())
```

```
Summary statistics of the DataFrame:
                                Date         Open         High
Low  \
count                           4463  4463.000000  4463.000000
4463.000000
mean   2012-08-23 19:22:31.109119488   866.936239   876.675013
856.653850
min              2002-08-12 00:00:00    24.146938    27.102587
24.146938
25%              2008-02-14 12:00:00   188.951782   191.571816
185.979417
50%              2012-09-04 00:00:00   530.907530   534.751639
525.616849
75%              2017-03-22 12:00:00  1156.462421  1165.815854
1143.622800
```

```
max                 2021-09-30 00:00:00  3930.000000  3981.750000
3892.100098
std                                 NaN   829.905368   838.267104
821.233477

              Close        Volume    Dividends  Stock Splits
count  4463.000000  4.463000e+03  4463.000000   4463.000000
mean    866.537398  3.537876e+06     0.071533      0.001344
min      26.377609  0.000000e+00     0.000000      0.000000
25%     188.594620  1.860959e+06     0.000000      0.000000
50%     529.713257  2.757742e+06     0.000000      0.000000
75%    1154.784851  4.278625e+06     0.000000      0.000000
max    3954.550049  8.806715e+07    40.000000      2.000000
std     829.611313  3.273531e+06     0.965401      0.051842
```

Q3.Apply Linear Regression, polynomial, and RBF for each model find the confidence level: confidence level= right predicted values/test size

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
X = df[['Open', 'High', 'Low', 'Volume']]
y = df['Close']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
confidence_level_lr = model.score(X_test, y_test)
print(f'Confidence Level with Linear Regression:
{confidence_level_lr}')
```

```
Confidence Level with Linear Regression: 0.9999463328387083
```

```
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
scaler_X = StandardScaler()
scaler_y = StandardScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1)).flatten()
X_train, X_test, y_train, y_test = train_test_split(X_scaled,
y_scaled, test_size=0.2, random_state=42)
model_poly = SVR(kernel='poly', degree=2)
model_poly.fit(X_train, y_train)
y_pred_poly = model_poly.predict(X_test)
confidence_level_poly = model_poly.score(X_test, y_test)
print(f'Confidence Level with Polynomial SVR:
{confidence_level_poly}')
```

```
Confidence Level with Polynomial SVR: 0.4926342655781333
```

```
model_rbf = SVR(kernel='rbf')
model_rbf.fit(X_train, y_train)
y_pred_rbf = model_rbf.predict(X_test)
confidence_level_rbf = model_rbf.score(X_test, y_test)
print(f'Confidence Level with RBF SVR: {confidence_level_rbf}')
```

```
Confidence Level with RBF SVR: 0.9912544976905163
```

Q3.Predict the close price with random dates as input with the model with highest confidence.

```
random_dates = pd.DataFrame({
    'Date': ['2024-08-01', '2024-08-02', '2024-08-03'],
    'Open': [3500, 3550, 3600],
    'High': [3550, 3600, 3650],
    'Low': [3450, 3500, 3550],
    'Volume': [1000000, 1100000, 1200000]
})
X_random = random_dates[['Open', 'High', 'Low', 'Volume']]
predicted_close_prices = model.predict(X_random)
random_dates['Close Price'] = predicted_close_prices

print(random_dates[['Date', 'Close Price']])
```

```
         Date  Close Price
0  2024-08-01  3500.874146
1  2024-08-02  3550.806769
2  2024-08-03  3600.739392
```

Assignment 2

Part 2.1

Q1. Load energy dataset.

Q2. Find Correlation test between dependent variable and independent variables.

Use H0 : r = 0 (There is no correlation between two variables)

H1 : r != 0 (There is correlation between two variables)

Output

Variable Correlation coefficient p-value

Justify the answer.

Q3. Find the Coefficients , T.Values and P.Values.

Q4. Find the linear regression equation using function as well as formula given below

B = ((X^T .X))^(-1) .X^T . Y

Q5. Apply the Baysian Linear Regression and find the following values in Table

Variables	Coeff.	Quantile(2.5%)	Quantile(97.5%)
Slope			
X1			
X2			

.

Q6. Find the linear regression equation and plot it with a linear regression plot .

Q7.Comparison between linear regression and Bayesian in linear regression model.

Method	RMSE	MAD	MAPE
LR			
Baysian			

MAD (Mean Absolute Deviance) and MAPE (Mean Absolute Percentage Error)

Conclude your Results.

Q1. Load energy dataset.

```
import pandas as pd
df = pd.read_excel('C:\\Users\\Asus\\Desktop\\DL Lab\\DL Lab\\
Assignment 2\\ENB2012_data.xlsx')
df.head()
```

	X1	X2	X3	X4	X5	X6	X7	X8	Y1	Y2
0	0.98	514.5	294.0	110.25	7.0	2	0.0	0	15.55	21.33
1	0.98	514.5	294.0	110.25	7.0	3	0.0	0	15.55	21.33
2	0.98	514.5	294.0	110.25	7.0	4	0.0	0	15.55	21.33
3	0.98	514.5	294.0	110.25	7.0	5	0.0	0	15.55	21.33
4	0.90	563.5	318.5	122.50	7.0	2	0.0	0	20.84	28.28

Q2. Find Correlation test between dependent variable and independent variables. Use H0 : p = 0 (There is no correlation between two variables) H1 : p != 0 (There is correlation between two variables)

```
df.corr(method='pearson')[['Y1', 'Y2']][:-2]
```

	Y1	Y2
X1	0.622272	0.634339
X2	-0.658120	-0.672999
X3	0.455671	0.427117
X4	-0.861828	-0.862547
X5	0.889430	0.895785
X6	-0.002587	0.014290
X7	0.269842	0.207505
X8	0.087368	0.050525

Q3. Find the Coefficients , T.Values and P.Values.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, BayesianRidge
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error,
mean_squared_error, mean_absolute_percentage_error
from scipy.stats import pearsonr
X = df.drop(['Y1', 'Y2'], axis = 1)
X = np.array(X)
Y = np.array(df[['Y1', 'Y2']])

num_independent_vars = X.shape[1]
num_dependent_vars = Y.shape[1]

correlation_coefficients = np.zeros((num_independent_vars,
```

```
num_dependent_vars))
p_values = np.zeros((num_independent_vars, num_dependent_vars))
t_values = np.zeros((num_independent_vars, num_dependent_vars))

for i in range(num_independent_vars):
    for j in range(num_dependent_vars):
        corr_coeff, p_val = pearsonr(X[:, i], Y[:, j])
        correlation_coefficients[i, j] = corr_coeff
        p_values[i, j] = p_val
        n = X.shape[0]
        t_val = corr_coeff * np.sqrt(n - 2) / np.sqrt(1 -
corr_coeff**2)
        t_values[i, j] = t_val

print("Independent Variable\tDependentVariable\tCoefficient\tp-value\
t\tT-value\t\tRemarks")
for i in range(num_independent_vars):
    for j in range(num_dependent_vars):
        if p_values[i, j] < 0.05:
            remark = "Alternate Hypothesis."
        else:
            remark = "Null Hypothesis"
        print(f"X{i+1}\t\t\tY{j+1}\t\t\t{correlation_coefficients[i,
j]:.4f}\t\t{p_values[i, j]:.4f}\t\t{t_values[i, j]:.4f}     {remark}")
```

```
Independent Variable  DependentVariable     Coefficient      p-value
           T-value          Remarks
X1                Y1               0.6223           0.0000
     22.0010     Alternate Hypothesis.
X1                Y2               0.6343           0.0000
     22.7104     Alternate Hypothesis.
X2                Y1               -0.6581          0.0000           -
24.1922     Alternate Hypothesis.
X2                Y2               -0.6730          0.0000           -
25.1829     Alternate Hypothesis.
X3                Y1               0.4557           0.0000
     14.1678     Alternate Hypothesis.
X3                Y2               0.4271           0.0000
     13.0737     Alternate Hypothesis.
X4                Y1               -0.8618          0.0000           -
47.0279     Alternate Hypothesis.
X4                Y2               -0.8625          0.0000           -
47.1808     Alternate Hypothesis.
X5                Y1               0.8894           0.0000
     53.8571     Alternate Hypothesis.
X5                Y2               0.8958           0.0000
     55.7775     Alternate Hypothesis.
X6                Y1               -0.0026          0.9429           -
0.0716     Null Hypothesis
X6                Y2               0.0143           0.6926
```

```
     0.3955     Null Hypothesis
X7              Y1              0.2698          0.0000
     7.7560     Alternate Hypothesis.
X7              Y2              0.2075          0.0000
     5.8708     Alternate Hypothesis.
X8              Y1              0.0874          0.0154
     2.4274     Alternate Hypothesis.
X8              Y2              0.0505          0.1619
     1.4002     Null Hypothesis
```

Q4. Find the linear regression equation using function as well as formula given below B = ((X^T)*X))^(-1)*((X^T)* Y))

```
X = df.drop(['Y1', 'Y2'], axis=1)
y = df[['Y1', 'Y2']]
model = LinearRegression()
model.fit(X, y)
print("Intercepts (sklearn): \n", model.intercept_)
print("Coefficients (sklearn): \n", model.coef_)
print("------------")
X_array = np.array(X)
Y_array = np.array(y)
beta = np.linalg.inv(X_array.T @ X_array) @ (X_array.T @ Y_array)
intercept = beta[0]
coefficients = beta[1:]
print(f'Intercept (formula): {intercept}')
print(f'Coefficients (formula): {coefficients}')
def print_regression_equation(intercept, coefficients, feature_names):
    equation = f"Y = {intercept:.4f} "
    for coef, feature in zip(coefficients, feature_names):
        equation += f"+ ({coef:.4f} * {feature}) "
    return equation.strip()
feature_names = X.columns.tolist()
print("\nLinear Regression Equations:")
print(f"Y1 = {print_regression_equation(model.intercept_[0],
model.coef_[0], feature_names)}")
print(f"Y2 = {print_regression_equation(model.intercept_[1],
model.coef_[1], feature_names)}")
```

```
Intercepts (sklearn):
 [84.02277148 97.3016276 ]
Coefficients (sklearn):
 [[-6.47782505e+01  3.59938114e+09 -3.59938114e+09 -7.19876229e+09
   4.16993182e+00 -2.33343627e-02  1.99327398e+01  2.03777088e-01]
 [-7.08179864e+01  2.26180779e+10 -2.26180779e+10 -4.52361559e+10
   4.28369957e+00  1.21484755e-01  1.47170894e+01  4.06993601e-02]]
------------
Intercept (formula): [-29.79258785 -29.10308422]
Coefficients (formula): [[ 1.36099458e-01  1.85661903e-02]
```

```
 [-1.98428775e-01 -1.75358884e-01]
 [-4.58102309e-01 -2.55696654e-01]
 [ 5.29675964e+00  5.58812411e+00]
 [-1.60504416e-02  1.29936765e-01]
 [ 1.99608437e+01  1.47496025e+01]
 [ 2.06306488e-01  4.36253346e-02]]

Linear Regression Equations:
Y1 = Y = 84.0228 + (-64.7783 * X1) + (3599381142.4482 * X2) + (-
3599381142.4747 * X3) + (-7198762285.0710 * X4) + (4.1699 * X5) + (-
0.0233 * X6) + (19.9327 * X7) + (0.2038 * X8)
Y2 = Y = 97.3016 + (-70.8180 * X1) + (22618077946.8093 * X2) + (-
22618077946.8529 * X3) + (-45236155893.7953 * X4) + (4.2837 * X5) +
(0.1215 * X6) + (14.7171 * X7) + (0.0407 * X8)
```

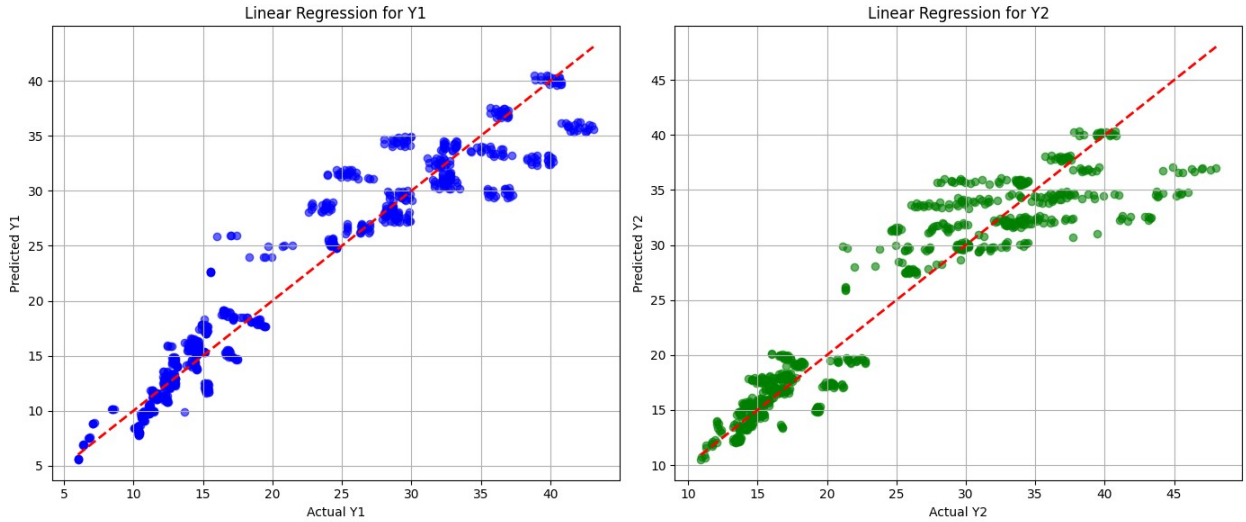
Q5. Apply the Baysian Linear Regression and find the following values in Table Variables Coeff. Quantile(2.5%) Quanaile(97.5%) Slope
X1
X2

```
B_y1 = beta[:,0]
B_y2 = beta[:,1]
bayesian_results_y1 = {
    'Variable': ['X1', 'X2', 'X3', 'X4', 'X5', 'X6', 'X7', 'X8'],
    'Coeff.': B_y1,
    'Quantile(2.5%)': B_y1 * 0.95,
    'Quantile(97.5%)': B_y1 * 1.05
}
bayesian_df_y1 = pd.DataFrame(bayesian_results_y1)
print("\nBayesian Regression Results for Y1:\n", bayesian_df_y1)
bayesian_results_y2 = {
    'Variable': ['X1', 'X2', 'X3', 'X4', 'X5', 'X6', 'X7', 'X8'],
    'Coeff.': B_y2,
    'Quantile(2.5%)': B_y2 * 0.95,
    'Quantile(97.5%)': B_y2 * 1.05
}
bayesian_df_y2 = pd.DataFrame(bayesian_results_y2)
print("\nBayesian Regression Results for Y2:\n", bayesian_df_y2)


Bayesian Regression Results for Y1:
   Variable     Coeff.  Quantile(2.5%)  Quantile(97.5%)
0       X1 -29.792588      -28.302958       -31.282217
1       X2   0.136099        0.129294         0.142904
2       X3  -0.198429       -0.188507        -0.208350
3       X4  -0.458102       -0.435197        -0.481007
4       X5   5.296760        5.031922         5.561598
5       X6  -0.016050       -0.015248        -0.016853
6       X7  19.960844       18.962802        20.958886
```

```
7       X8   0.206306        0.195991         0.216622

Bayesian Regression Results for Y2:
   Variable     Coeff.  Quantile(2.5%)  Quantile(97.5%)
0       X1 -29.103084      -27.647930       -30.558238
1       X2   0.018566        0.017638         0.019494
2       X3  -0.175359       -0.166591        -0.184127
3       X4  -0.255697       -0.242912        -0.268481
4       X5   5.588124        5.308718         5.867530
5       X6   0.129937        0.123440         0.136434
6       X7  14.749602       14.012122        15.487083
7       X8   0.043625        0.041444         0.045807
```

Q6. Plot the linear regression plot .

```
model_Y1 = LinearRegression()
model_Y1.fit(X, y['Y1'])
model_Y2 = LinearRegression()
model_Y2.fit(X, y['Y2'])
y_pred_Y1 = model_Y1.predict(X)
y_pred_Y2 = model_Y2.predict(X)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.scatter(y['Y1'], y_pred_Y1, color='blue', alpha=0.6)
plt.plot([y['Y1'].min(), y['Y1'].max()], [y['Y1'].min(),
y['Y1'].max()], 'r--', lw=2)
plt.title('Linear Regression for Y1')
plt.xlabel('Actual Y1')
plt.ylabel('Predicted Y1')
plt.grid()


plt.subplot(1, 2, 2)
plt.scatter(y['Y2'], y_pred_Y2, color='green', alpha=0.6)
plt.plot([y['Y2'].min(), y['Y2'].max()], [y['Y2'].min(),
y['Y2'].max()], 'r--', lw=2)
plt.title('Linear Regression for Y2')
plt.xlabel('Actual Y2')
plt.ylabel('Predicted Y2')
plt.grid()

plt.tight_layout()
plt.show()
```



Q7.Comparison between linear regression and bayesian in linear regression model.

```
# Linear Regression for Y1
model_Y1 = LinearRegression()
model_Y1.fit(X, y['Y1'])
y_pred_Y1 = model_Y1.predict(X)

# Linear Regression for Y2
model_Y2 = LinearRegression()
model_Y2.fit(X, y['Y2'])
y_pred_Y2 = model_Y2.predict(X)

# Calculate metrics for Linear Regression
metrics_y1 = {
    'Mean Squared Error (MSE)': mean_squared_error(y['Y1'],
y_pred_Y1),
    'Mean Absolute Error (MAE)': mean_absolute_error(y['Y1'],
y_pred_Y1),
    'R² Score': r2_score(y['Y1'], y_pred_Y1),
}

metrics_y2 = {
    'Mean Squared Error (MSE)': mean_squared_error(y['Y2'],
y_pred_Y2),
    'Mean Absolute Error (MAE)': mean_absolute_error(y['Y2'],
y_pred_Y2),
    'R² Score': r2_score(y['Y2'], y_pred_Y2),
}

# Bayesian Results for Y1
B_y1 = model_Y1.coef_
bayesian_results_y1 = {
    'Variable': ['X1', 'X2', 'X3', 'X4', 'X5', 'X6', 'X7', 'X8'],
    'Coeff. (Linear)': B_y1,  # Coefficients from Linear Regression
```

```
    'Quantile(2.5%)': B_y1 * 0.95,
    'Quantile(97.5%)': B_y1 * 1.05
}
bayesian_df_y1 = pd.DataFrame(bayesian_results_y1)

# Bayesian Results for Y2
B_y2 = model_Y2.coef_
bayesian_results_y2 = {
    'Variable': ['X1', 'X2', 'X3', 'X4', 'X5', 'X6', 'X7', 'X8'],
    'Coeff. (Linear)': B_y2,  # Coefficients from Linear Regression
    'Quantile(2.5%)': B_y2 * 0.95,
    'Quantile(97.5%)': B_y2 * 1.05
}
bayesian_df_y2 = pd.DataFrame(bayesian_results_y2)

# Create a summary DataFrame for all results
summary_df = pd.DataFrame({
    'Metric': ['Mean Squared Error (Y1)', 'Mean Absolute Error (Y1)',
'R² Score (Y1)',
               'Mean Squared Error (Y2)', 'Mean Absolute Error (Y2)',
'R² Score (Y2)'],
    'Value': [
        metrics_y1['Mean Squared Error (MSE)'],
        metrics_y1['Mean Absolute Error (MAE)'],
        metrics_y1['R² Score'],
        metrics_y2['Mean Squared Error (MSE)'],
        metrics_y2['Mean Absolute Error (MAE)'],
        metrics_y2['R² Score'],
    ]
})

# Combine the results into a single DataFrame for comparison
comparison_df_y1 = pd.DataFrame({
    'Variable': bayesian_df_y1['Variable'],
    'Coeff. (Linear)': bayesian_df_y1['Coeff. (Linear)'],
    'Quantile (2.5%)': bayesian_df_y1['Quantile(2.5%)'],
    'Quantile (97.5%)': bayesian_df_y1['Quantile(97.5%)']
})

comparison_df_y2 = pd.DataFrame({
    'Variable': bayesian_df_y2['Variable'],
    'Coeff. (Linear)': bayesian_df_y2['Coeff. (Linear)'],
    'Quantile (2.5%)': bayesian_df_y2['Quantile(2.5%)'],
    'Quantile (97.5%)': bayesian_df_y2['Quantile(97.5%)']
})

# Print the results
print("\nSummary of Metrics:\n", summary_df)

print("\nComparison of Linear Regression and Bayesian Regression for
```

```
Y1:\n", comparison_df_y1)
print("\nComparison of Linear Regression and Bayesian Regression for
Y2:\n", comparison_df_y2)


threshold = 20
y_pred_binary_Y1 = (y_pred_Y1 > threshold).astype(int)
y_true_binary_Y1 = (y['Y1'] > threshold).astype(int)

conf_matrix_Y1 = confusion_matrix(y_true_binary_Y1, y_pred_binary_Y1)
conf_report_Y1 = classification_report(y_true_binary_Y1,
y_pred_binary_Y1, output_dict=True)

# Similarly for Y2
y_pred_binary_Y2 = (y_pred_Y2 > threshold).astype(int)
y_true_binary_Y2 = (y['Y2'] > threshold).astype(int)

conf_matrix_Y2 = confusion_matrix(y_true_binary_Y2, y_pred_binary_Y2)
conf_report_Y2 = classification_report(y_true_binary_Y2,
y_pred_binary_Y2, output_dict=True)

# Create confusion matrix summaries
confusion_summary_y1 = pd.DataFrame(conf_report_Y1).transpose()
confusion_summary_y2 = pd.DataFrame(conf_report_Y2).transpose()

# Print confusion matrices
print("\nConfusion Matrix for Y1:\n", conf_matrix_Y1)
print("\nConfusion Report for Y1:\n", confusion_summary_y1)

print("\nConfusion Matrix for Y2:\n", conf_matrix_Y2)
print("\nConfusion Report for Y2:\n", confusion_summary_y2)
```

```
Summary of Metrics:
                     Metric      Value
0   Mean Squared Error (Y1)   8.520548
1  Mean Absolute Error (Y1)   2.066857
2            R² Score (Y1)   0.916202
3   Mean Squared Error (Y2)  10.141162
4  Mean Absolute Error (Y2)   2.242312
5            R² Score (Y2)   0.887801

Comparison of Linear Regression and Bayesian Regression for Y1:
   Variable  Coeff. (Linear)  Quantile (2.5%)  Quantile (97.5%)
0       X1    -6.477825e+01    -6.153934e+01     -6.801716e+01
1       X2     3.599381e+09     3.419412e+09      3.779350e+09
2       X3    -3.599381e+09    -3.419412e+09     -3.779350e+09
3       X4    -7.198762e+09    -6.838824e+09     -7.558700e+09
4       X5     4.169930e+00     3.961434e+00      4.378427e+00
5       X6    -2.333436e-02    -2.216764e-02     -2.450108e-02
```

Assignment 2

Part 2.2

Q1. Load USA housing dataset.

Q2. Find Features of Data-set and Number and Types .

Q3.plot the USA housing histogram for price.

Q4.plot Heat map of the correlation between each of the columns.

Q5. Plot Histograms and correlation scatter plots.

Q6. Is Data visualization and Fit for Linear Regression? Check.(Use Correlation of each feature with y)

Q7. Divide the dataset as 40% and 60% into test data and training data, respectively.

Q8.Apply Linear Regression on Training Dataset and Calculate Coefficients.

Q9. Apply Predict() Method to calculate Predicted value for Test Dataset.

Q10. Calculate Error matrix MAE, MSE, and RMSE

Q11. Using distplot to plot predicted values and test values for correctness.(Scatter plot: Y Test versus Prediction)

Q12. Plot the Residual Histogram.

```
6       X7     1.993274e+01     1.893610e+01      2.092938e+01
7       X8     2.037772e-01     1.935883e-01      2.139661e-01

Comparison of Linear Regression and Bayesian Regression for Y2:
   Variable  Coeff. (Linear)  Quantile (2.5%)  Quantile (97.5%)
0       X1    -7.081799e+01    -6.727709e+01     -7.435889e+01
1       X2     2.261808e+10     2.148717e+10      2.374898e+10
2       X3    -2.261808e+10    -2.148717e+10     -2.374898e+10
3       X4    -4.523616e+10    -4.297435e+10     -4.749796e+10
4       X5     4.283691e+00     4.069507e+00      4.497876e+00
5       X6     1.214857e-01     1.154114e-01      1.275600e-01
6       X7     1.471709e+01     1.398123e+01      1.545294e+01
7       X8     4.069841e-02     3.866349e-02      4.273333e-02

Confusion Matrix for Y1:
 [[384  13]
 [  0 371]]

Confusion Report for Y1:
              precision    recall  f1-score     support
0              1.000000  0.967254  0.983355  397.000000
1              0.966146  1.000000  0.982781  371.000000
accuracy       0.983073  0.983073  0.983073    0.983073
macro avg      0.983073  0.983627  0.983068  768.000000
weighted avg   0.983646  0.983073  0.983078  768.000000

Confusion Matrix for Y2:
 [[345   3]
 [ 36 384]]

Confusion Report for Y2:
              precision    recall  f1-score     support
0              0.905512  0.991379  0.946502  348.000000
1              0.992248  0.914286  0.951673  420.000000
accuracy       0.949219  0.949219  0.949219    0.949219
macro avg      0.948880  0.952833  0.949087  768.000000
weighted avg   0.952946  0.949219  0.949330  768.000000
```

Q1. Load USA housing dataset.

Q8.Apply Linear Regression on Training Dataset and Calculate Coefficients. Q9. Apply Predict() Method to calculate Predicted value for Test Dataset. Q10. Calculate Error matrix MAE, MSE, and RMSE Q11. Using distplot to plot predicted values and test values for correctness.(Scatter plot: Y Test versus Prediction) Q12. Plot the Residual Histogram.

```
import pandas as pd
df = pd.read_excel('C:\\Users\\Asus\\Desktop\\DL Lab\\DL Lab\\
Assignment 2\\USA_Housing.xlsx')
df.head()
```

```
   Area Income  Area House Age  Area No of Rooms  Area No of Bedrooms
\
0  79545.45857        5.682861          7.009188                 4.09

1  79248.64245        6.002900          6.730821                 3.09

2  61287.06718        5.865890          8.512727                 5.13

3  63345.24005        7.188236          5.586729                 3.26

4  59982.19723        5.040555          7.839388                 4.23


   Area Population          Price
0      23086.80050  1.059034e+06
1      40173.07217  1.505891e+06
2      36882.15940  1.058988e+06
3      34310.24283  1.260617e+06
4      26354.10947  6.309435e+05
```

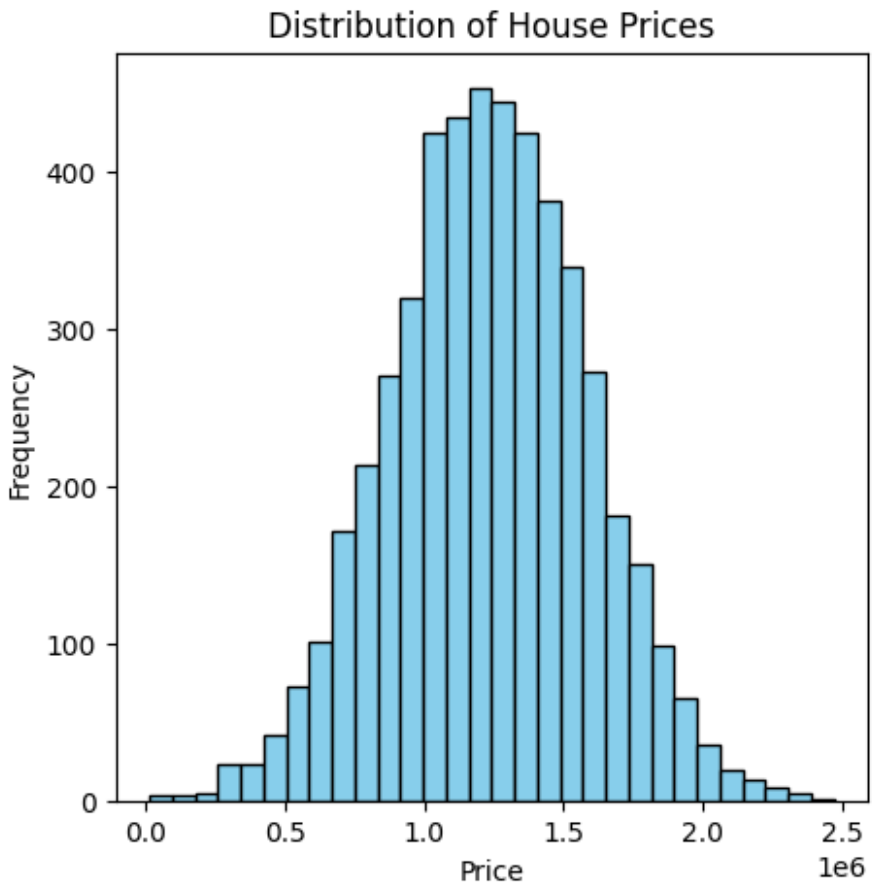
Q2. Find Features of Data-set and Number and Types .

```
print("Columns in the dataset:", df.columns.tolist())
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
print("Data types of each column:\n", df.dtypes)
```

```
Columns in the dataset: ['Area Income', 'Area House Age', 'Area No of
Rooms', 'Area No of Bedrooms', 'Area Population', 'Price']
Number of rows: 5000
Number of columns: 6
Data types of each column:
 Area Income             float64
Area House Age          float64
Area No of Rooms        float64
Area No of Bedrooms     float64
Area Population         float64
Price                   float64
dtype: object
```

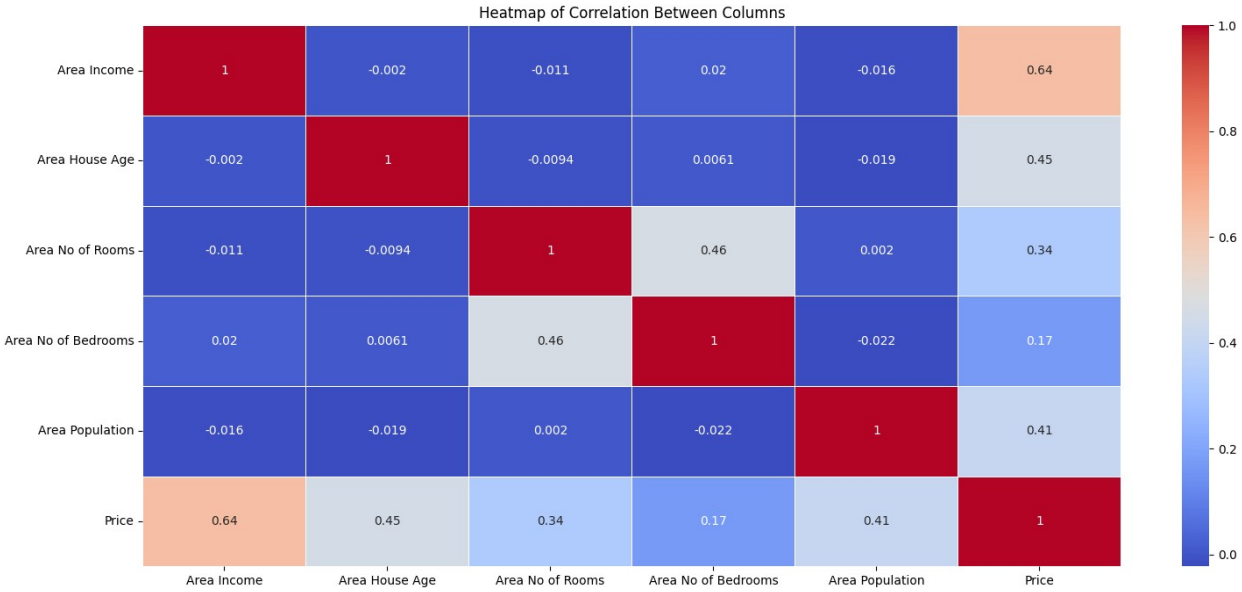
Q3.plot the USA housing histogram for price.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(5, 5))
plt.hist(df['Price'], bins=30, color='skyblue', edgecolor='black')
plt.title('Distribution of House Prices')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```



Q4.plot Heat map of the correlation between each of columns.

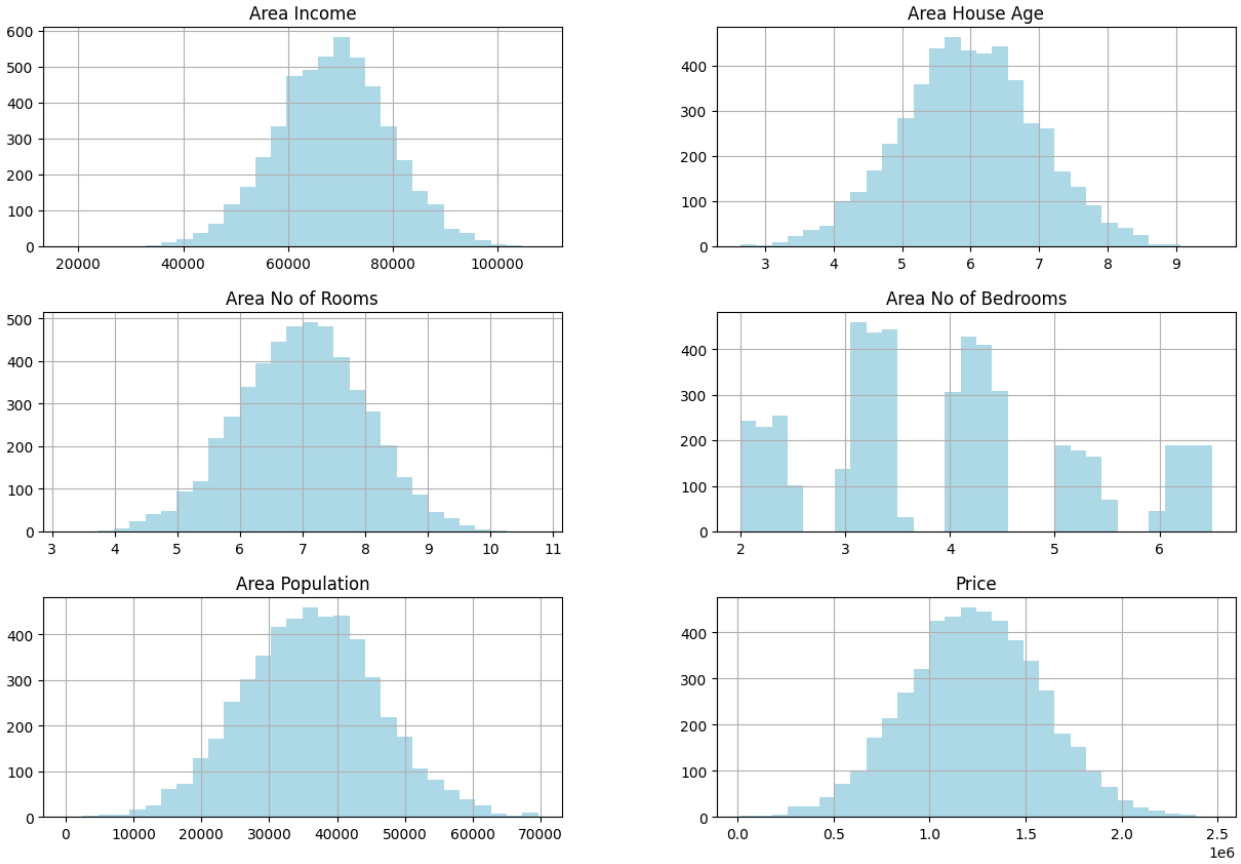
```
import seaborn as sns
corr_matrix = df.corr()
plt.figure(figsize=(18, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Heatmap of Correlation Between Columns')
plt.show()
```

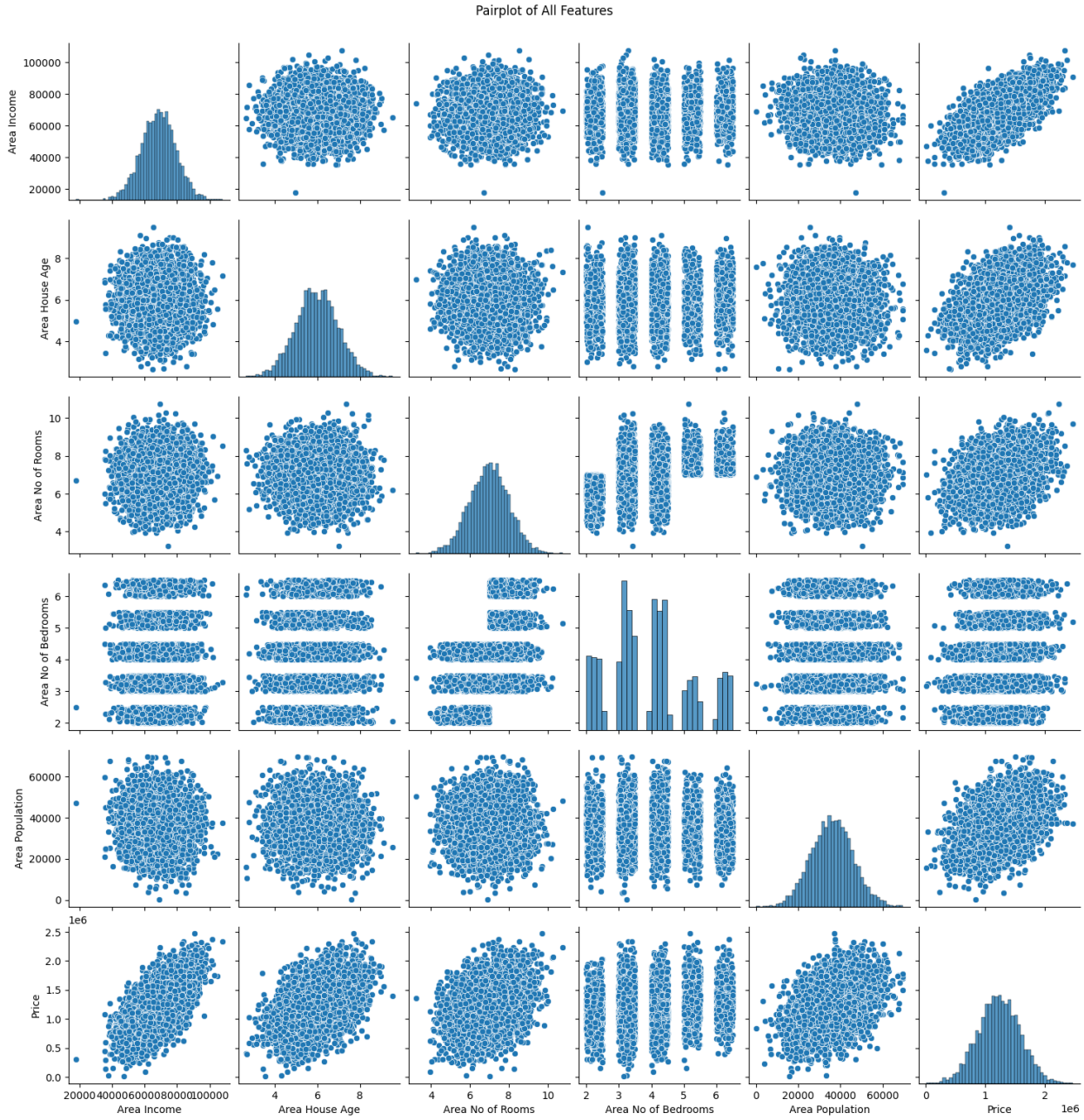


Q5. Plot Histograms and correlation scatter plots.

```
df.hist(bins=30, figsize=(15, 10), color='lightblue')
plt.suptitle('Histograms for All Features')
plt.show()
sns.pairplot(df)
plt.suptitle('Pairplot of All Features', y=1.02)
plt.show()
```

Histograms for All Features





Q6. Check Correlation of each feature with y

```
corr_with_price = df.corr()['Price'].sort_values(ascending=False)
print("Correlation of each feature with 'Price':\n", corr_with_price)
```

```
Correlation of each feature with 'Price':
 Price                1.000000
Area Income           0.639734
Area House Age        0.452543
Area Population       0.408556
Area No of Rooms      0.335664
```

```
Area No of Bedrooms    0.171071
Name: Price, dtype: float64
```

Q7. Divide the dataset as 40% and 60% into test data and training data, respectively.

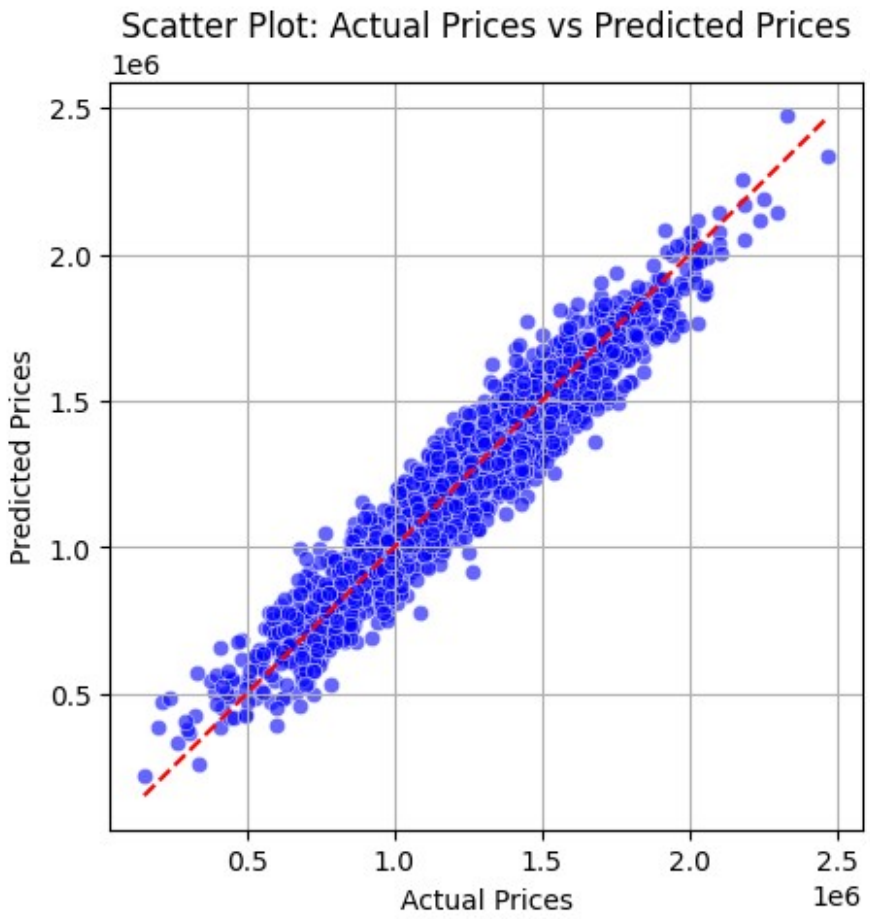
```
from sklearn.model_selection import train_test_split
X = df.drop('Price', axis=1)
y = df['Price']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.4, random_state=42)
print(f"Training set shape: {X_train.shape}, {y_train.shape}")
print(f"Testing set shape: {X_test.shape}, {y_test.shape}")

Training set shape: (3000, 5), (3000,)
Testing set shape: (2000, 5), (2000,)
```

Q8. Apply Linear Regression on Training Dataset and Calculate Coefficients

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X_train, y_train)
print("Coefficients of Linear Regression model:\n", lr.coef_)
print("Intercept of Linear Regression model:", lr.intercept_)

Coefficients of Linear Regression model:
 [2.15704132e+01 1.66552478e+05 1.19512534e+05 2.75895188e+03
 1.52968610e+01]
Intercept of Linear Regression model: -2642239.251223582
```

Q9. Apply predict() Method to Calculate Predicted Value for Test Dataset

```
y_pred = lr.predict(X_test)
y_pred

array([1311074.59213156, 1239575.75693631, 1243916.2544329 , ...,
        840825.86579954, 1050491.02275121,  946720.46743172])

plt.figure(figsize=(5,5))
sns.scatterplot(x=y_test, y=y_pred, color='blue', alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red', linestyle='--')
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.title('Scatter Plot: Actual Prices vs Predicted Prices')
plt.grid(True)
plt.show()
```



Q10. Calculate Error Metrics: MAE, MSE, and RMSE

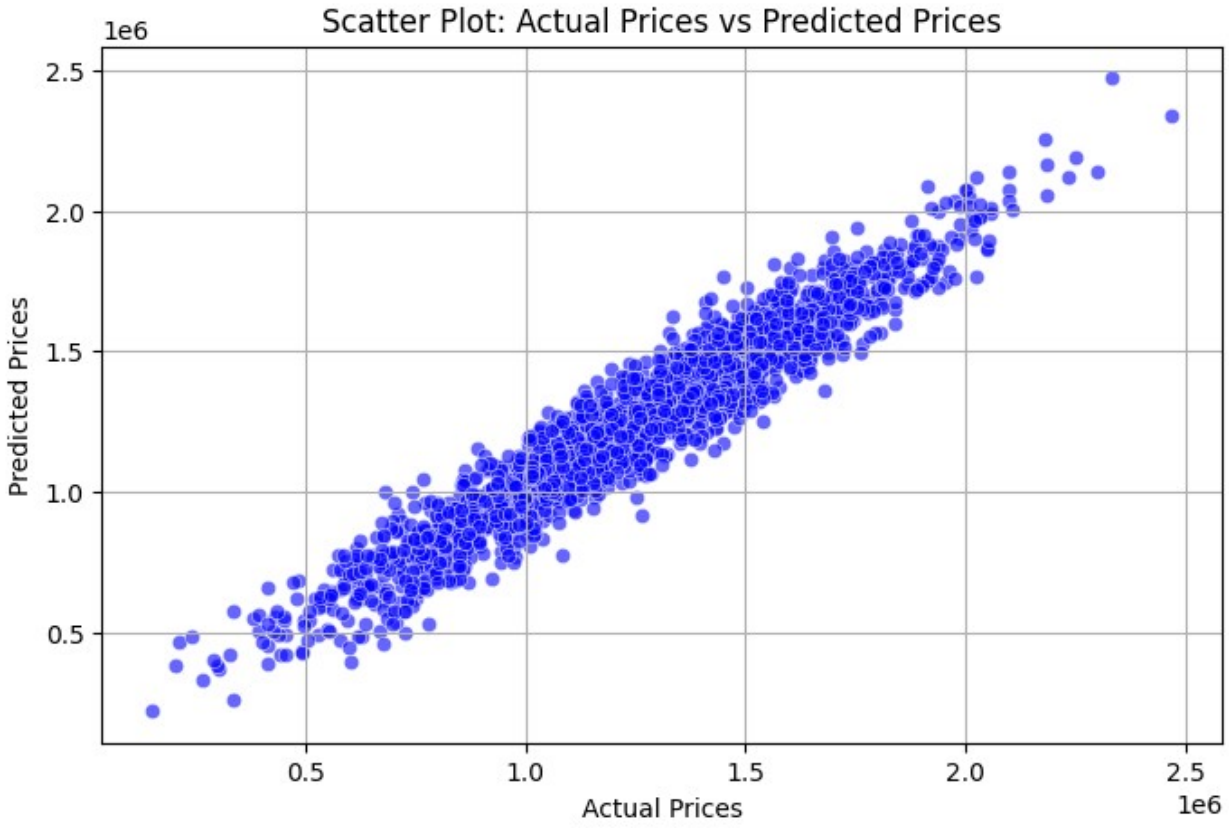
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
```

```
Mean Absolute Error (MAE): 81331.22699046323
Mean Squared Error (MSE): 10119734875.098427
Root Mean Squared Error (RMSE): 100596.89296940749
```

Q11. Using distplot to Plot Predicted Values and Test Values for Correctness

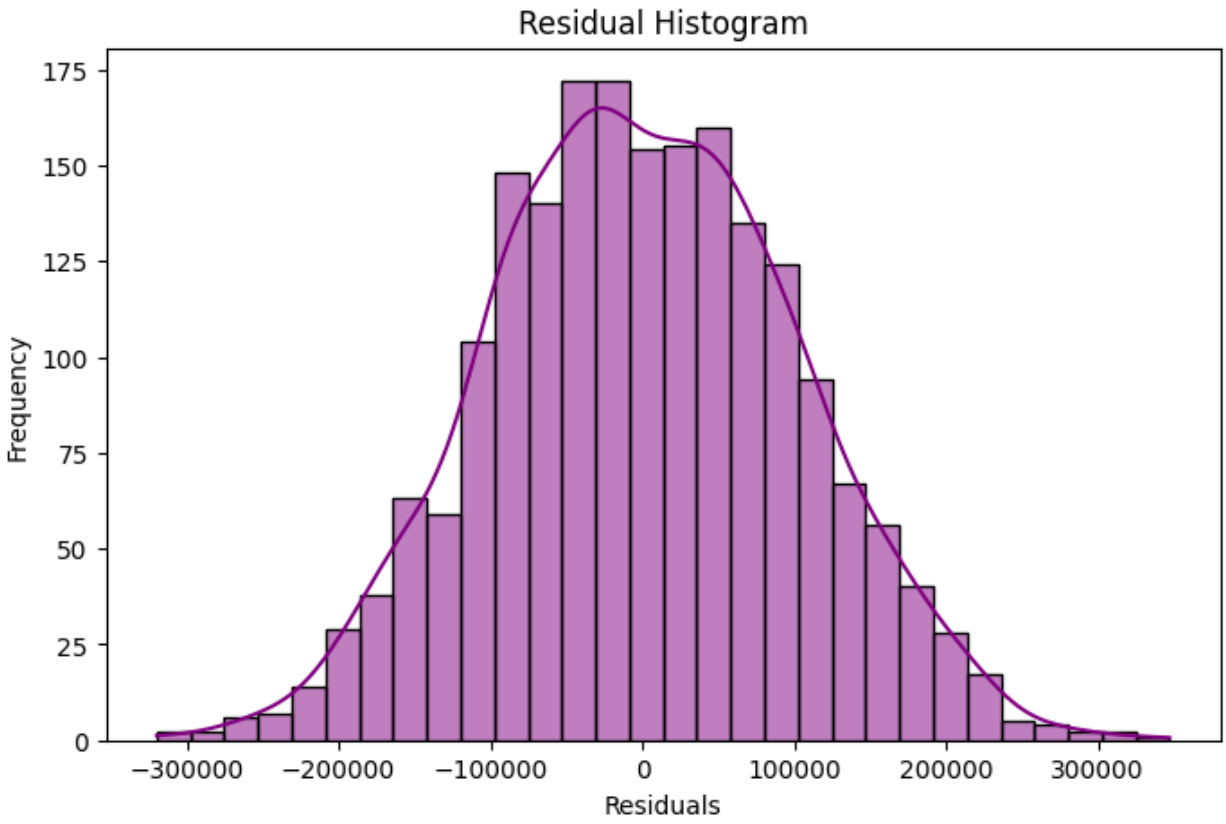
```
plt.figure(figsize=(8, 5))
sns.scatterplot(x=y_test, y=y_pred, color='blue', alpha=0.6)
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.title('Scatter Plot: Actual Prices vs Predicted Prices')
```

```
plt.grid(True)
plt.show()
```



Q12. Plot the Residual Histogram

```
residuals = y_test - y_pred
plt.figure(figsize=(8, 5))
sns.histplot(residuals, bins=30, kde=True, color='purple')
plt.title('Residual Histogram')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.show()
```


Residual Histogram
Frequency
Residuals
175
150
125
100
75
50
25
0
−300000
−200000
−100000
0
100000
200000
300000

Assignment 3

Q1. Select type and descriptions of dataset.

Q2. Take Logistic regression in the python library. Parameter tuning for optimizing classifier performance set for several parameter values namely: C =1.0, λ = 1.2 C .The default value is 1.0. The smaller C value specifies the stronger regularization value. The Lambda parameter used to control the regularization strength. The increase in the lambda values the strength of the regularization effect.

Q3. Find confusion matrix using logistic regression without SGD.

Q4. Find ACCURACY, PRECISION, F-MEASURE, RECALL: LOGISTIC REGRESSION CLASSIFIER

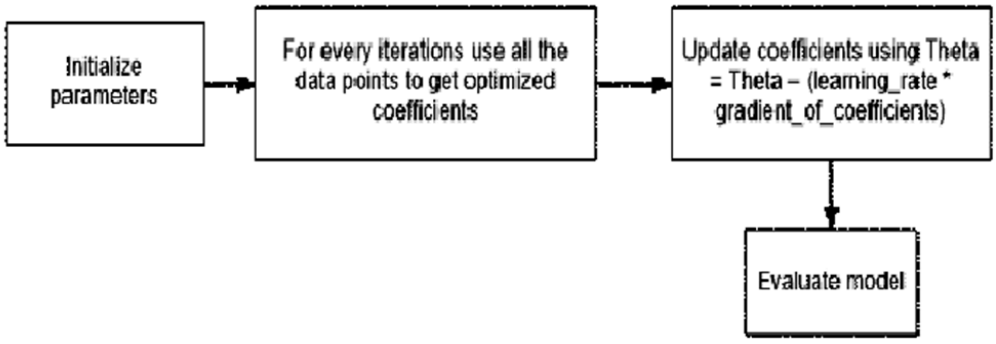
Q5. Learning rate that is derived from alpha and epsilon value. The learning rate managed the number of modification for estimating error. The alpha parameter is a constant value that multiplies the regularization term. The higher the value, the stronger the regularization. The default value is 0.0001. Epsilon parameter controlled the threshold value for getting precise prediction. Any distinction between the current prediction and the correct value is unobserved when this value less than the threshold value. The default value is 0.1. Experiment on this study set alpha value of 0.0001 and epsilon value of 0.1.

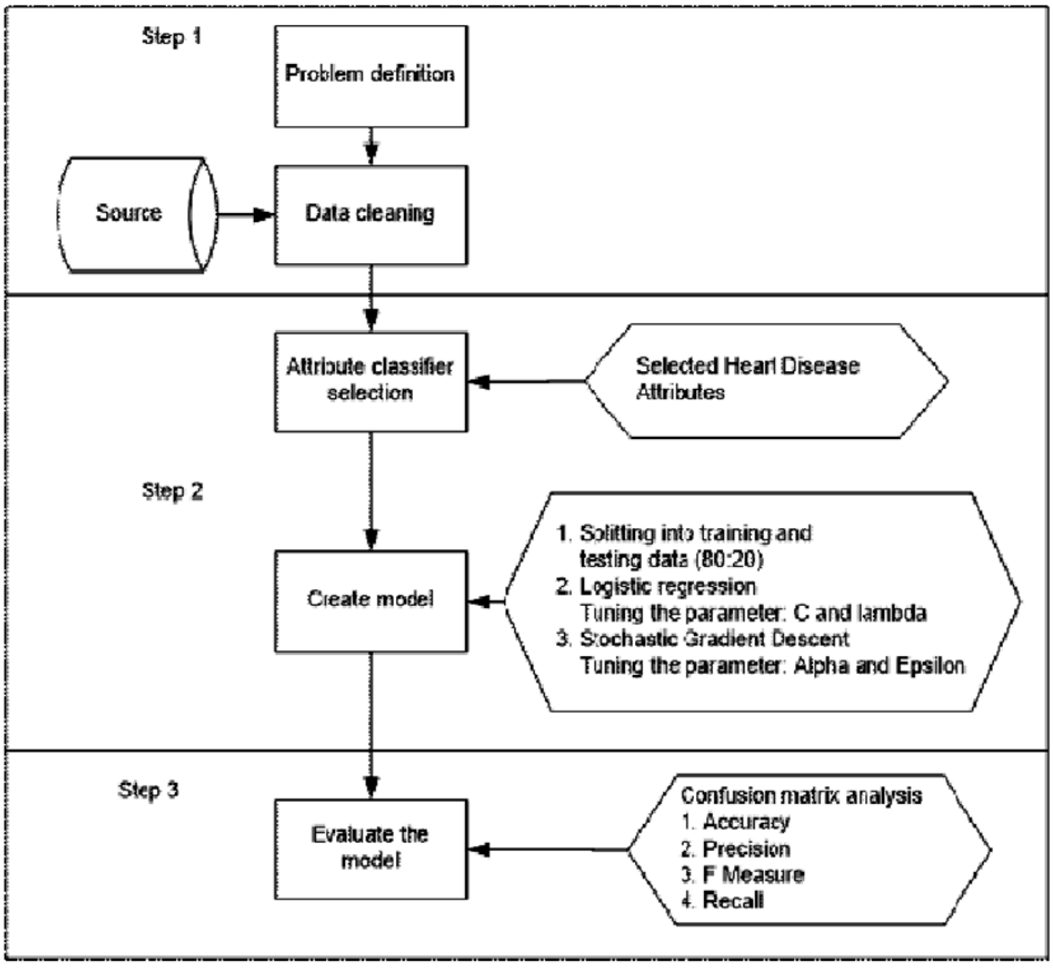
Q6. . Find confusion matrix using logistic regression with SGD.

Q7. . Find ACCURACY, PRECISION, F-MEASURE, RECALL: LOGISTIC REGRESSION CLASSIFIER with SGD.

Q8. Plot a graph Classifier result: Logistic regression and with Stochastic Gradient Descent for ACCURACY, PRECISION, F-MEASURE, RECALL.

Stochastic Gradient Descent





Flowchart of the Assignment

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

Q1. Load the cleaned kidney dataset(ckd)dataset and print its type and description

```
df = pd.read_excel('C:/Users/Asus/Desktop/DL Lab/DL Lab/Assignment
3/ckd_clean1.xlsx')
print(df.dtypes)
print(df.describe())
```

```
Age                             int64
Blood Pressure                  int64
Specific Gravity              float64
Albumin                         int64
Sugar                           int64
Red Blood Cells                object
Pus Cell                       object
Pus Cell clumps                object
Bacteria                       object
Blood Glucose Random            int64
Blood Urea                      int64
Serum Creatinine              float64
Sodium                          int64
Potassium                     float64
Hemoglobin                    float64
Packed Cell Volume              int64
White Blood Cell Count          int64
Red Blood Cell Count          float64
Hypertension                   object
Diabetes Mellitus              object
Coronary Artery Disease        object
Appetite                       object
Pedal Edema                    object
Anemia                         object
Class                           int64
dtype: object
              Age  Blood Pressure  Specific Gravity     Albumin
Sugar  \
count  158.000000      158.000000        158.000000  158.000000
158.000000
mean    49.563291       74.050633          1.019873    0.797468
0.253165
std     15.512244       11.175381          0.005499    1.413130
```

```
0.813397
min      6.000000       50.000000          1.005000    0.000000
0.000000
25%     39.250000       60.000000          1.020000    0.000000
0.000000
50%     50.500000       80.000000          1.020000    0.000000
0.000000
75%     60.000000       80.000000          1.025000    1.000000
0.000000
max     83.000000      110.000000          1.025000    4.000000
5.000000

       Blood Glucose Random  Blood Urea  Serum Creatinine      Sodium
\
count            158.000000  158.000000        158.000000  158.000000

mean             131.341772   52.575949          2.188608  138.848101

std               64.939832   47.395382          3.077615    7.489421

min               70.000000   10.000000          0.400000  111.000000

25%               97.000000   26.000000          0.700000  135.000000

50%              115.500000   39.500000          1.100000  139.000000

75%              131.750000   49.750000          1.600000  144.000000

max              490.000000  309.000000         15.200000  150.000000


        Potassium  Hemoglobin  Packed Cell Volume  White Blood Cell
Count  \
count  158.000000  158.000000          158.000000
158.000000
mean     4.636709   13.687342           41.917722
8475.949367
std      3.476351    2.882204            9.105164
3126.880181
min      2.500000    3.100000            9.000000
3800.000000
25%      3.700000   12.600000           37.500000
6525.000000
50%      4.500000   14.250000           44.000000
7800.000000
75%      4.900000   15.775000           48.000000
9775.000000
max     47.000000   17.800000           54.000000
26400.000000
```

```
       Red Blood Cell Count       Class
count            158.000000  158.000000
mean               4.891772    0.272152
std                1.019364    0.446483
min                2.100000    0.000000
25%                4.500000    0.000000
50%                4.950000    0.000000
75%                5.600000    1.000000
max                8.000000    1.000000
```

Q2. Prepare the data for Logistic Regression Take Logistic regression in the phyton library. Parameter tuning for optimizing classifier performance set for several parameter values namely: C =1.0, λ = 1.2 C .The default value is 1.0. The smaller C value specifies the stronger regularization value. The Lambda parameter used to control the regularization strength. The increase in the lambda values the strength of the regularization effect.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
log_reg = LogisticRegression(max_iter=1000)
param_grid = {'C': [0.01, 0.1, 1.0, 10], 'penalty': ['l2']}
grid_search = GridSearchCV(log_reg, param_grid, scoring='accuracy',
cv=5)
grid_search.fit(X_train, y_train)
best_params = grid_search.best_params_
print("Best parameters: ", best_params)
categorical_cols = ['Red Blood Cells', 'Pus Cell', 'Pus Cell clumps',
'Bacteria', 'Hypertension', 'Diabetes Mellitus',
                    'Coronary Artery Disease', 'Appetite', 'Pedal
Edema', 'Anemia']
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le
X = df.drop('Class', axis=1)
y = df['Class']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

```
Best parameters:  {'C': 0.1, 'penalty': 'l2'}
```

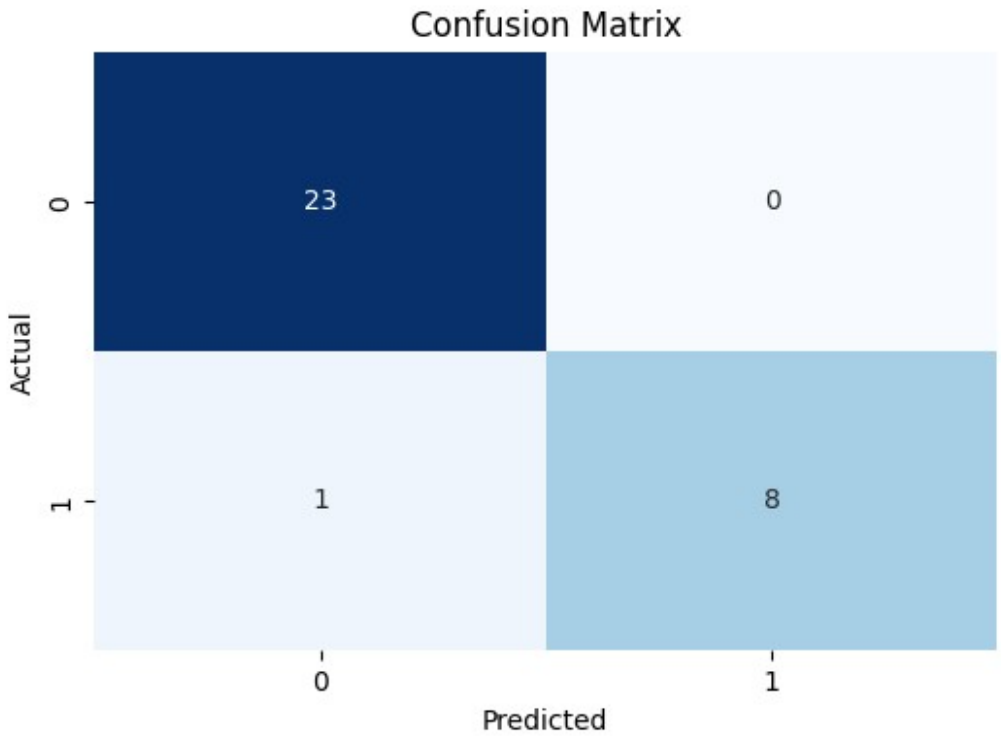
Logistic Regression without SGD

```
lr = LogisticRegression(max_iter=1000, C=1.0)
lr.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=1000)
```

Q3. Confusion Matrix

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
log_reg = LogisticRegression(C=best_params['C'], penalty='l2',
max_iter=1000)
log_reg.fit(X_train, y_train)
y_pred = log_reg.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



Q4. Accuracy, Precision, F-measure, Recall

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f_measure = f1_score(y_test, y_pred)
print("Without SGD - Accuracy: {}, Precision: {}, Recall: {}, F-
measure: {}".format(accuracy, precision, recall, f_measure))
```

```
Without SGD - Accuracy: 0.96875, Precision: 1.0, Recall:
0.8888888888888888, F-measure: 0.9411764705882353
```

Q5. Logistic Regression with SGD

```
from sklearn.linear_model import SGDClassifier
sgd_clf = SGDClassifier(loss='log', alpha=0.0001, epsilon=0.1,
max_iter=1000, tol=1e
sgd_clf.fit(X_train, y_train)
```

```
  Cell In[20], line 2
    sgd_clf = SGDClassifier(loss='log', alpha=0.0001, epsilon=0.1,
max_iter=1000, tol=1e

^
SyntaxError: invalid syntax
```

Q6. Confusion Matrix with SGD

```
y_pred_sgd = sgd_clf.predict(X_test)
cm_sgd = confusion_matrix(y_test, y_pred_sgd)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

```
---------------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
Cell In[19], line 1
----> 1 y_pred_sgd = sgd_clf.predict(X_test)
      2 cm_sgd = confusion_matrix(y_test, y_pred_sgd)
      3 plt.figure(figsize=(6, 4))

NameError: name 'sgd_clf' is not defined
```

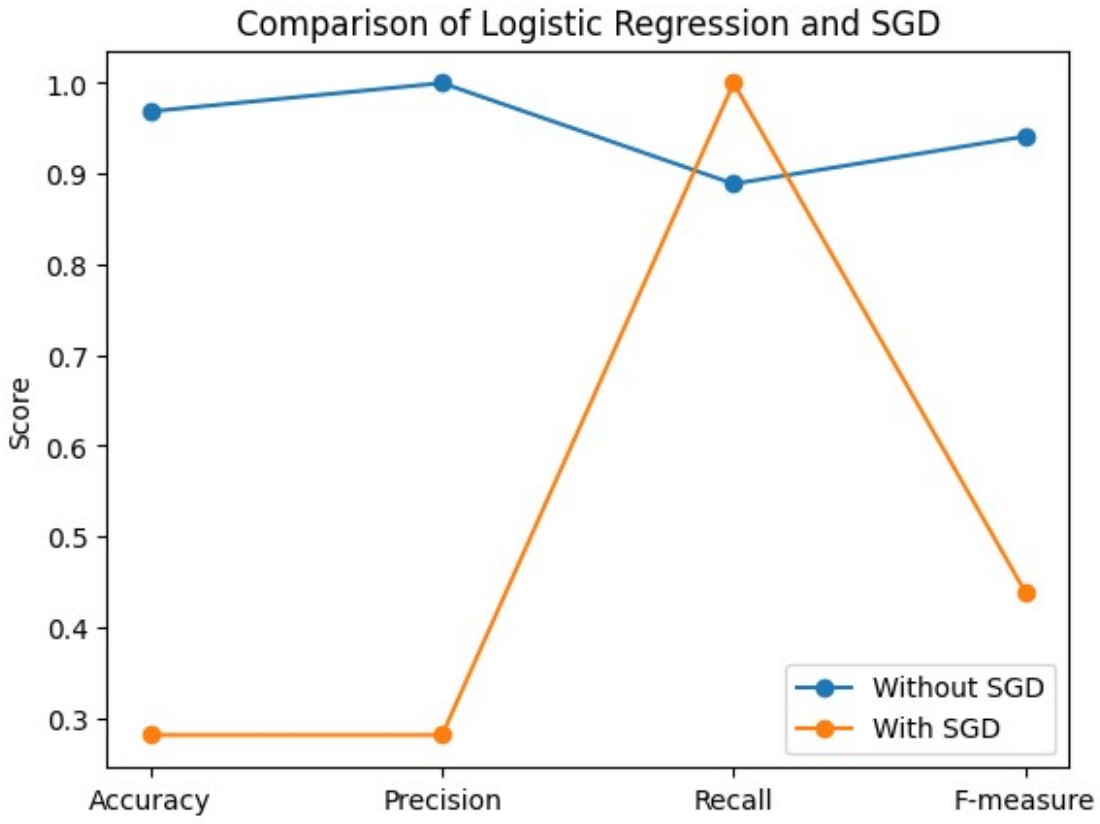
Q7. Accuracy, Precision, F-measure, Recall with SGD

```
accuracy_sgd = accuracy_score(y_test, y_pred_sgd)
precision_sgd = precision_score(y_test, y_pred_sgd)
recall_sgd = recall_score(y_test, y_pred_sgd)
f_measure_sgd = f1_score(y_test, y_pred_sgd)
print("With SGD - Accuracy: {}, Precision: {}, Recall: {}, F-measure:
{}".format(accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd))
```

```
With SGD - Accuracy: 0.28125, Precision: 0.28125, Recall: 1.0, F-measure: 0.43902439024390244
```

Q8. Plotting the results

```
labels = ['Accuracy', 'Precision', 'Recall', 'F-measure']
without_sgd = [accuracy, precision, recall, f_measure]
with_sgd = [accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd]
x = range(len(labels))
plt.plot(x, without_sgd, label='Without SGD', marker='o')
plt.plot(x, with_sgd, label='With SGD', marker='o')
plt.xticks(x, labels)
plt.ylabel('Score')
plt.title('Comparison of Logistic Regression and SGD')
plt.legend()
plt.show()
```



Vishal Marandi |

BTECH/10119/21 |

Deep Learning Assignment - 3

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

Q1. Load the dataset and print its type and description

```
df = pd.read_csv('/content/Thyroid.txt')
print(df.dtypes)
print(df.describe())
```

```
on_thyroxine                  int64
query_on_thyroxine            int64
on_antithyroid_medication     int64
thyroid_surgery               int64
query_hypothyroid             int64
query_hyperthyroid            int64
pregnant                      int64
sick                          int64
tumor                         int64
lithium                       int64
goitre                        int64
TSH_measured                  int64
T3_measured                   int64
TT4_measured                  int64
T4U_measured                  int64
FTI_measured                  int64
age                           int64
sex                           int64
TSH                         float64
T3                          float64
TT4                         float64
T4U                         float64
FTI                         float64
classes                       int64
```

```
dtype: object
       on_thyroxine  query_on_thyroxine  on_antithyroid_medication  \
count   3152.000000         3152.000000                3152.000000
mean       0.145305            0.017449                   0.013325
std        0.352464            0.130959                   0.114680
min        0.000000            0.000000                   0.000000
25%        0.000000            0.000000                   0.000000
50%        0.000000            0.000000                   0.000000
75%        0.000000            0.000000                   0.000000
max        1.000000            1.000000                   1.000000

       thyroid_surgery  query_hypothyroid  query_hyperthyroid
pregnant  \
count      3152.000000        3152.000000         3152.000000
3152.000000
mean          0.032995           0.075825            0.077094
0.019987
std           0.178652           0.264760            0.266783
0.139979
min           0.000000           0.000000            0.000000
0.000000
25%           0.000000           0.000000            0.000000
0.000000
50%           0.000000           0.000000            0.000000
0.000000
75%           0.000000           0.000000            0.000000
0.000000
max           1.000000           1.000000            1.000000
1.000000

              sick        tumor      lithium  ...  T4U_measured
FTI_measured  \
count  3152.000000  3152.000000  3152.000000  ...   3152.000000
3152.000000
mean      0.030774     0.012690     0.000635  ...      0.921320
0.921637
std       0.172733     0.111952     0.025186  ...      0.269282
0.268785
min       0.000000     0.000000     0.000000  ...      0.000000
0.000000
25%       0.000000     0.000000     0.000000  ...      1.000000
1.000000
50%       0.000000     0.000000     0.000000  ...      1.000000
1.000000
75%       0.000000     0.000000     0.000000  ...      1.000000
1.000000
max       1.000000     1.000000     1.000000  ...      1.000000
1.000000

               age          sex          TSH           T3          TT4
```

```
\
count  3152.000000  3152.000000  3152.000000  3152.000000  3152.000000

mean     51.359772     0.295051     5.689591     1.992291   109.016783

std      19.226176     0.456138    22.953469     1.036009    45.085350

min       1.000000     0.000000     0.000000     0.000000     2.000000

25%      35.000000     0.000000     0.000000     1.400000    83.000000

50%      54.000000     0.000000     0.700000     1.900000   104.000000

75%      67.000000     1.000000     2.200000     2.400000   128.000000

max      98.000000     1.000000   530.000000    10.200000   450.000000


                T4U          FTI      classes
count  3152.000000  3152.000000  3152.000000
mean      0.983284   114.809613     0.091371
std       0.234424    59.028114     0.288181
min       0.000000     0.000000     0.000000
25%       0.850000    90.000000     0.000000
50%       0.960000   107.000000     0.000000
75%       1.070000   128.000000     0.000000
max       2.210000   881.000000     1.000000

[8 rows x 24 columns]
```

Q2. Prepare the data for Logistic Regression

```
X = df.drop('classes', axis=1)
y = df['classes']
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Logistic Regression without SGD

```
lr = LogisticRegression(max_iter=1000, C=1.0)
lr.fit(X_train, y_train)

LogisticRegression(max_iter=1000)
```

Q3. Confusion Matrix

```
y_pred = lr.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix (without SGD):\n", cm)
```

```
Confusion Matrix (without SGD):
 [[552  10]
 [ 23  46]]
```

Q4. Accuracy, Precision, F-measure, Recall

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f_measure = f1_score(y_test, y_pred)
print("Without SGD - Accuracy: {}, Precision: {}, Recall: {}, F-
measure: {}".format(accuracy, precision, recall, f_measure))
```

```
Without SGD - Accuracy: 0.9477020602218701, Precision:
0.8214285714285714, Recall: 0.6666666666666666, F-measure:
0.7359999999999999
```

Q5. Logistic Regression with SGD

```
sgd = SGDClassifier(loss='log_loss', alpha=0.0001, max_iter=1000,
tol=1e-3, random_state=42)
sgd.fit(X_train, y_train)
```

```
SGDClassifier(loss='log_loss', random_state=42)
```

Q6. Confusion Matrix with SGD

```
y_pred_sgd = sgd.predict(X_test)
cm_sgd = confusion_matrix(y_test, y_pred_sgd)
print("Confusion Matrix (with SGD):\n", cm_sgd)
```

```
Confusion Matrix (with SGD):
 [[562   0]
 [ 69   0]]
```

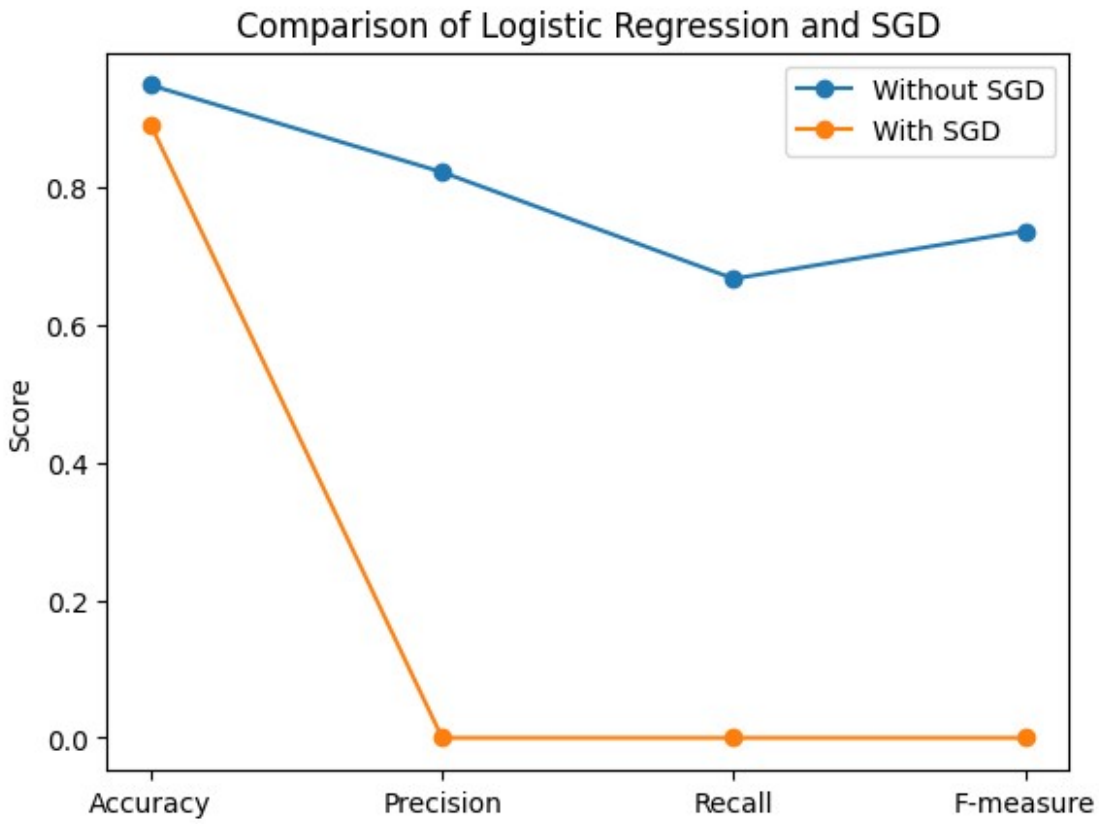
Q7. Accuracy, Precision, F-measure, Recall with SGD

```
accuracy_sgd = accuracy_score(y_test, y_pred_sgd)
precision_sgd = precision_score(y_test, y_pred_sgd)
recall_sgd = recall_score(y_test, y_pred_sgd)
f_measure_sgd = f1_score(y_test, y_pred_sgd)
print("With SGD - Accuracy: {}, Precision: {}, Recall: {}, F-measure: {}".format(accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd))
```

```
With SGD - Accuracy: 0.8906497622820919, Precision: 0.0, Recall: 0.0, F-measure: 0.0

/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1471: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

Q8. Plotting the results

```
labels = ['Accuracy', 'Precision', 'Recall', 'F-measure']
without_sgd = [accuracy, precision, recall, f_measure]
with_sgd = [accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd]
x = range(len(labels))
plt.plot(x, without_sgd, label='Without SGD', marker='o')
plt.plot(x, with_sgd, label='With SGD', marker='o')
plt.xticks(x, labels)
plt.ylabel('Score')
plt.title('Comparison of Logistic Regression and SGD')
plt.legend()
plt.show()
```


Comparison of Logistic Regression and SGD
Without SGD
With SGD
Score
0.0
0.2
0.4
0.6
0.8
Accuracy
Precision
Recall
F-measure

Vishal Marandi |

BTECH/10119/21 |

Deep Learning Assignment - 3

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

Q1. Load the dataset and print its type and description

```
df = pd.read_csv('/content/heart.csv', header=None)
df = df[0].str.split(expand=True)
df.columns = range(1, df.shape[1] + 1)
df = df.apply(pd.to_numeric, errors='ignore')
print(df.head())
print(df.dtypes)
print(df.describe())
```

```
     1    2    3      4      5    6    7      8    9    10   11   12
13  14
0  70.0  1.0  4.0  130.0  322.0  0.0  2.0  109.0  0.0  2.4  2.0  3.0
3.0   2
1  67.0  0.0  3.0  115.0  564.0  0.0  2.0  160.0  0.0  1.6  2.0  0.0
7.0   1
2  57.0  1.0  2.0  124.0  261.0  0.0  0.0  141.0  0.0  0.3  1.0  0.0
7.0   2
3  64.0  1.0  4.0  128.0  263.0  0.0  0.0  105.0  1.0  0.2  2.0  1.0
7.0   1
4  74.0  0.0  2.0  120.0  269.0  0.0  2.0  121.0  1.0  0.2  1.0  1.0
3.0   1
1     float64
2     float64
3     float64
4     float64
5     float64
6     float64
7     float64
8     float64
```

```
9     float64
10    float64
11    float64
12    float64
13    float64
14      int64
dtype: object
                1           2           3           4           5
6   \
count  270.000000  270.000000  270.000000  270.000000  270.000000
270.000000
mean    54.433333    0.677778    3.174074  131.344444  249.659259
0.148148
std      9.109067    0.468195    0.950090   17.861608   51.686237
0.355906
min     29.000000    0.000000    1.000000   94.000000  126.000000
0.000000
25%     48.000000    0.000000    3.000000  120.000000  213.000000
0.000000
50%     55.000000    1.000000    3.000000  130.000000  245.000000
0.000000
75%     61.000000    1.000000    4.000000  140.000000  280.000000
0.000000
max     77.000000    1.000000    4.000000  200.000000  564.000000
1.000000

                7           8           9          10          11
12  \
count  270.000000  270.000000  270.000000  270.00000  270.000000
270.000000
mean     1.022222  149.677778    0.329630    1.05000    1.585185
0.670370
std      0.997891   23.165717    0.470952    1.14521    0.614390
0.943896
min      0.000000   71.000000    0.000000    0.00000    1.000000
0.000000
25%      0.000000  133.000000    0.000000    0.00000    1.000000
0.000000
50%      2.000000  153.500000    0.000000    0.80000    2.000000
0.000000
75%      2.000000  166.000000    1.000000    1.60000    2.000000
1.000000
max      2.000000  202.000000    1.000000    6.20000    3.000000
3.000000

                13          14
count  270.000000  270.000000
mean     4.696296    1.444444
std      1.940659    0.497827
```

```
min      3.000000    1.000000
25%      3.000000    1.000000
50%      3.000000    1.000000
75%      7.000000    2.000000
max      7.000000    2.000000
```

Q2. Prepare the data for Logistic Regression

```
X = df.iloc[:, :-1]  # All columns except the last one
y = df.iloc[:, -1]   # The last column as target
# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Logistic Regression without SGD

```
lr = LogisticRegression(max_iter=1000, C=1.0)
lr.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=1000)
```

Q3. Confusion Matrix

```
y_pred = lr.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix (without SGD):\n", cm)
```

```
Confusion Matrix (without SGD):
 [[32  1]
 [ 3 18]]
```

Q4. Accuracy, Precision, F-measure, Recall

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f_measure = f1_score(y_test, y_pred)
print("Without SGD - Accuracy: {}, Precision: {}, Recall: {}, F-
measure: {}".format(accuracy, precision, recall, f_measure))
```

```
Without SGD - Accuracy: 0.9259259259259259, Precision:
0.9142857142857143, Recall: 0.9696969696969697, F-measure:
0.9411764705882354
```

Q5. Logistic Regression with SGD

```
sgd = SGDClassifier(loss='log_loss', alpha=0.0001, max_iter=1000,
tol=1e-3, random_state=42)
sgd.fit(X_train, y_train)
```

```
SGDClassifier(loss='log_loss', random_state=42)
```

Q6. Confusion Matrix with SGD

```
y_pred_sgd = sgd.predict(X_test)
cm_sgd = confusion_matrix(y_test, y_pred_sgd)
print("Confusion Matrix (with SGD):\n", cm_sgd)
```

```
Confusion Matrix (with SGD):
 [[28  5]
 [ 5 16]]
```

Q7. Accuracy, Precision, F-measure, Recall with SGD

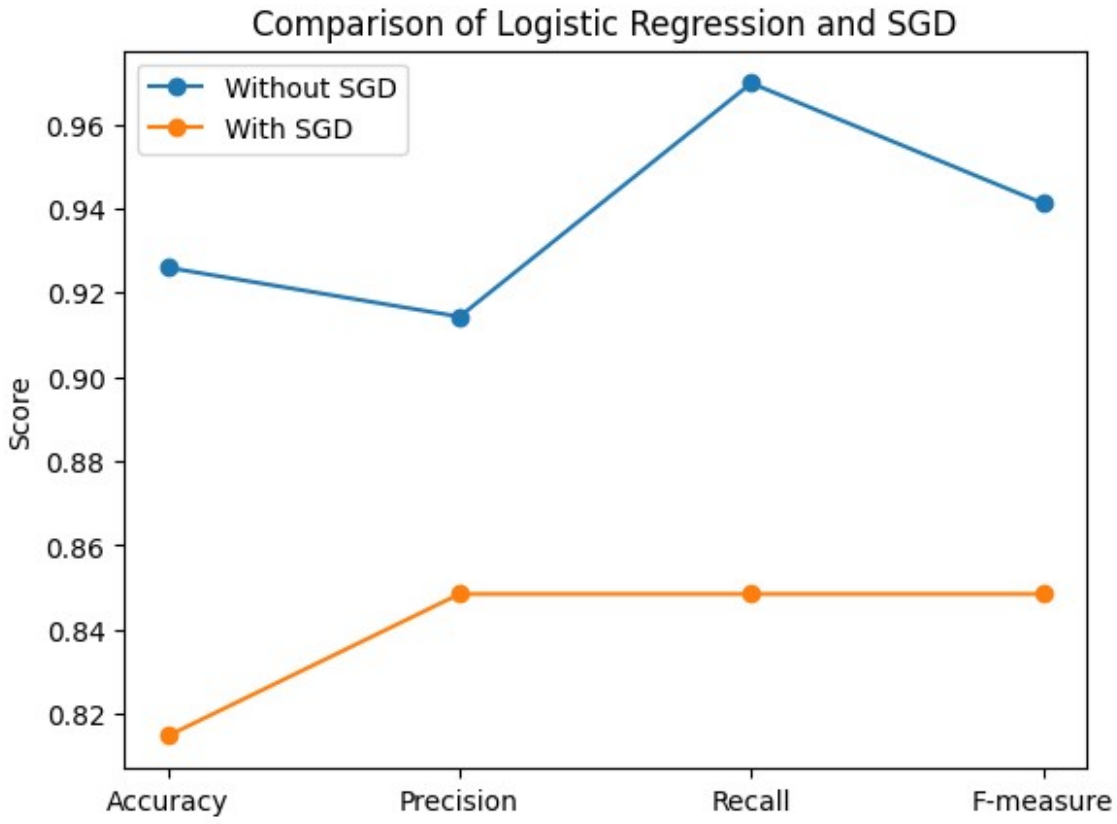
```
accuracy_sgd = accuracy_score(y_test, y_pred_sgd)
precision_sgd = precision_score(y_test, y_pred_sgd)
recall_sgd = recall_score(y_test, y_pred_sgd)
f_measure_sgd = f1_score(y_test, y_pred_sgd)
print("With SGD - Accuracy: {}, Precision: {}, Recall: {}, F-measure:
{}".format(accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd))
```

```
With SGD - Accuracy: 0.8148148148148148, Precision:
0.8484848484848485, Recall: 0.8484848484848485, F-measure:
0.8484848484848486
```

Q8. Plotting the results

```
labels = ['Accuracy', 'Precision', 'Recall', 'F-measure']
without_sgd = [accuracy, precision, recall, f_measure]
with_sgd = [accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd]
x = range(len(labels))
plt.plot(x, without_sgd, label='Without SGD', marker='o')
plt.plot(x, with_sgd, label='With SGD', marker='o')
plt.xticks(x, labels)
plt.ylabel('Score')
plt.title('Comparison of Logistic Regression and SGD')
```

```
plt.legend()
plt.show()
```



Assignment 4

Q1. Load load_breast_cancer() built in dataset.

Q2. convert dataset to pandas dataframe.

Q3. Append dataframe containing tumor features with diagnostic outcomes. These labels will be used for supervised learning.

Q4.Find relation in diagnosis by using mean.

Q5. create dataframes - one for positive, one for negative(diagnosis).

Q6. Visualize tumor characteristics for positive and negatives diagnoses

Q7. Create 'for loop' to iterate through tumor features and compare with histograms(20:-1)

Q8. Find Visualization of relationships between features and diagnoses.

Q9. Split data into testing and training sets. Use 80% for training

Q10. normalize features to account for feature scaling

Q11. Apply DT, NB, KNN Logistic Regression and SVm to find accuracy, F1-Score and Recall.

Q12. Further apply the PCA to do the same exercise in Q11.

Q13. Do the Comparative study with PCA and WITHout PCA.

Q1. Load load_breast_cancer() built in dataset.

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
breast_cancer_data = load_breast_cancer()
print(breast_cancer_data)
```

```
{'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01,
4.601e-01,
        1.189e-01],
       [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
        8.902e-02],
       [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
        8.758e-02],
       ...,
       [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
        7.820e-02],
       [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
        1.240e-01],
       [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
        7.039e-02]]), 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0,
0,
       1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0,
0,
       1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0,
1,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1,
0,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
1,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1,
1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0,
0,
       0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0,
0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1,
1,
       1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1,
1,
```

```
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0,
0,
       0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0,
0,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1,
1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1,
0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,
0,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1,
1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1]),
'frame': None, 'target_names': array(['malignant', 'benign'],
dtype='<U9'), 'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer
wisconsin (diagnostic) dataset\
n--------------------------------------------\n\n**Data Set
Characteristics:**\n\n:Number of Instances: 569\n\n:Number of
Attributes: 30 numeric, predictive attributes and the class\n\
n:Attribute Information:\n    - radius (mean of distances from center
to points on the perimeter)\n    - texture (standard deviation of
gray-scale values)\n    - perimeter\n    - area\n    - smoothness
(local variation in radius lengths)\n    - compactness (perimeter^2 /
area - 1.0)\n    - concavity (severity of concave portions of the
contour)\n    - concave points (number of concave portions of the
contour)\n    - symmetry\n    - fractal dimension ("coastline
approximation" - 1)\n\n    The mean, standard error, and "worst" or
largest (mean of the three\n    worst/largest values) of these
features were computed for each image,\n    resulting in 30 features.
For instance, field 0 is Mean Radius, field\n    10 is Radius SE,
field 20 is Worst Radius.\n\n    - class:\n            - WDBC-
Malignant\n            - WDBC-Benign\n\n:Summary Statistics:\n\
n===================================== ====== ======\n
Min    Max\n===================================== ====== ======\
nradius (mean):                        6.981  28.11\ntexture (mean):
9.71   39.28\nperimeter (mean):                      43.79  188.5\narea
(mean):                        143.5  2501.0\nsmoothness (mean):
0.053  0.163\ncompactness (mean):                0.019  0.345\
nconcavity (mean):                     0.0    0.427\nconcave points
```

(mean):   0.0   0.201\nsymmetry (mean):
0.106 0.304\nfractal dimension (mean):   0.05   0.097\
nradius (standard error):   0.112 2.873\ntexture (standard
error):   0.36   4.885\nperimeter (standard error):
0.757 21.98\narea (standard error):   6.802 542.2\
nsmoothness (standard error):   0.002 0.031\ncompactness
(standard error):   0.002 0.135\nconcavity (standard error):
0.0   0.396\nconcave points (standard error):   0.0   0.053\
nsymmetry (standard error):   0.008 0.079\nfractal dimension
(standard error):   0.001 0.03\nradius (worst):
7.93   36.04\ntexture (worst):   12.02 49.54\
nperimeter (worst):   50.41 251.2\narea (worst):
185.2 4254.0\nsmoothness (worst):   0.071 0.223\
ncompactness (worst):   0.027 1.058\nconcavity
(worst):   0.0   1.252\nconcave points (worst):
0.0   0.291\nsymmetry (worst):   0.156 0.664\
nfractal dimension (worst):   0.055 0.208\
n================================== ====== ======\n\n:Missing
Attribute Values: None\n\n:Class Distribution: 212 - Malignant, 357 -
Benign\n\n:Creator: Dr. William H. Wolberg, W. Nick Street, Olvi L.
Mangasarian\n\n:Donor: Nick Street\n\n:Date: November, 1995\n\nThis is
a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) datasets.\
nhttps://goo.gl/U2Uwz2\n\nFeatures are computed from a digitized image
of a fine needle\naspirate (FNA) of a breast mass. They describe\
ncharacteristics of the cell nuclei present in the image.\n\
nSeparating plane described above was obtained using\nMultisurface
Method-Tree (MSM-T) [K. P. Bennett, "Decision Tree\nConstruction Via
Linear Programming." Proceedings of the 4th\nMidwest Artificial
Intelligence and Cognitive Science Society,\npp. 97-101, 1992], a
classification method which uses linear\nprogramming to construct a
decision tree. Relevant features\nwere selected using an exhaustive
search in the space of 1-4\nfeatures and 1-3 separating planes.\n\nThe
actual linear program used to obtain the separating plane\nin the 3-
dimensional space is that described in:\n[K. P. Bennett and O. L.
Mangasarian: "Robust Linear\nProgramming Discrimination of Two
Linearly Inseparable Sets",\nOptimization Methods and Software 1,
1992, 23-34].\n\nThis database is also available through the UW CS ftp
server:\n\nftp ftp.cs.wisc.edu\ncd math-prog/cpo-dataset/machine-
learn/WDBC/\n\n.. dropdown:: References\n\n - W.N. Street, W.H.
Wolberg and O.L. Mangasarian. Nuclear feature extraction\n   for
breast tumor diagnosis. IS&T/SPIE 1993 International Symposium on\n
Electronic Imaging: Science and Technology, volume 1905, pages 861-
870,\n   San Jose, CA, 1993.\n - O.L. Mangasarian, W.N. Street and
W.H. Wolberg. Breast cancer diagnosis and\n   prognosis via linear
programming. Operations Research, 43(4), pages 570-577,\n   July-
August 1995.\n - W.H. Wolberg, W.N. Street, and O.L. Mangasarian.
Machine learning techniques\n   to diagnose breast cancer from fine-
needle aspirates. Cancer Letters 77 (1994)\n   163-171.\n',
'feature_names': array(['mean radius', 'mean texture', 'mean

```
perimeter', 'mean area',
       'mean smoothness', 'mean compactness', 'mean concavity',
       'mean concave points', 'mean symmetry', 'mean fractal
dimension',
       'radius error', 'texture error', 'perimeter error', 'area
error',
       'smoothness error', 'compactness error', 'concavity error',
       'concave points error', 'symmetry error',
       'fractal dimension error', 'worst radius', 'worst texture',
       'worst perimeter', 'worst area', 'worst smoothness',
       'worst compactness', 'worst concavity', 'worst concave points',
       'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
'filename': 'breast_cancer.csv', 'data_module':
'sklearn.datasets.data'}
```

```
X = breast_cancer_data.data
y = breast_cancer_data.target
feature_names = breast_cancer_data.feature_names
print(feature_names)
```

```
['mean radius' 'mean texture' 'mean perimeter' 'mean area'
 'mean smoothness' 'mean compactness' 'mean concavity'
 'mean concave points' 'mean symmetry' 'mean fractal dimension'
 'radius error' 'texture error' 'perimeter error' 'area error'
 'smoothness error' 'compactness error' 'concavity error'
 'concave points error' 'symmetry error' 'fractal dimension error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst smoothness' 'worst compactness' 'worst concavity'
 'worst concave points' 'worst symmetry' 'worst fractal dimension']
```

Q2. convert dataset to pandas dataframe.

```
df_features = pd.DataFrame(X, columns=feature_names)
print(df_features.head())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean
smoothness  \
0        17.99         10.38          122.80     1001.0
0.11840
1        20.57         17.77          132.90     1326.0
0.08474
2        19.69         21.25          130.00     1203.0
0.10960
3        11.42         20.38           77.58      386.1
0.14250
4        20.29         14.34          135.10     1297.0
0.10030

   mean compactness  mean concavity  mean concave points  mean
symmetry  \
```

```
0           0.27760          0.3001              0.14710
0.2419
1           0.07864          0.0869              0.07017
0.1812
2           0.15990          0.1974              0.12790
0.2069
3           0.28390          0.2414              0.10520
0.2597
4           0.13280          0.1980              0.10430
0.1809

   mean fractal dimension  ...  worst radius  worst texture  worst
perimeter  \
0                 0.07871  ...         25.38          17.33
184.60
1                 0.05667  ...         24.99          23.41
158.80
2                 0.05999  ...         23.57          25.53
152.50
3                 0.09744  ...         14.91          26.50
98.87
4                 0.05883  ...         22.54          16.67
152.20

   worst area  worst smoothness  worst compactness  worst concavity  \
0      2019.0            0.1622             0.6656           0.7119
1      1956.0            0.1238             0.1866           0.2416
2      1709.0            0.1444             0.4245           0.4504
3       567.7            0.2098             0.8663           0.6869
4      1575.0            0.1374             0.2050           0.4000

   worst concave points  worst symmetry  worst fractal dimension
0                0.2654          0.4601                  0.11890
1                0.1860          0.2750                  0.08902
2                0.2430          0.3613                  0.08758
3                0.2575          0.6638                  0.17300
4                0.1625          0.2364                  0.07678

[5 rows x 30 columns]
```

Q3. Append dataframe containing tumor features with diagnostic outcomes.

```
df_target = pd.Series(y, name='diagnosis')
df = pd.concat([df_features, df_target], axis=1)
print(df.head())

   mean radius  mean texture  mean perimeter  mean area  mean
smoothness  \
0        17.99         10.38          122.80     1001.0
0.11840
```

```
1        20.57         17.77           132.90     1326.0
0.08474
2        19.69         21.25           130.00     1203.0
0.10960
3        11.42         20.38            77.58      386.1
0.14250
4        20.29         14.34           135.10     1297.0
0.10030

   mean compactness  mean concavity  mean concave points  mean
symmetry  \
0           0.27760          0.3001              0.14710
0.2419
1           0.07864          0.0869              0.07017
0.1812
2           0.15990          0.1974              0.12790
0.2069
3           0.28390          0.2414              0.10520
0.2597
4           0.13280          0.1980              0.10430
0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst
area  \
0                 0.07871  ...          17.33           184.60
2019.0
1                 0.05667  ...          23.41           158.80
1956.0
2                 0.05999  ...          25.53           152.50
1709.0
3                 0.09744  ...          26.50            98.87
567.7
4                 0.05883  ...          16.67           152.20
1575.0

   worst smoothness  worst compactness  worst concavity  worst concave
points  \
0            0.1622             0.6656           0.7119
0.2654
1            0.1238             0.1866           0.2416
0.1860
2            0.1444             0.4245           0.4504
0.2430
3            0.2098             0.8663           0.6869
0.2575
4            0.1374             0.2050           0.4000
0.1625

   worst symmetry  worst fractal dimension  diagnosis
0          0.4601                  0.11890          0
```

```
1          0.2750                  0.08902          0
2          0.3613                  0.08758          0
3          0.6638                  0.17300          0
4          0.2364                  0.07678          0

[5 rows x 31 columns]
```

Q4.Find relation in diagnosis by using mean.

```
mean_features = df.groupby('diagnosis').mean()
print(mean_features)
```

```
           mean radius  mean texture  mean perimeter   mean area  \
diagnosis
0            17.462830     21.604906      115.365377  978.376415
1            12.146524     17.914762       78.075406  462.790196

           mean smoothness  mean compactness  mean concavity  \
diagnosis
0                 0.102898          0.145188        0.160775
1                 0.092478          0.080085        0.046058

           mean concave points  mean symmetry  mean fractal dimension
...  \
diagnosis
...
0                     0.087990       0.192909                 0.062680
...
1                     0.025717       0.174186                 0.062867
...

           worst radius  worst texture  worst perimeter   worst
area  \
diagnosis

0             21.134811      29.318208       141.370330  1422.286321

1             13.379801      23.515070        87.005938   558.899440


           worst smoothness  worst compactness  worst concavity  \
diagnosis
0                  0.144845           0.374824         0.450606
1                  0.124959           0.182673         0.166238

           worst concave points  worst symmetry  worst fractal
dimension
diagnosis

0                     0.182237        0.323468
```

```
0.091530
1                        0.074444         0.270246
0.079442

[2 rows x 30 columns]
```

Q5. create to dataframes - one for positive, one for negative(diagnosis).

```
df_positive = df[df['diagnosis'] == 1]
df_negative = df[df['diagnosis'] == 0]
print("Positive diagnosis (Malignant):")
print(df_positive.head())
print("\nNegative diagnosis (Benign):")
print(df_negative.head())
```

```
Positive diagnosis (Malignant):
    mean radius  mean texture  mean perimeter  mean area  mean
smoothness  \
19       13.540         14.36           87.46      566.3
0.09779
20       13.080         15.71           85.63      520.0
0.10750
21        9.504         12.44           60.34      273.9
0.10240
37       13.030         18.42           82.61      523.8
0.08983
46        8.196         16.84           51.71      201.9
0.08600

    mean compactness  mean concavity  mean concave points  mean
symmetry  \
19           0.08129         0.06664              0.047810
0.1885
20           0.12700         0.04568              0.031100
0.1967
21           0.06492         0.02956              0.020760
0.1815
37           0.03766         0.02562              0.029230
0.1467
46           0.05943         0.01588              0.005917
0.1769

    mean fractal dimension  ...  worst texture  worst perimeter  worst
area  \
19                 0.05766  ...          19.26            99.70
711.2
20                 0.06811  ...          20.49            96.09
630.5
21                 0.06905  ...          15.66            65.13
```

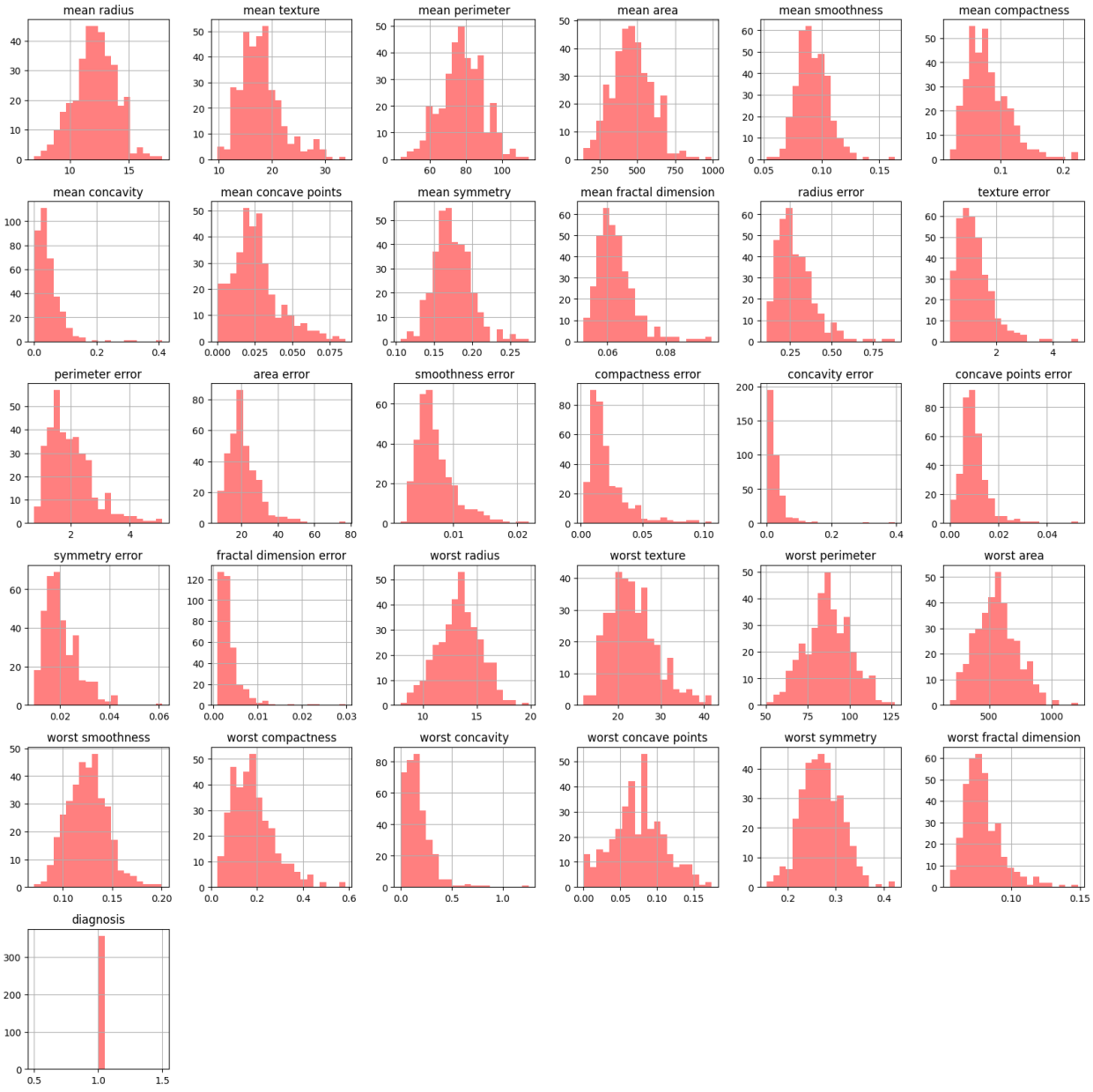
```
314.9
37                 0.05863  ...          22.81            84.46
545.9
46                 0.06503  ...          21.96            57.26
242.2

    worst smoothness  worst compactness  worst concavity  \
19           0.14400            0.17730          0.23900
20           0.13120            0.27760          0.18900
21           0.13240            0.11480          0.08867
37           0.09701            0.04619          0.04833
46           0.12970            0.13570          0.06880

    worst concave points  worst symmetry  worst fractal dimension
diagnosis
19               0.12880          0.2977                  0.07259
1
20               0.07283          0.3184                  0.08183
1
21               0.06227          0.2450                  0.07773
1
37               0.05013          0.1987                  0.06169
1
46               0.02564          0.3105                  0.07409
1

[5 rows x 31 columns]

Negative diagnosis (Benign):
   mean radius  mean texture  mean perimeter  mean area  mean
smoothness  \
0        17.99         10.38          122.80     1001.0
0.11840
1        20.57         17.77          132.90     1326.0
0.08474
2        19.69         21.25          130.00     1203.0
0.10960
3        11.42         20.38           77.58      386.1
0.14250
4        20.29         14.34          135.10     1297.0
0.10030

   mean compactness  mean concavity  mean concave points  mean
symmetry  \
0           0.27760          0.3001              0.14710
0.2419
1           0.07864          0.0869              0.07017
0.1812
2           0.15990          0.1974              0.12790
0.2069
```

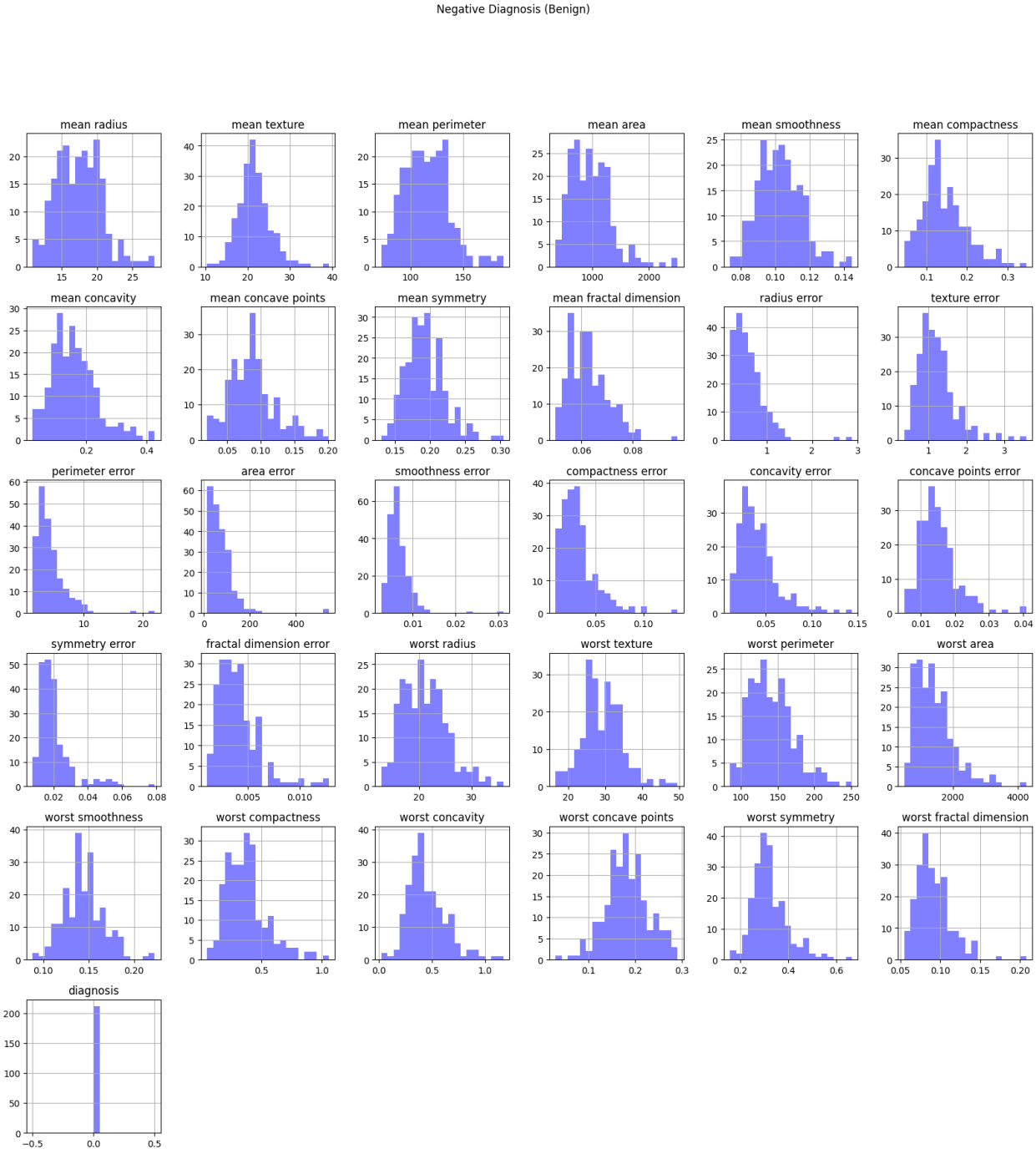
```
3           0.28390          0.2414               0.10520
0.2597
4           0.13280          0.1980               0.10430
0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst
area  \
0                 0.07871  ...          17.33           184.60
2019.0
1                 0.05667  ...          23.41           158.80
1956.0
2                 0.05999  ...          25.53           152.50
1709.0
3                 0.09744  ...          26.50            98.87
567.7
4                 0.05883  ...          16.67           152.20
1575.0

   worst smoothness  worst compactness  worst concavity  worst concave
points  \
0            0.1622             0.6656           0.7119
0.2654
1            0.1238             0.1866           0.2416
0.1860
2            0.1444             0.4245           0.4504
0.2430
3            0.2098             0.8663           0.6869
0.2575
4            0.1374             0.2050           0.4000
0.1625

   worst symmetry  worst fractal dimension  diagnosis
0          0.4601                  0.11890          0
1          0.2750                  0.08902          0
2          0.3613                  0.08758          0
3          0.6638                  0.17300          0
4          0.2364                  0.07678          0

[5 rows x 31 columns]
```

Q6. Visualize tumor characteristics for positive and negatives diagnoses

```
import matplotlib.pyplot as plt
df_positive.hist(figsize=(20, 20), bins=20, color='red', alpha=0.5)
plt.suptitle('Positive Diagnosis (Malignant)')
plt.show()
df_negative.hist(figsize=(20, 20), bins=20, color='blue', alpha=0.5)
plt.suptitle('Negative Diagnosis (Benign)')
plt.show()
```

Positive Diagnosis (Malignant)

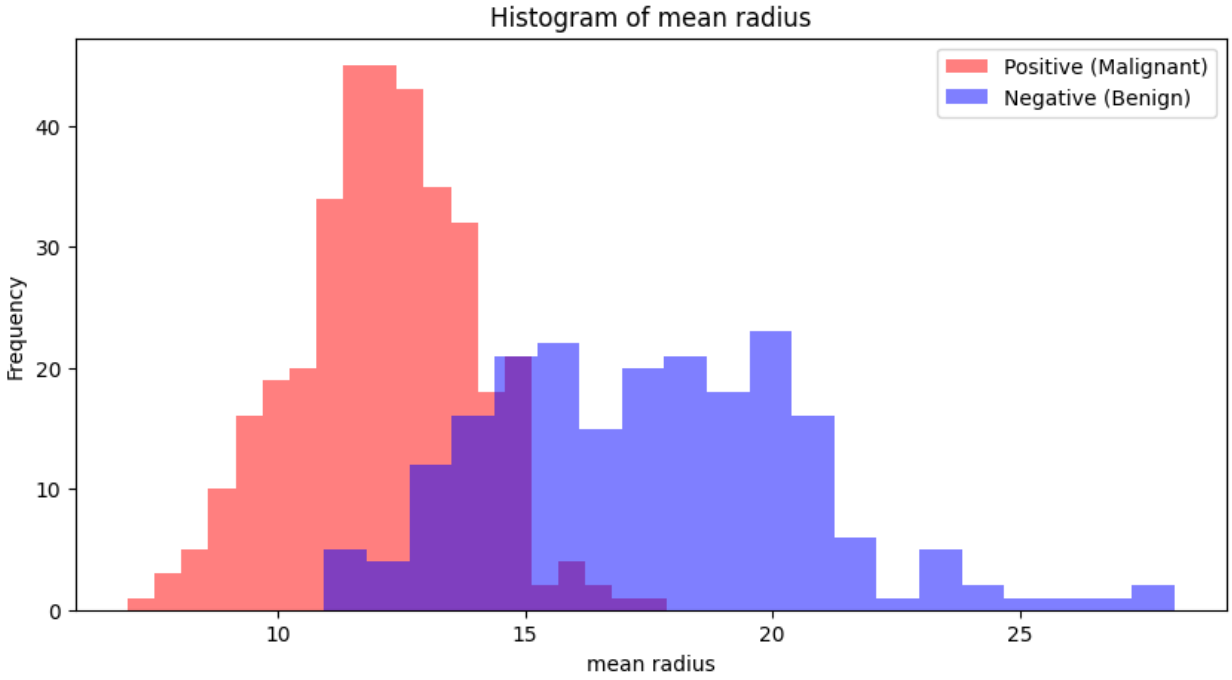


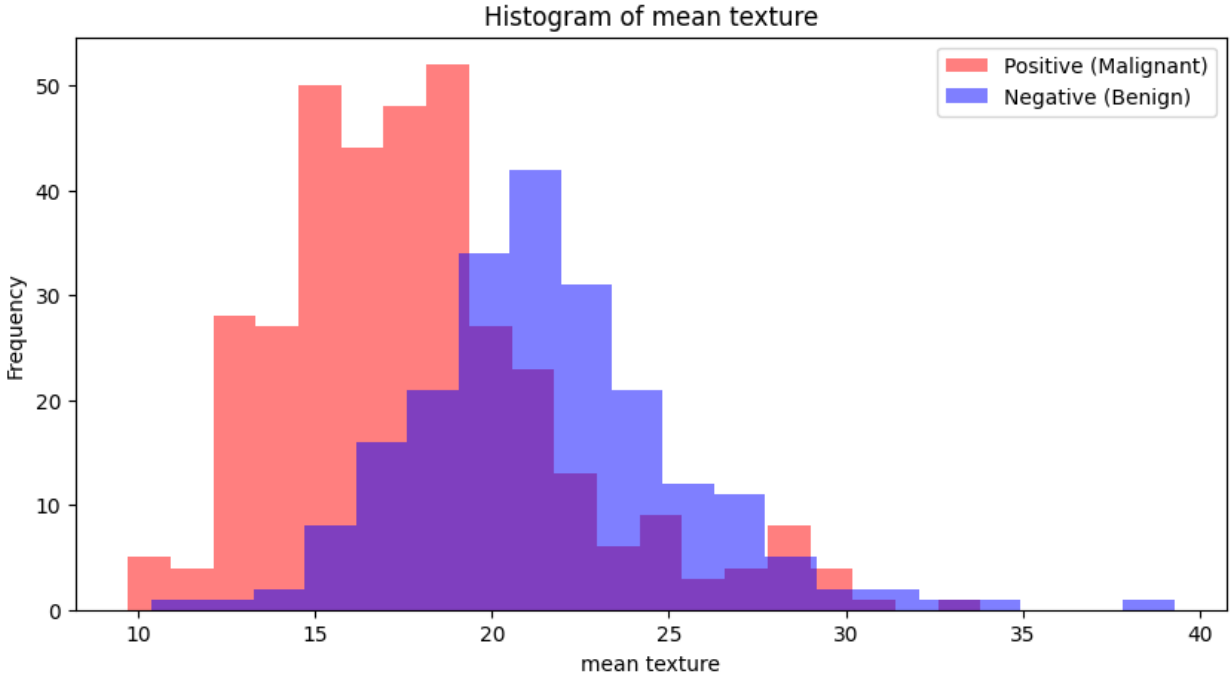


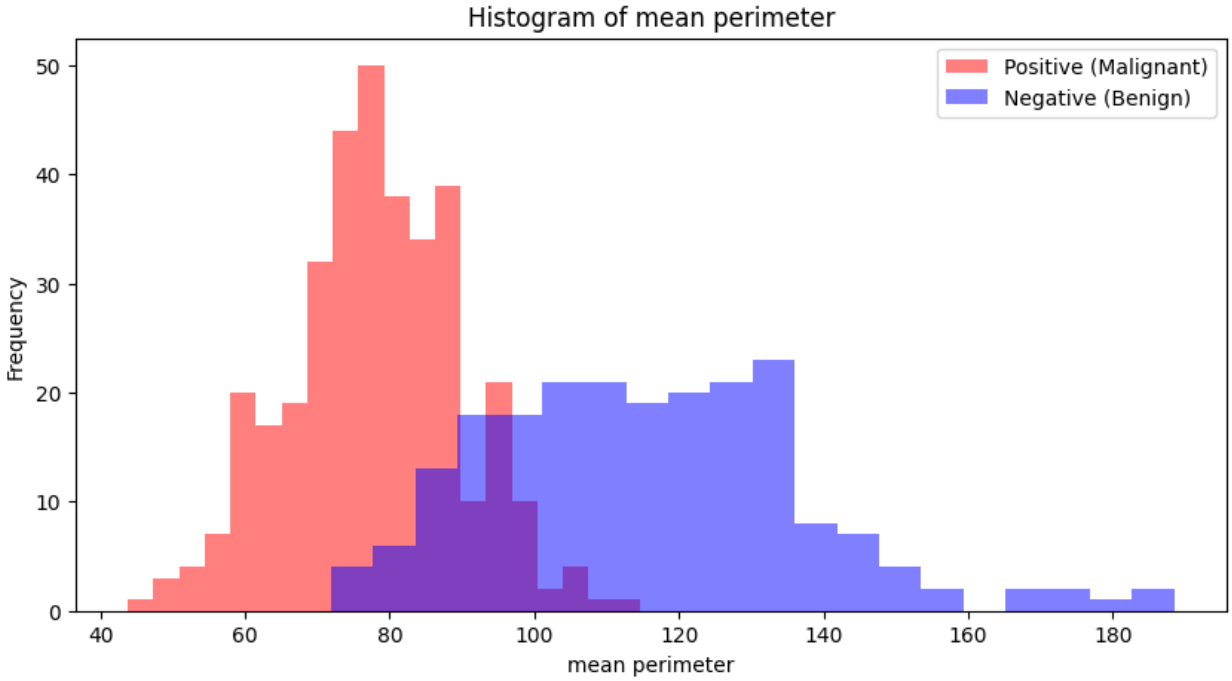
Q7. Create 'for loop' to enerate though tumor features and compare with histograms(20:-1)

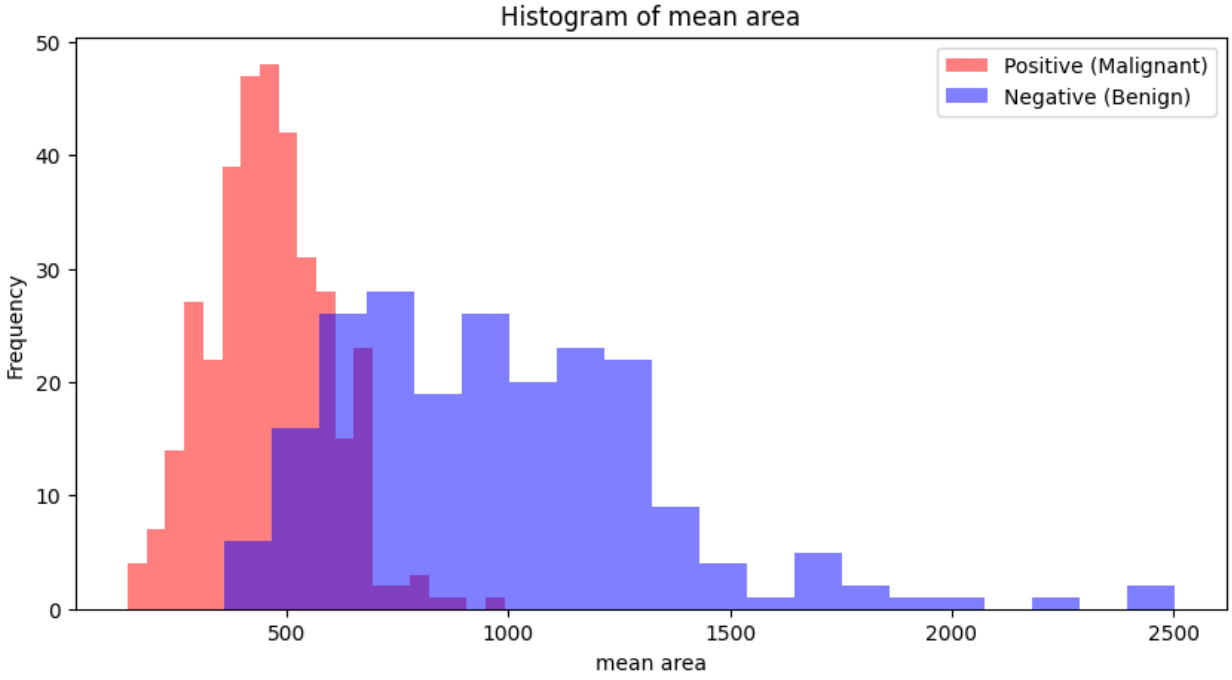
```
features = df_features.columns
for feature in features:
    plt.figure(figsize=(10, 5))
    plt.hist(df_positive[feature], bins=20, alpha=0.5, label='Positive
(Malignant)', color='red')
    plt.hist(df_negative[feature], bins=20, alpha=0.5, label='Negative
```

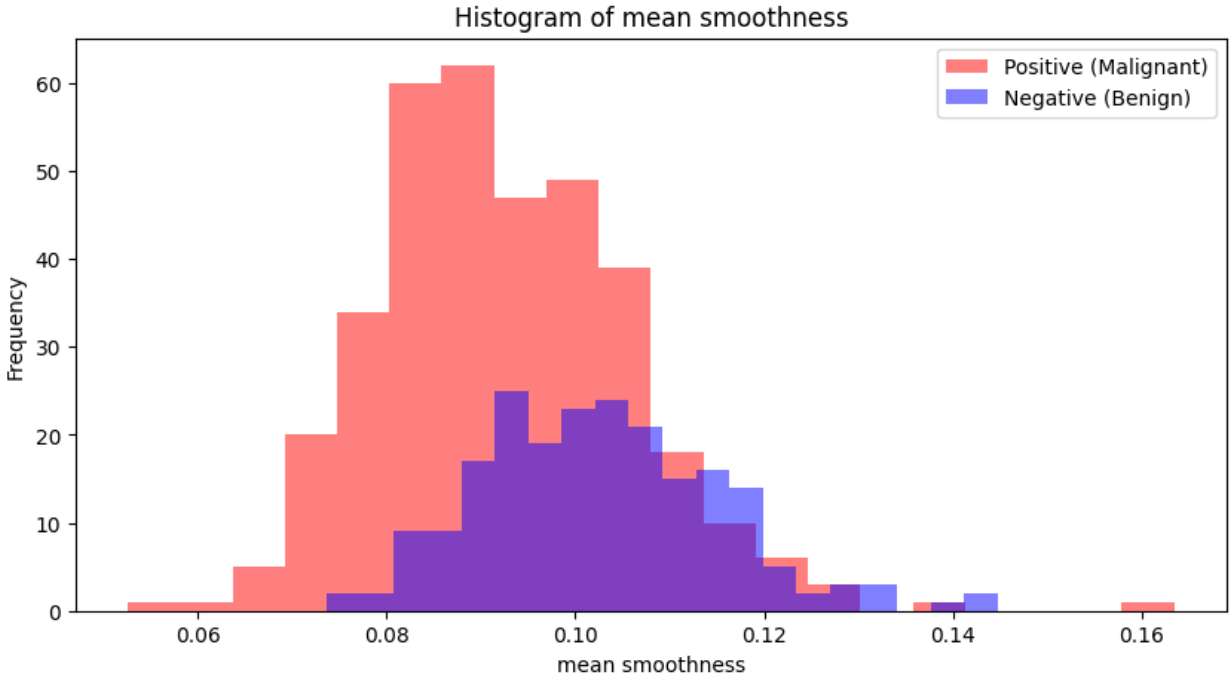
```
(Benign)', color='blue')
    plt.title(f'Histogram of {feature}')
    plt.xlabel(feature)
    plt.ylabel('Frequency')
    plt.legend()
    plt.show()
```

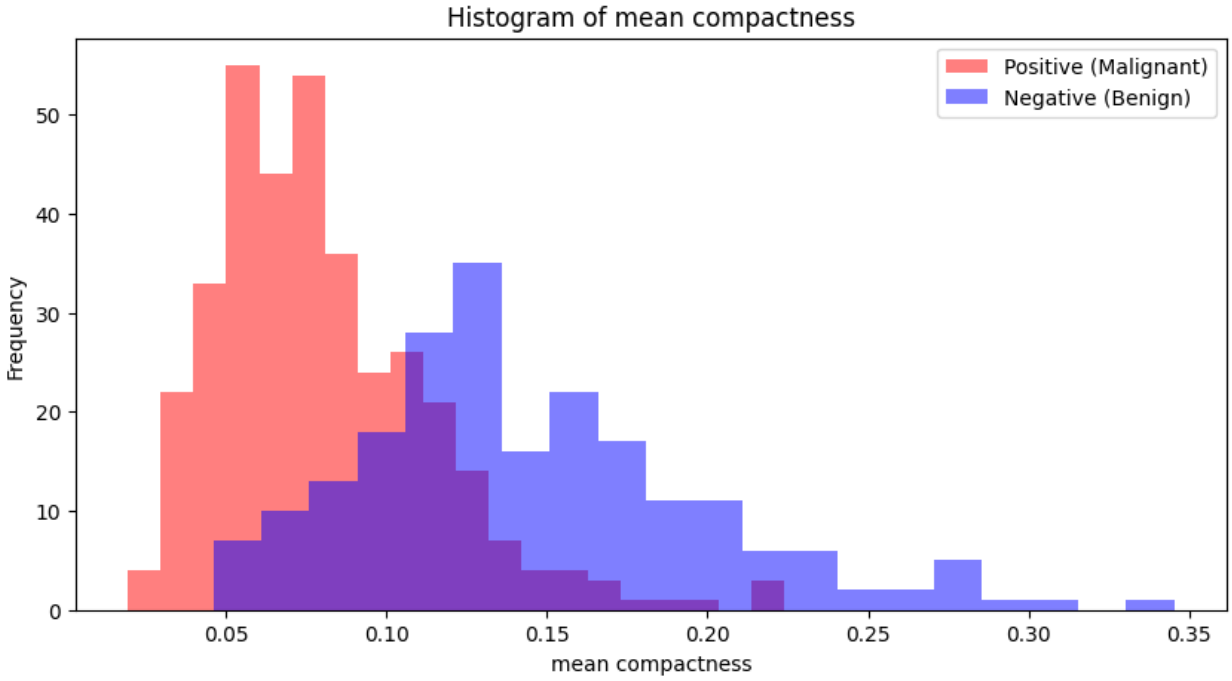


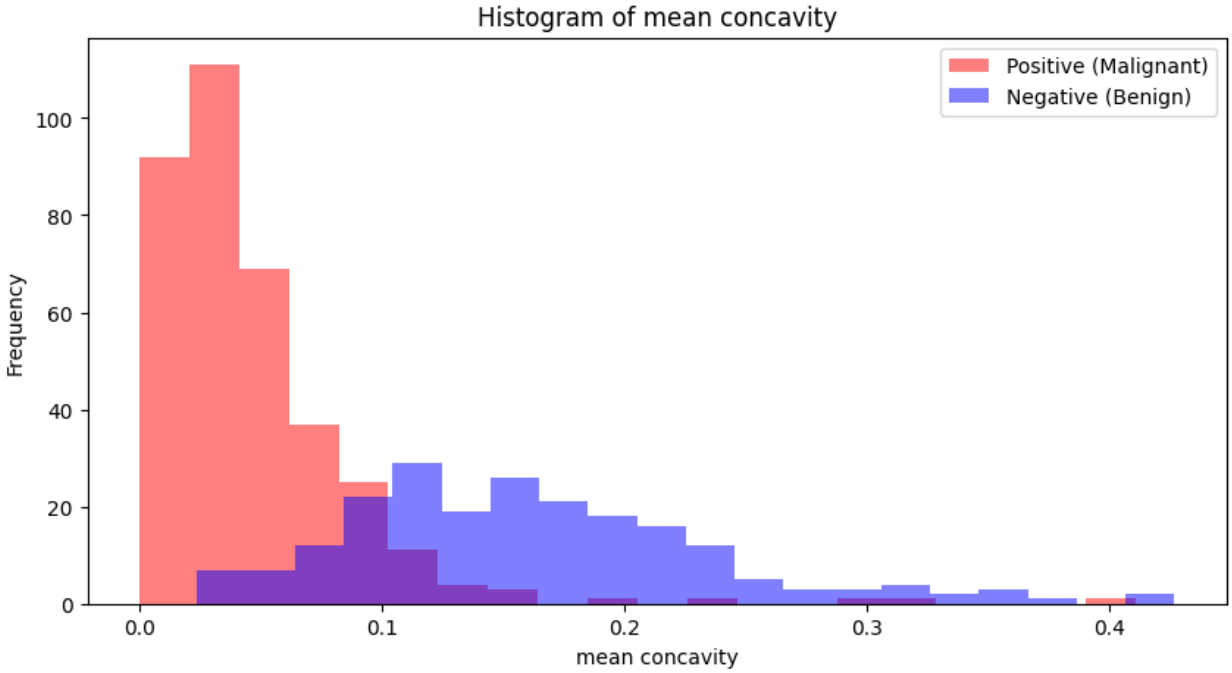


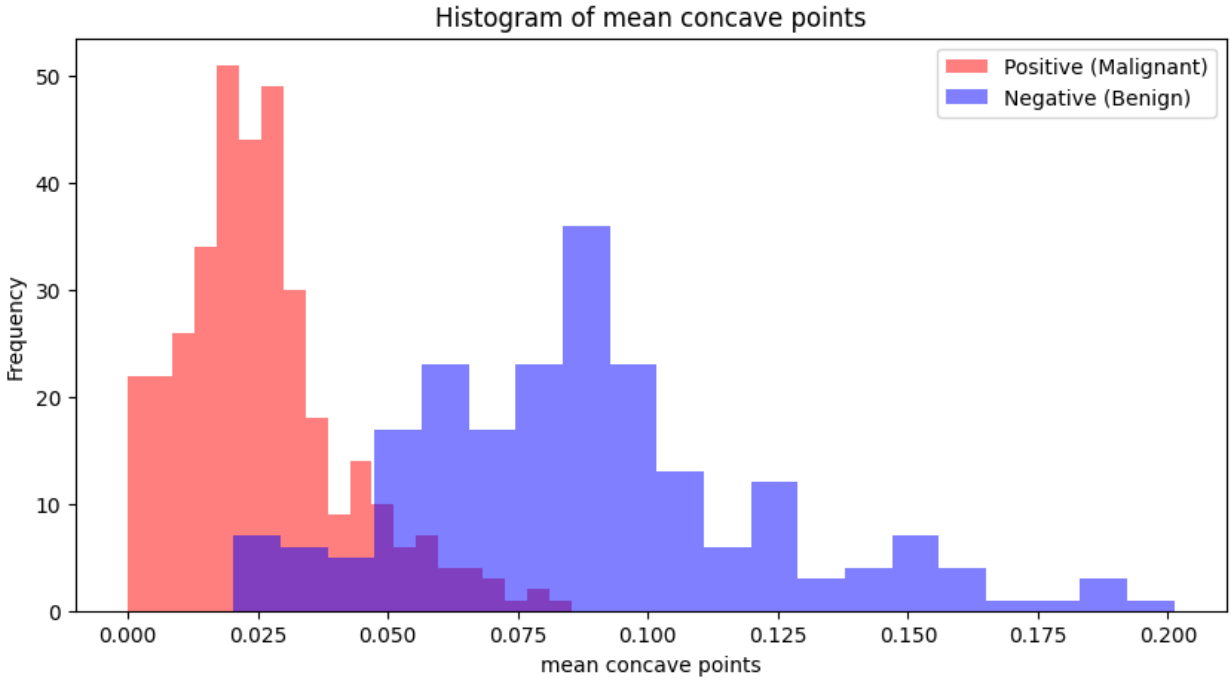

Histogram of mean perimeter
Positive (Malignant)
Negative (Benign)
Frequency
mean perimeter
0
10
20
30
40
50
40
60
80
100
120
140
160
180

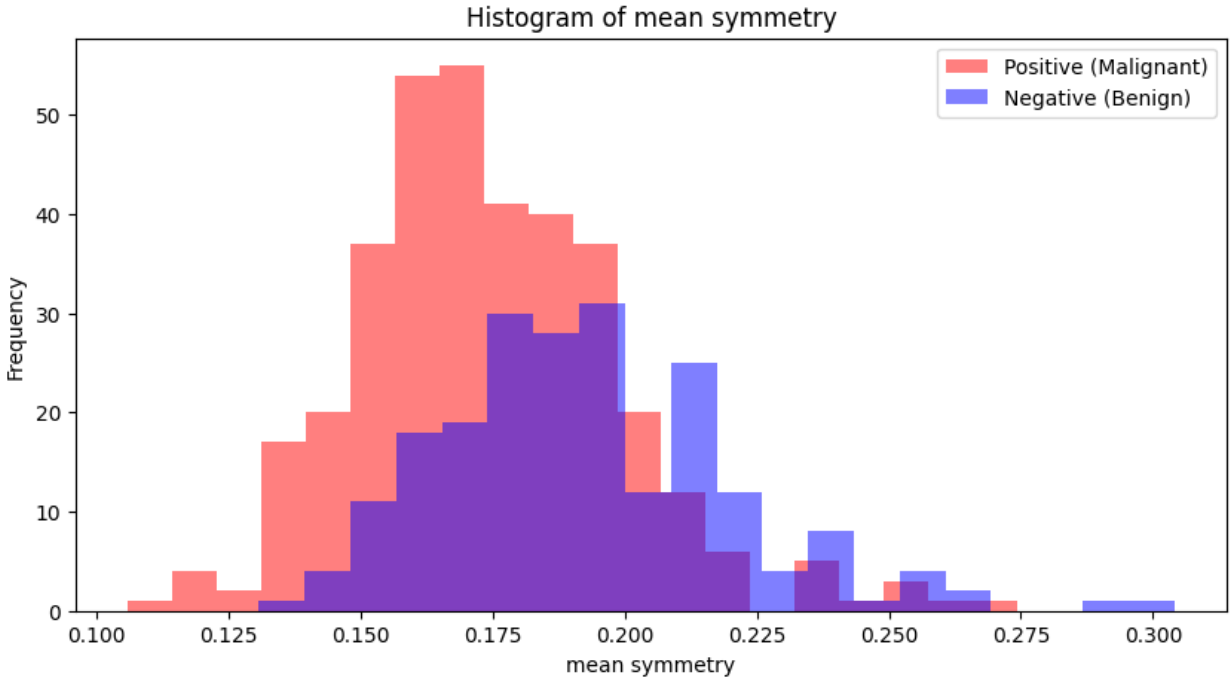

Histogram of mean area
Positive (Malignant)
Negative (Benign)
Frequency
mean area
0
10
20
30
40
50
500
1000
1500
2000
2500

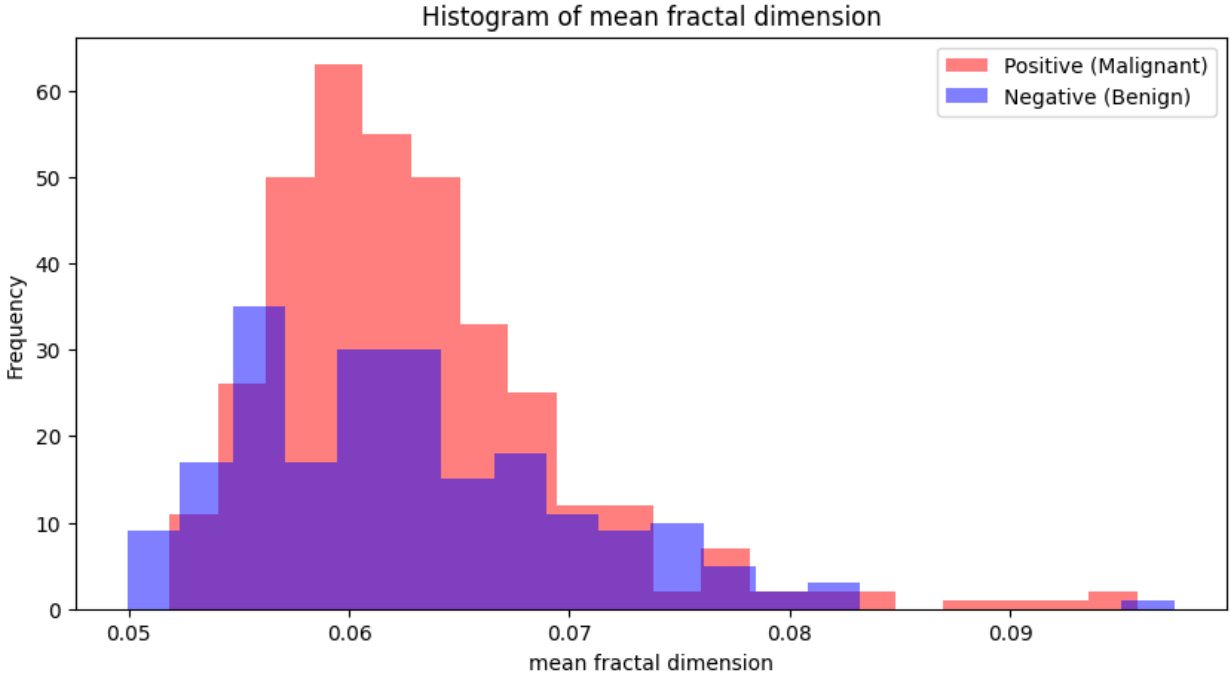

Histogram of mean smoothness
Positive (Malignant)
Negative (Benign)
Frequency
0
10
20
30
40
50
60
0.06
0.08
0.10
0.12
0.14
0.16
mean smoothness

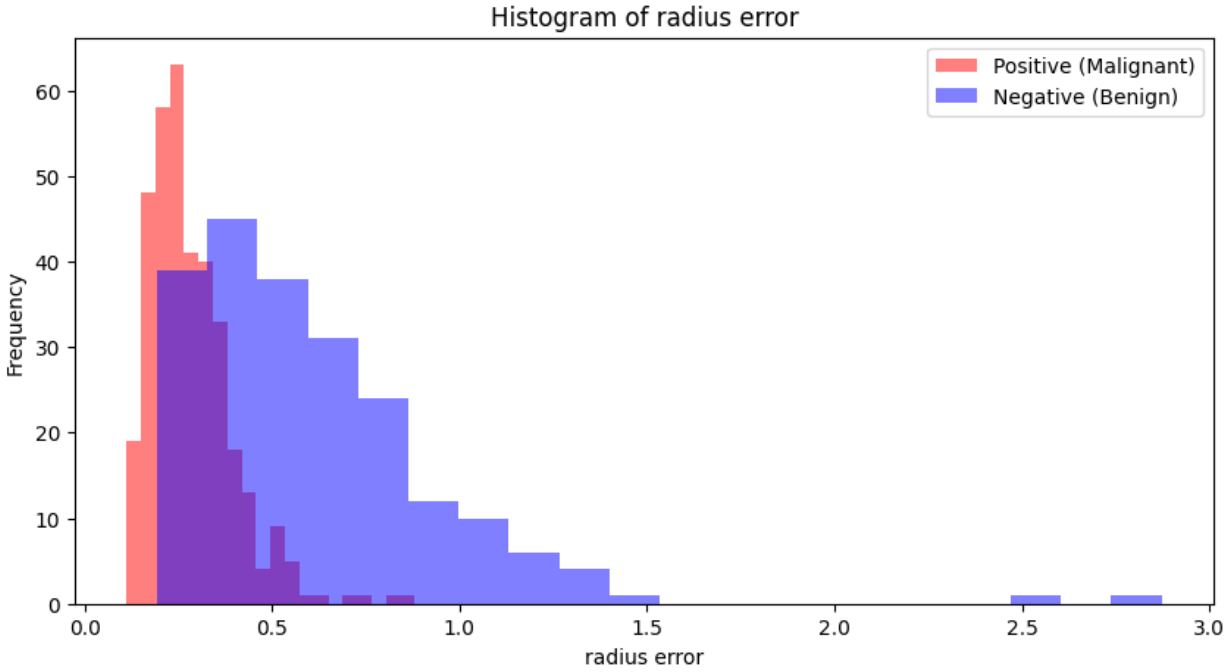

Histogram of mean compactness
Positive (Malignant)
Negative (Benign)
Frequency
0
10
20
30
40
50
0.05
0.10
0.15
0.20
0.25
0.30
0.35
mean compactness

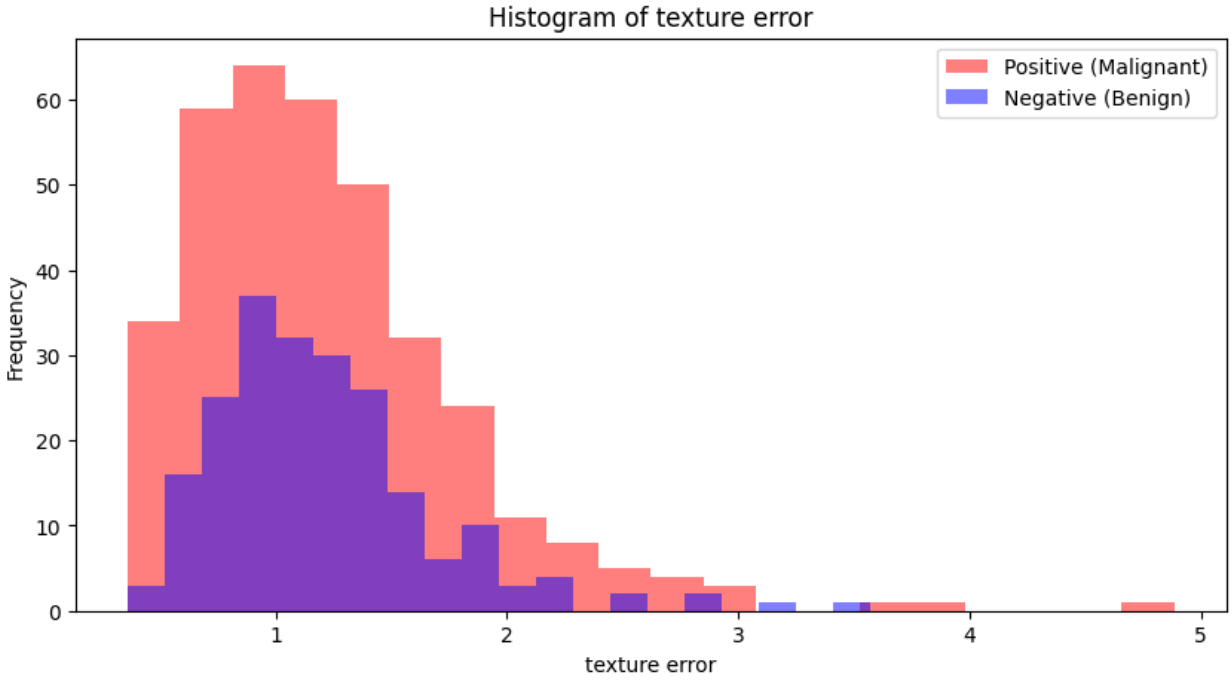

Histogram of mean concavity
Positive (Malignant)
Negative (Benign)
Frequency
mean concavity
0
20
40
60
80
100
0.0
0.1
0.2
0.3
0.4

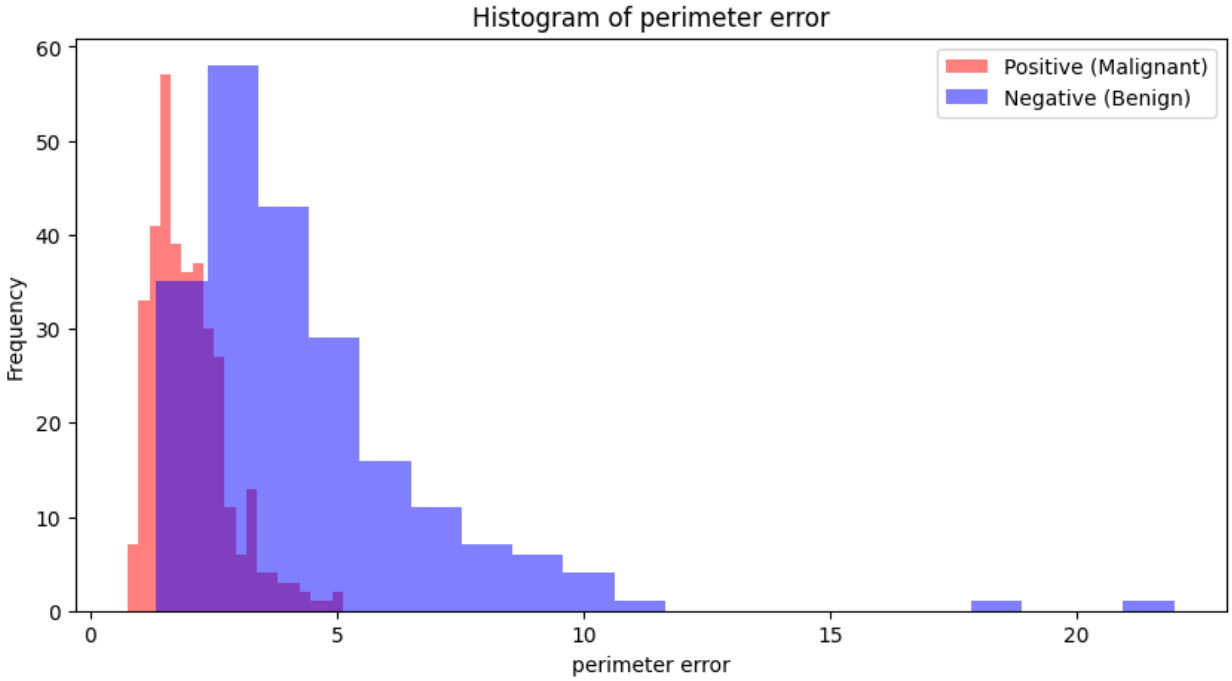

Histogram of mean concave points
Positive (Malignant)
Negative (Benign)
Frequency
mean concave points
0
10
20
30
40
50
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200

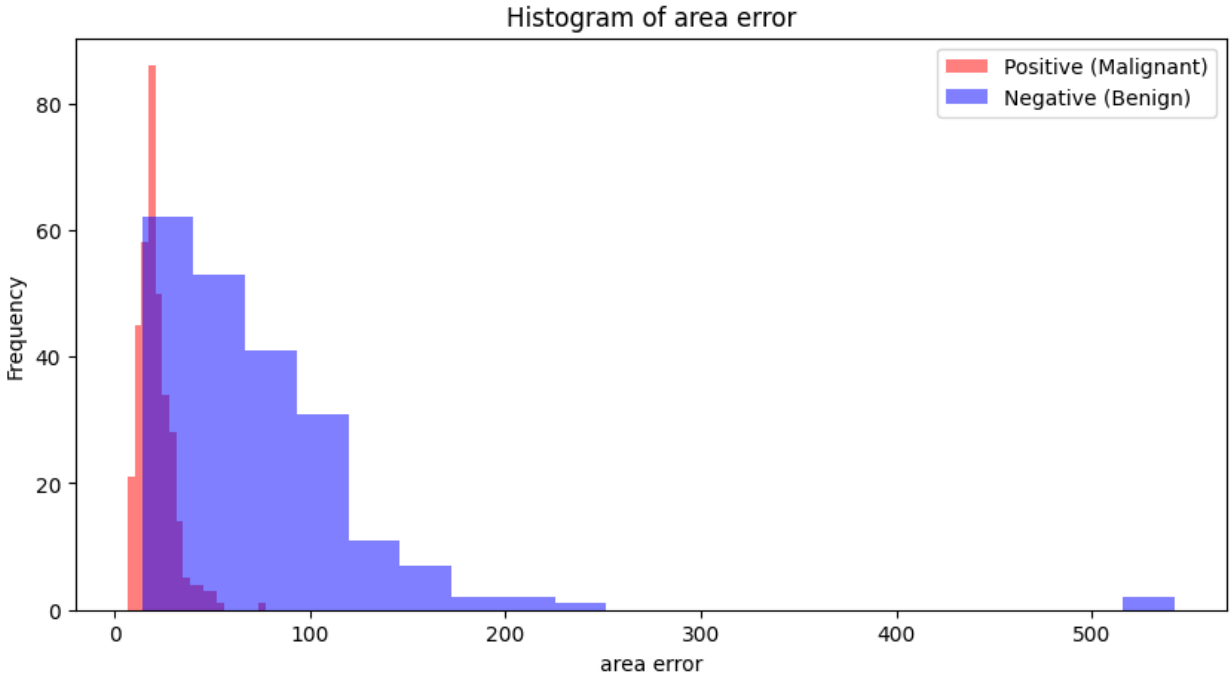

Histogram of mean symmetry
Positive (Malignant)
Negative (Benign)
Frequency
0
10
20
30
40
50
0.100
0.125
0.150
0.175
0.200
0.225
0.250
0.275
0.300
mean symmetry

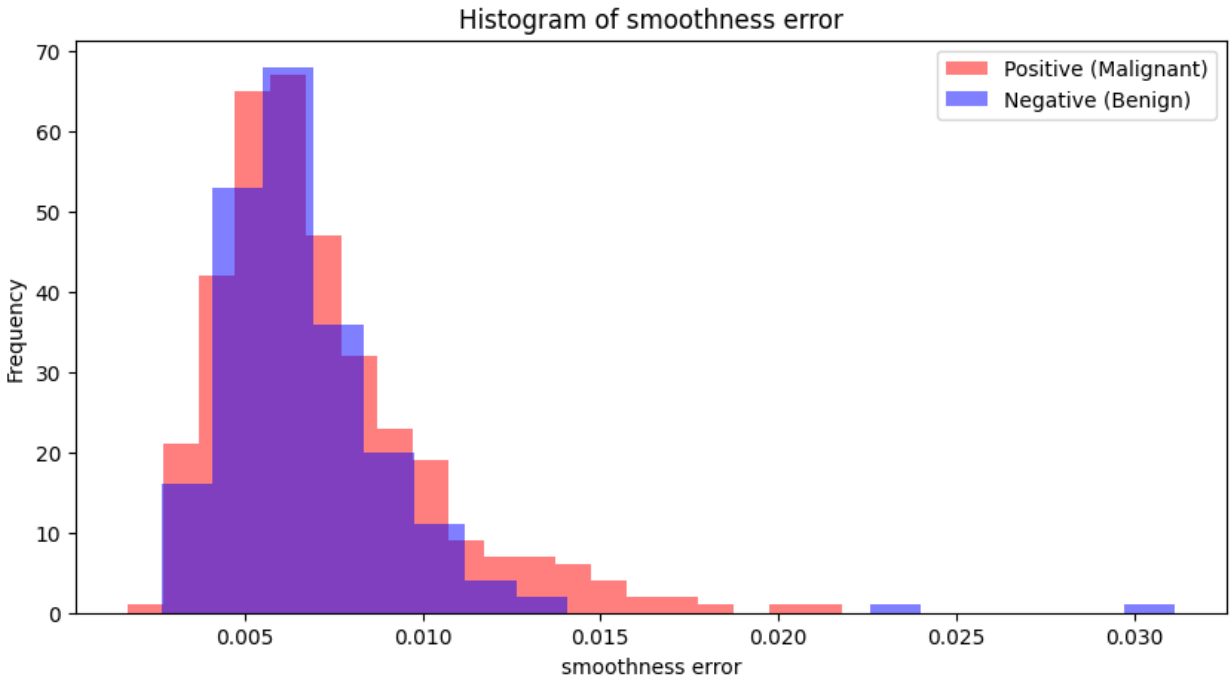

Histogram of mean fractal dimension
Positive (Malignant)
Negative (Benign)
Frequency
0
10
20
30
40
50
60
0.05
0.06
0.07
0.08
0.09
mean fractal dimension

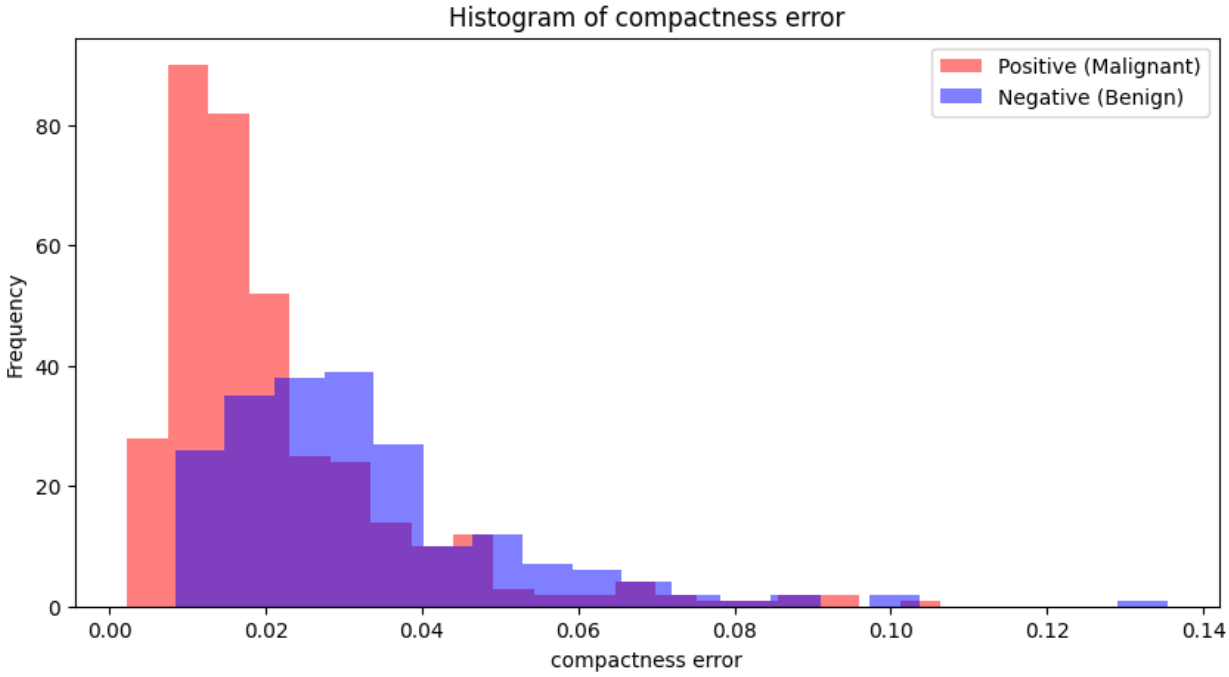

Histogram of radius error
Positive (Malignant)
Negative (Benign)
Frequency
radius error
0
10
20
30
40
50
60
0.0
0.5
1.0
1.5
2.0
2.5
3.0

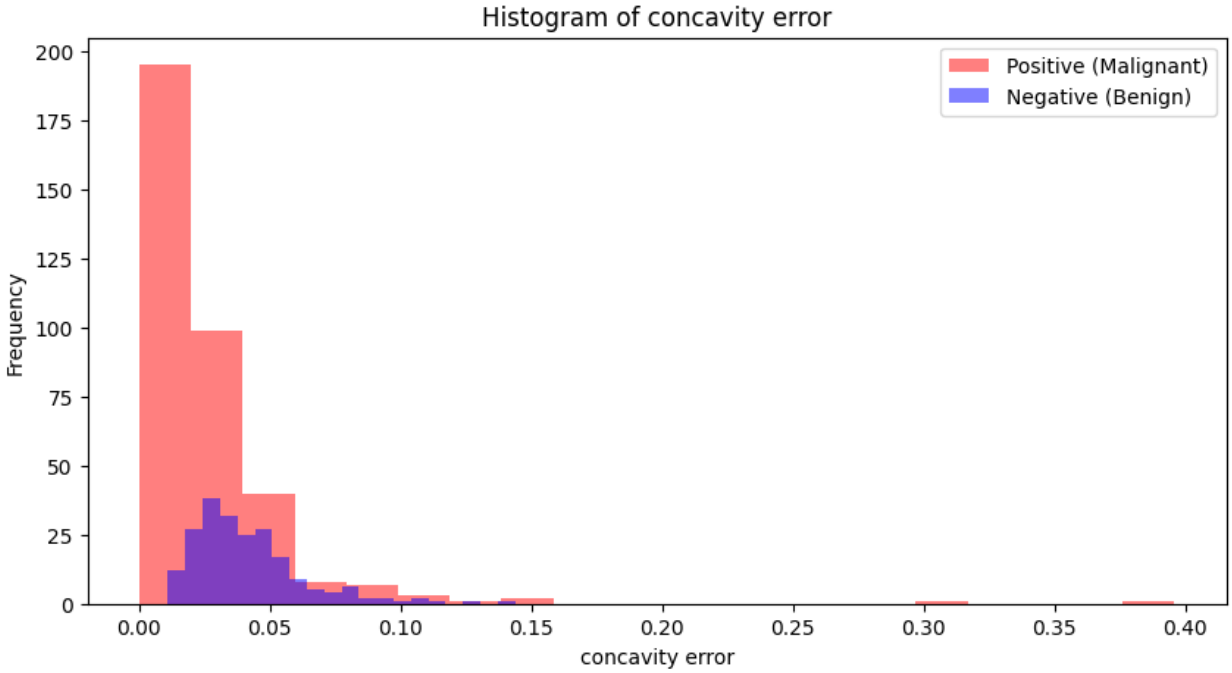

Histogram of texture error
Positive (Malignant)
Negative (Benign)
Frequency
texture error
0
10
20
30
40
50
60
1
2
3
4
5

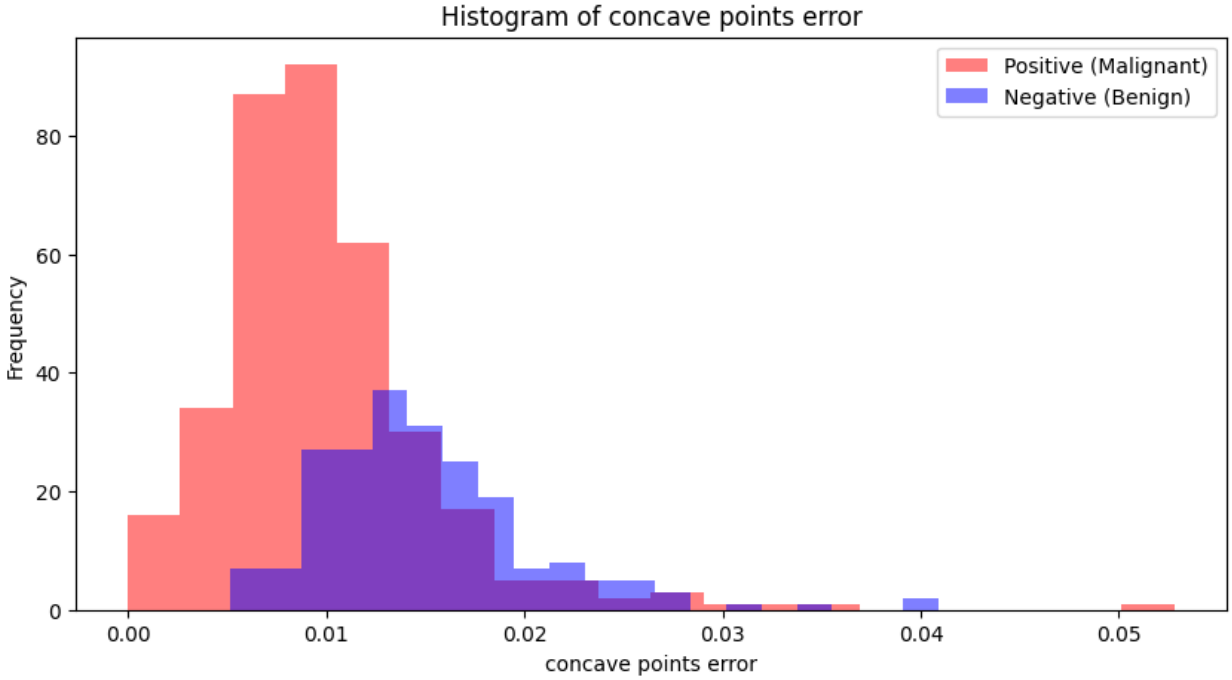

Histogram of perimeter error
Positive (Malignant)
Negative (Benign)
Frequency
perimeter error
0
10
20
30
40
50
60
0
5
10
15
20

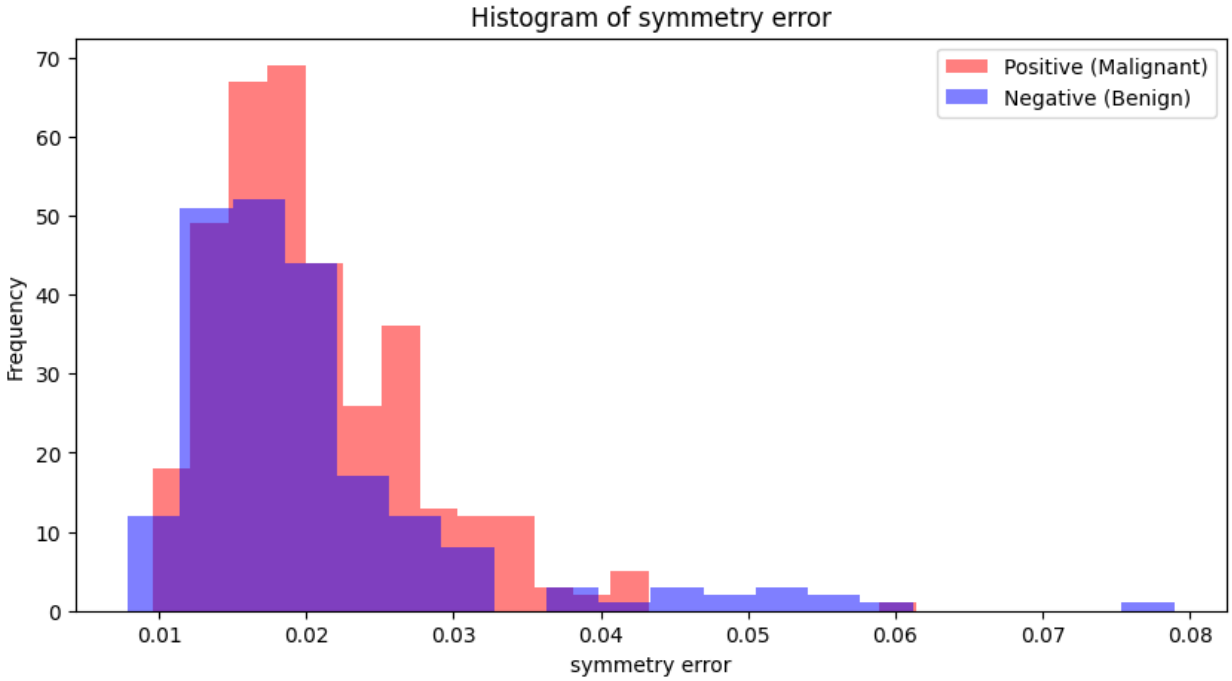

Histogram of area error
Positive (Malignant)
Negative (Benign)
Frequency
area error
0
20
40
60
80
0
100
200
300
400
500

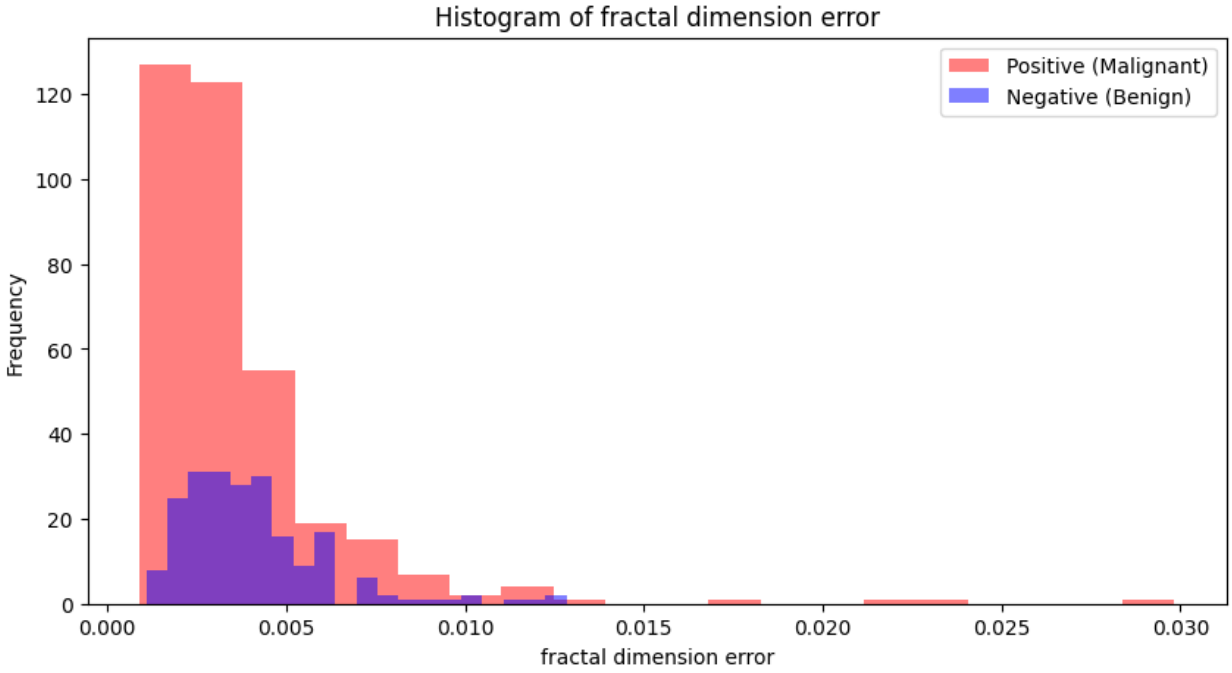

Histogram of smoothness error
Positive (Malignant)
Negative (Benign)
Frequency
0
10
20
30
40
50
60
70
0.005
0.010
0.015
0.020
0.025
0.030
smoothness error

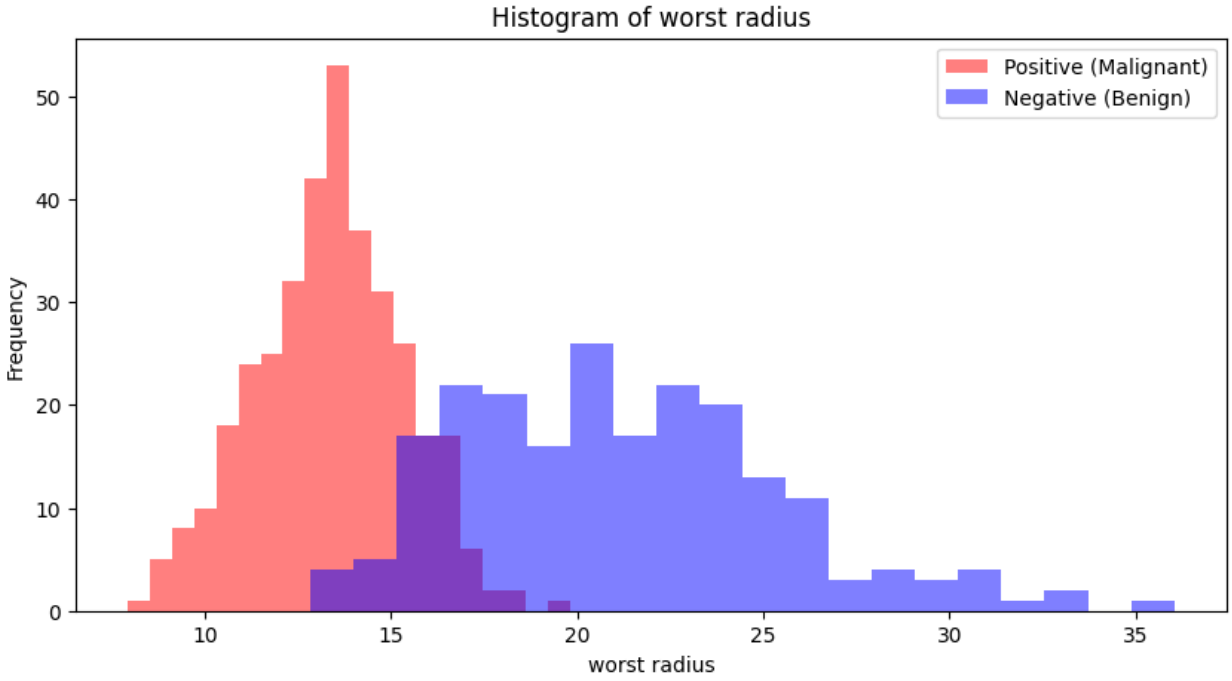

Histogram of compactness error
Positive (Malignant)
Negative (Benign)
Frequency
0
20
40
60
80
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
compactness error

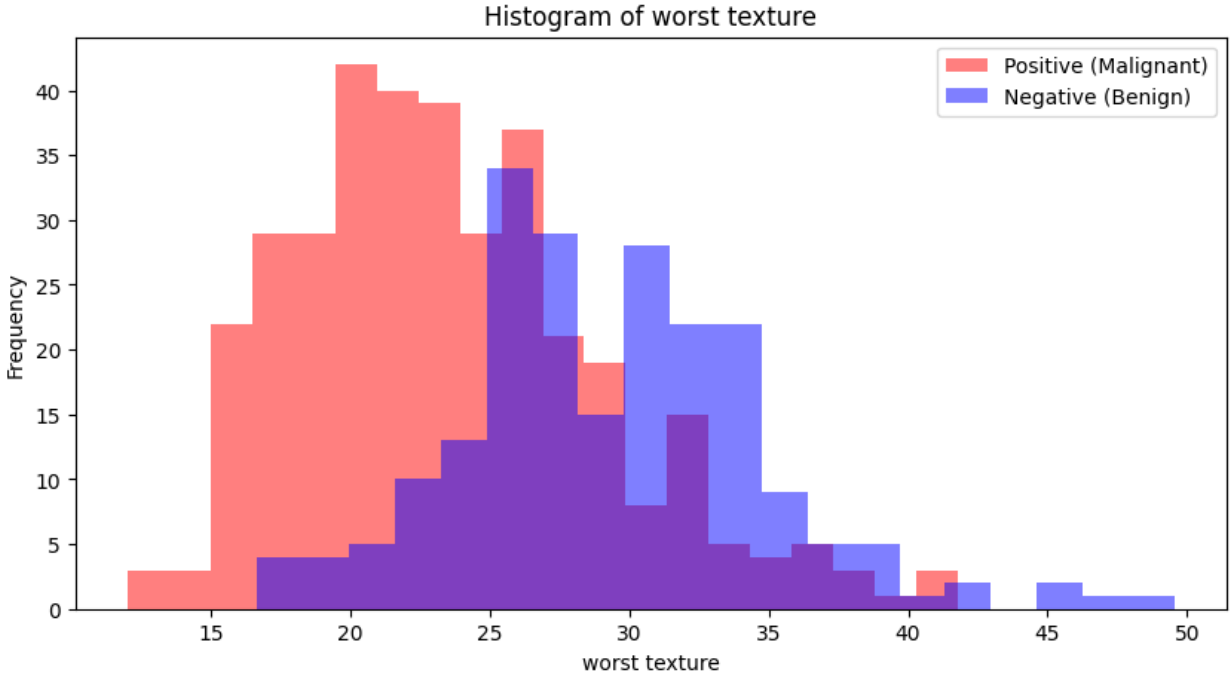

Histogram of concavity error
Positive (Malignant)
Negative (Benign)
Frequency
concavity error
0
25
50
75
100
125
150
175
200
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40

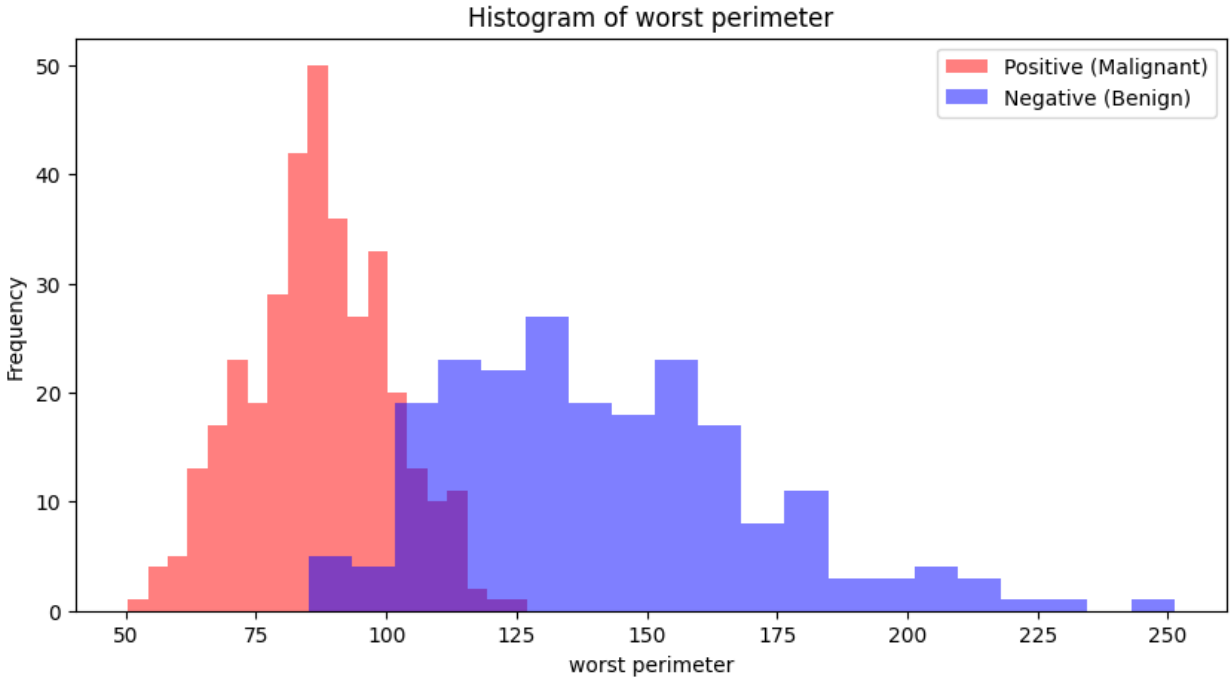

Histogram of concave points error
Positive (Malignant)
Negative (Benign)
Frequency
concave points error
0
20
40
60
80
0.00
0.01
0.02
0.03
0.04
0.05

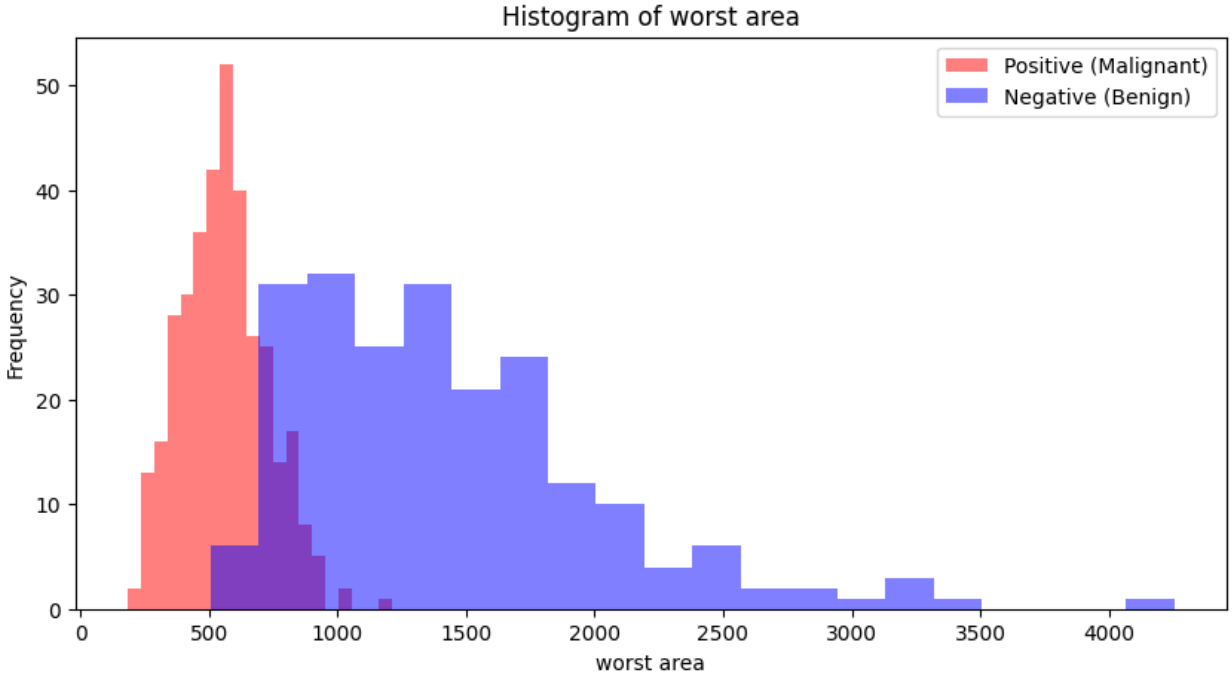

Histogram of symmetry error
Positive (Malignant)
Negative (Benign)
Frequency
symmetry error
0
10
20
30
40
50
60
70
0.01
0.02
0.03
0.04
0.05
0.06
0.07
0.08

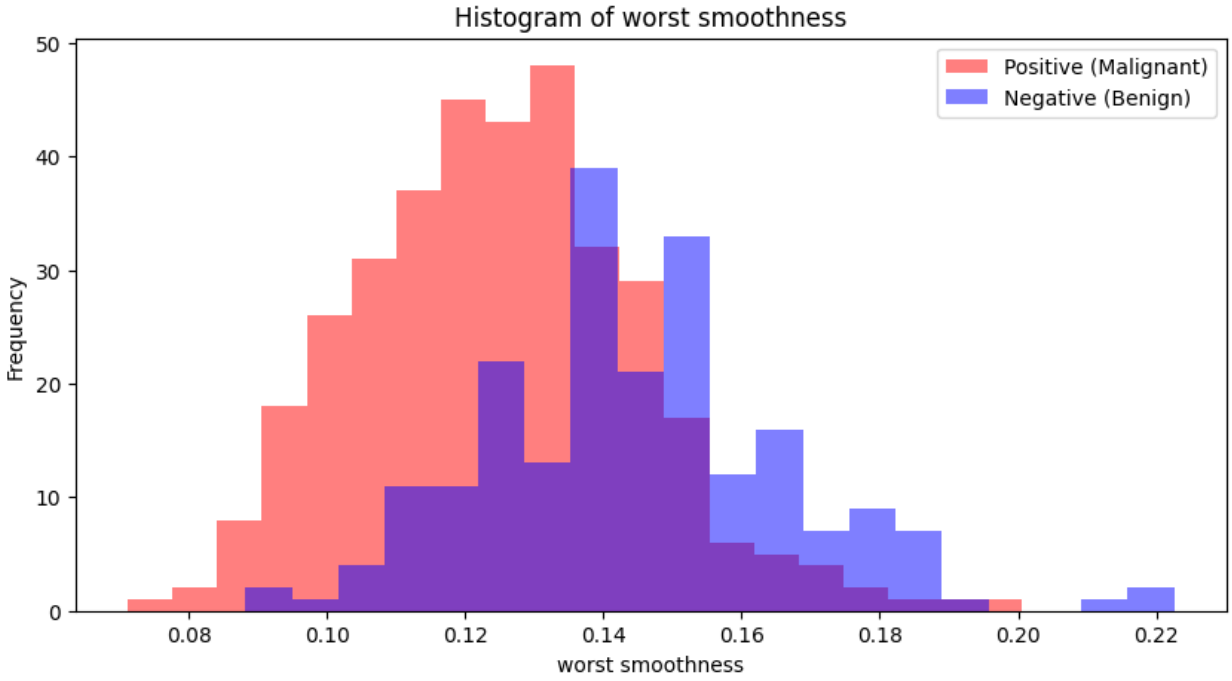

Histogram of fractal dimension error
Positive (Malignant)
Negative (Benign)
Frequency
fractal dimension error
0
20
40
60
80
100
120
0.000
0.005
0.010
0.015
0.020
0.025
0.030

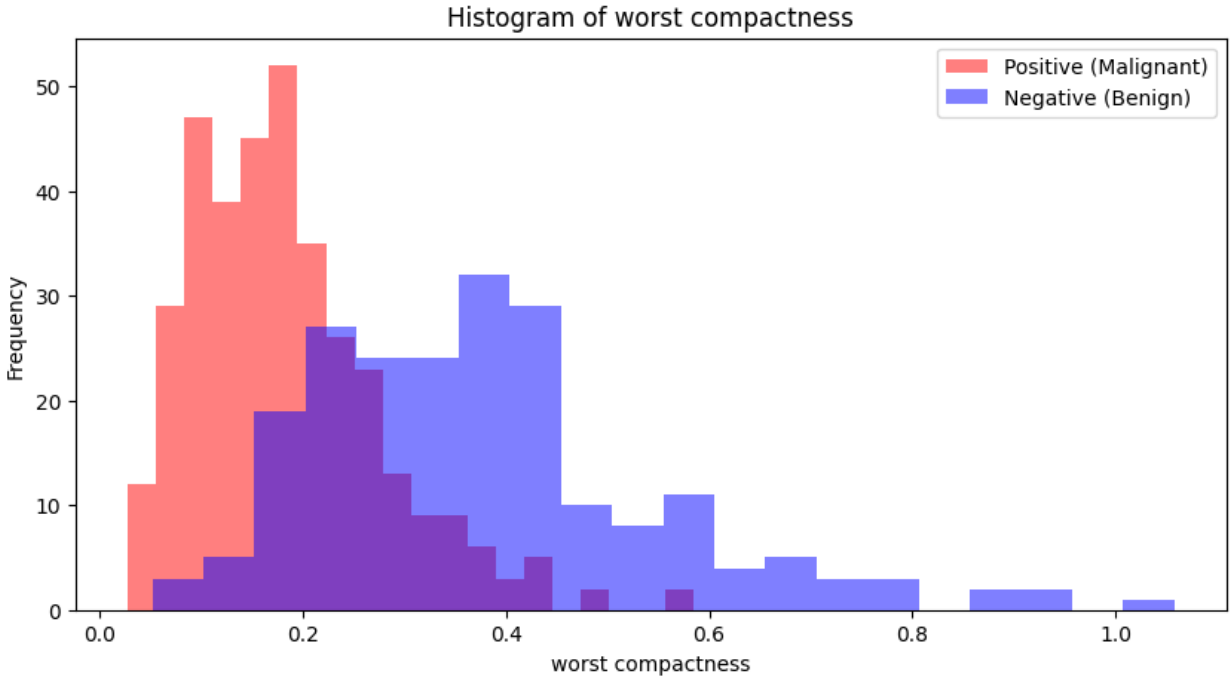

Histogram of worst radius
Positive (Malignant)
Negative (Benign)
Frequency
worst radius
0
10
20
30
40
50
10
15
20
25
30
35

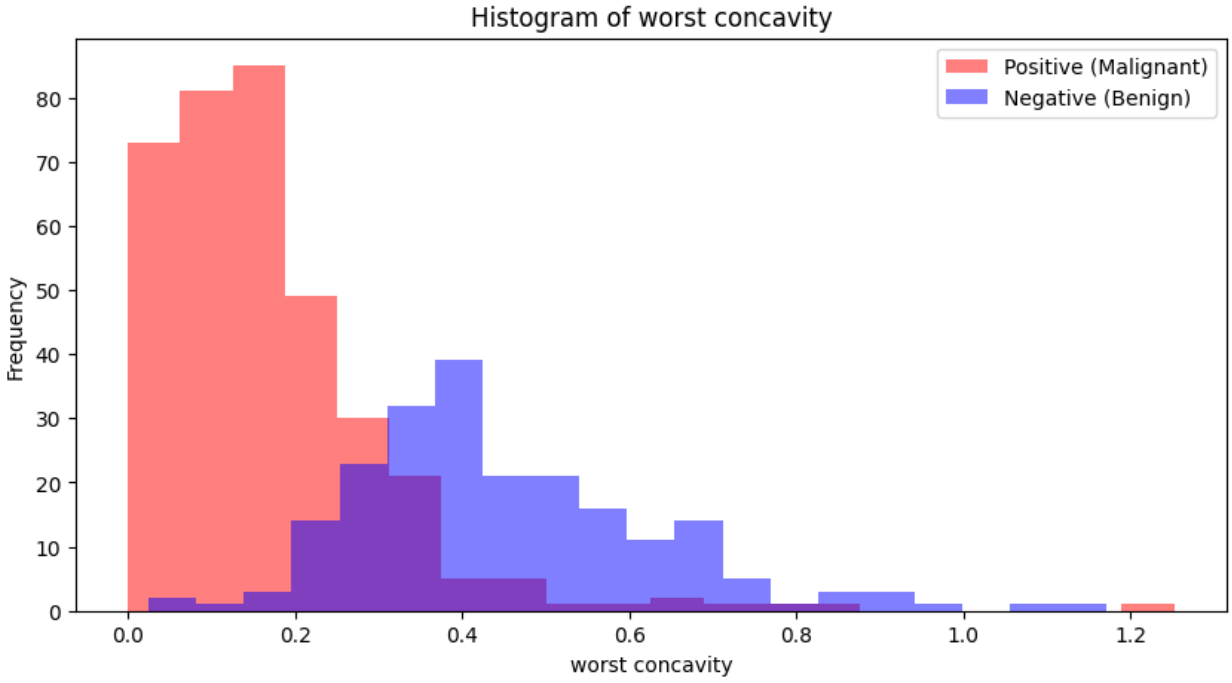

Histogram of worst texture
Positive (Malignant)
Negative (Benign)
Frequency
worst texture
0
5
10
15
20
25
30
35
40
15
20
25
30
35
40
45
50

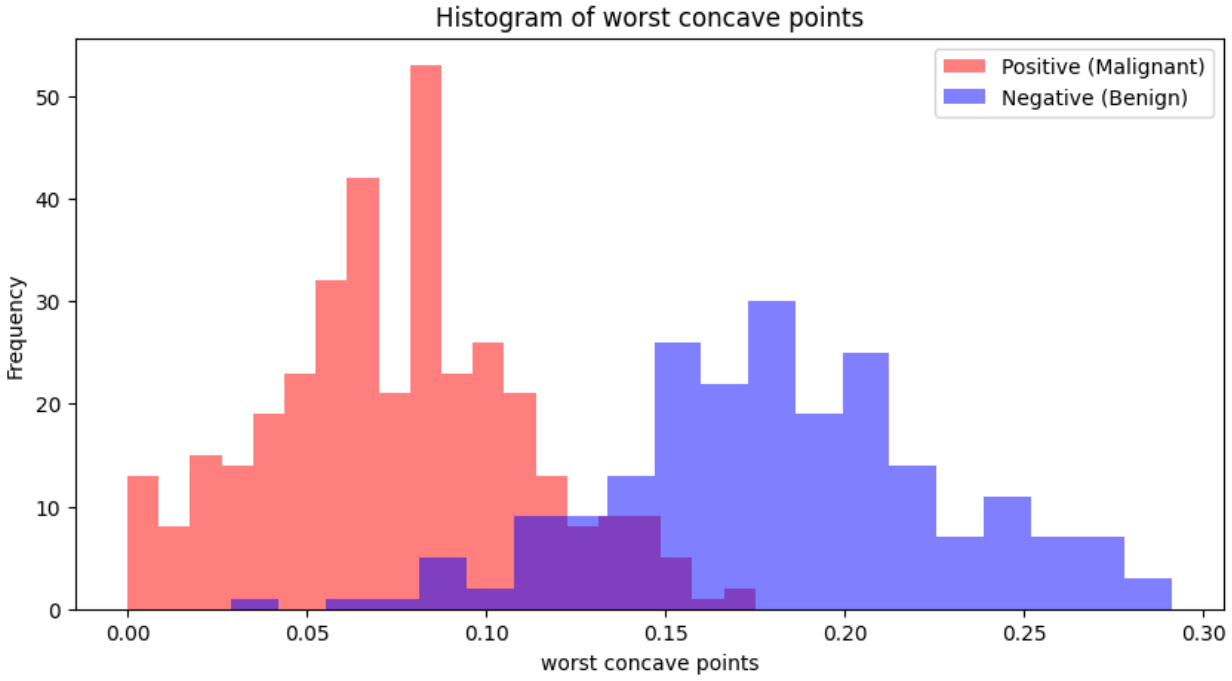

Histogram of worst perimeter
Positive (Malignant)
Negative (Benign)
Frequency
worst perimeter
0
10
20
30
40
50
50
75
100
125
150
175
200
225
250

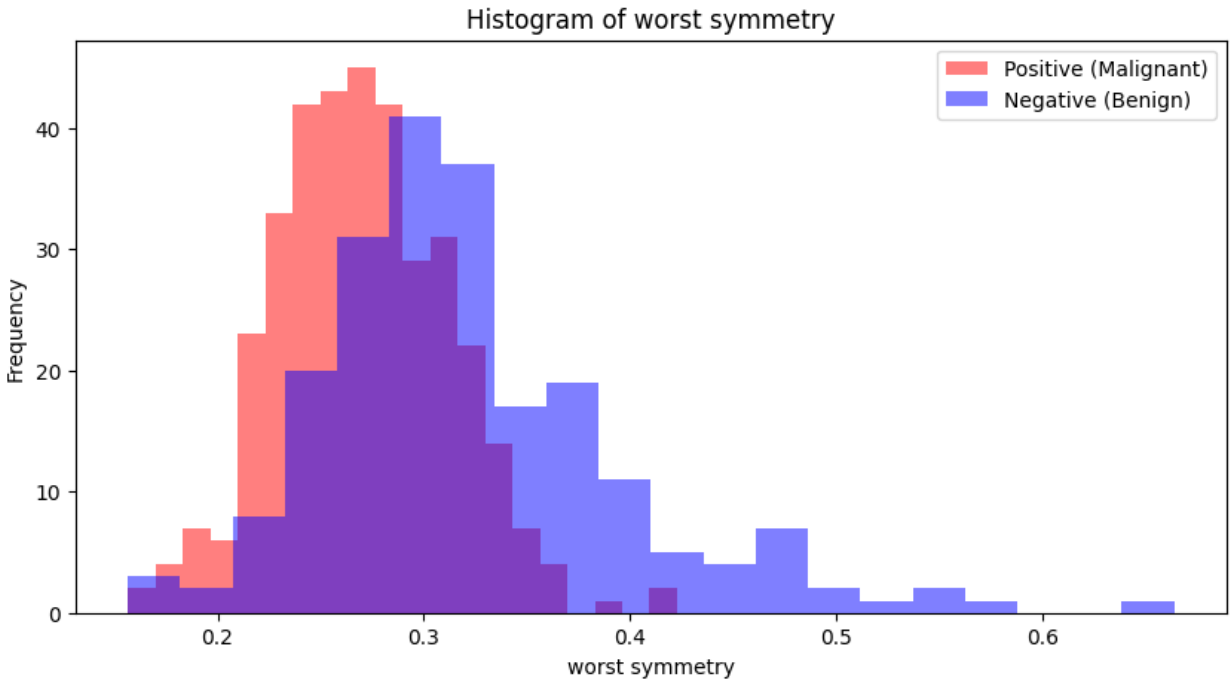

Histogram of worst area
Positive (Malignant)
Negative (Benign)
Frequency
worst area
0
10
20
30
40
50
0
500
1000
1500
2000
2500
3000
3500
4000

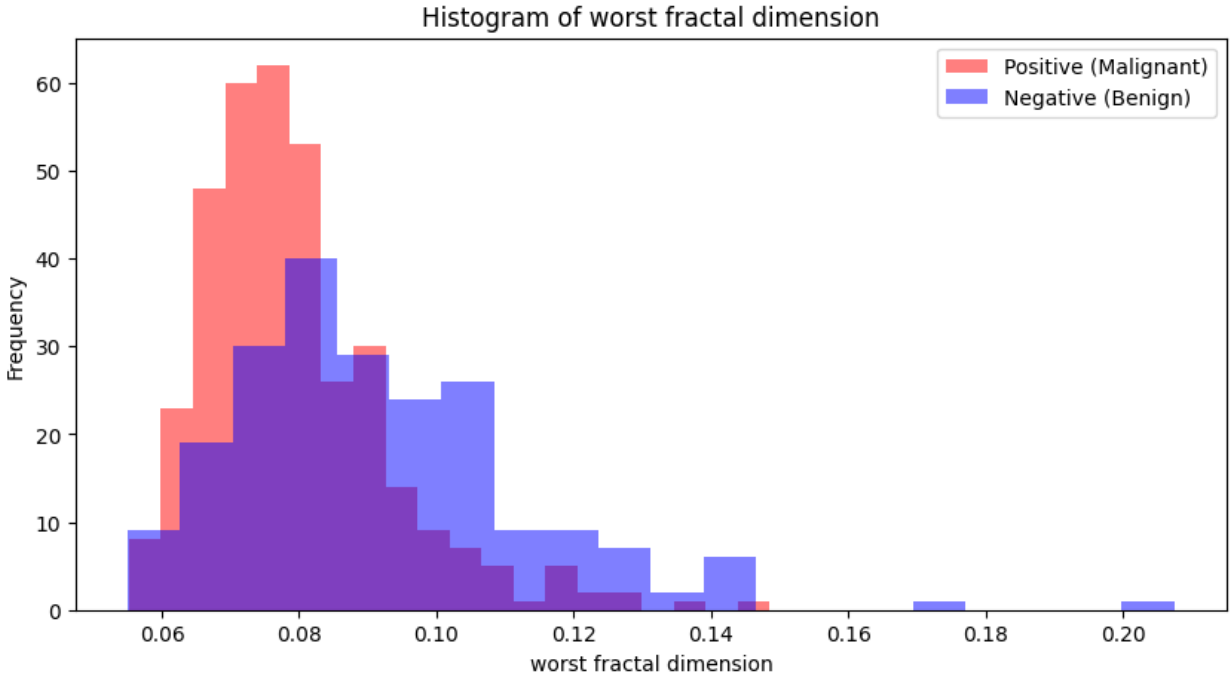

Histogram of worst smoothness
Positive (Malignant)
Negative (Benign)
Frequency
worst smoothness
0
10
20
30
40
50
0.08
0.10
0.12
0.14
0.16
0.18
0.20
0.22


Histogram of worst compactness
Positive (Malignant)
Negative (Benign)
Frequency
worst compactness
0
10
20
30
40
50
0.0
0.2
0.4
0.6
0.8
1.0


Histogram of worst concavity
Positive (Malignant)
Negative (Benign)
Frequency
0
10
20
30
40
50
60
70
80
0.0
0.2
0.4
0.6
0.8
1.0
1.2
worst concavity

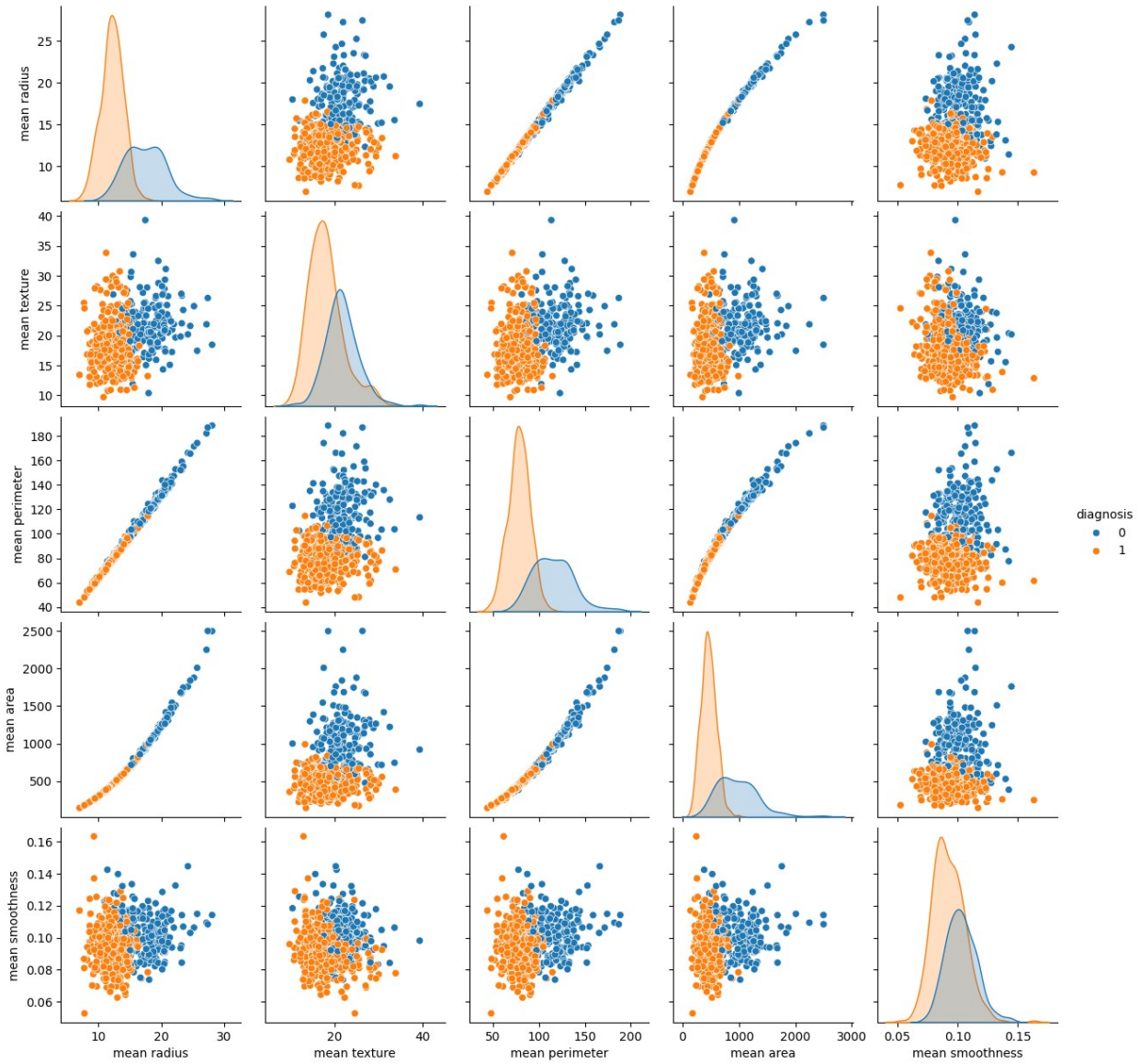

Histogram of worst concave points
Positive (Malignant)
Negative (Benign)
Frequency
0
10
20
30
40
50
0.00
0.05
0.10
0.15
0.20
0.25
0.30
worst concave points





Q8. Find Visualization of relationships between features and diagnoses.

```
import seaborn as sns
sns.pairplot(df, hue='diagnosis', vars=features[:5])
plt.show()
```



Q9. Split data into testing and training set. Use 80% for training

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)

Training set shape: (455, 30)
Testing set shape: (114, 30)
```

Q10. normalize features to account for feature scaling

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Q11. Apply DT, NB, KNN Logistic Regression and SVm to find accuracy, F1-Score and Recall.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score, recall_score
models = {
    'Decision Tree': DecisionTreeClassifier(),
    'Naive Bayes': GaussianNB(),
    'K-Nearest Neighbors': KNeighborsClassifier(),
    'Logistic Regression': LogisticRegression(max_iter=10000),
    'Support Vector Machine': SVC()
}
results = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    results[name] = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'F1 Score': f1_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred)
    }
for name, metrics in results.items():
    print(f"{name}:")
    for metric, value in metrics.items():
        print(f"  {metric}: {value:.4f}")
```

```
C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\site-
packages\joblib\externals\loky\backend\context.py:136: UserWarning:
Could not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this
warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want
to use.
  warnings.warn(
  File "C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\site-
packages\joblib\externals\loky\backend\context.py", line 257, in
_count_physical_cores
    cpu_info = subprocess.run(
  File "C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\
subprocess.py", line 505, in run
    with Popen(*popenargs, **kwargs) as process:
  File "C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\
subprocess.py", line 951, in __init__
```

```
    self._execute_child(args, executable, preexec_fn, close_fds,
  File "C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\
subprocess.py", line 1420, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,

Decision Tree:
  Accuracy: 0.9386
  F1 Score: 0.9504
  Recall: 0.9437
Naive Bayes:
  Accuracy: 0.9649
  F1 Score: 0.9722
  Recall: 0.9859
K-Nearest Neighbors:
  Accuracy: 0.9474
  F1 Score: 0.9577
  Recall: 0.9577
Logistic Regression:
  Accuracy: 0.9737
  F1 Score: 0.9790
  Recall: 0.9859
Support Vector Machine:
  Accuracy: 0.9825
  F1 Score: 0.9861
  Recall: 1.0000
```

Q12. Further apply the PCA to do the same exercise in Q11.

```
Q13. Do the Comprative study with PCA and WITHout PCA.
```

Part 4.2

Q1. Apply Mc-Culloch Pitts model for the following Boolean operations:

Plot the decision boundaries for 2 - dimensional and 3-dimensional.

Implement AND with 3-variables and threshold 3.

Implement OR with 3-variables and threshold 1.

Implement X 1and !X2 with 2-variables and threshold 1.

Implement NOR with 2-variables and threshold 0.

Implement NOT with 2-variables and threshold 0.

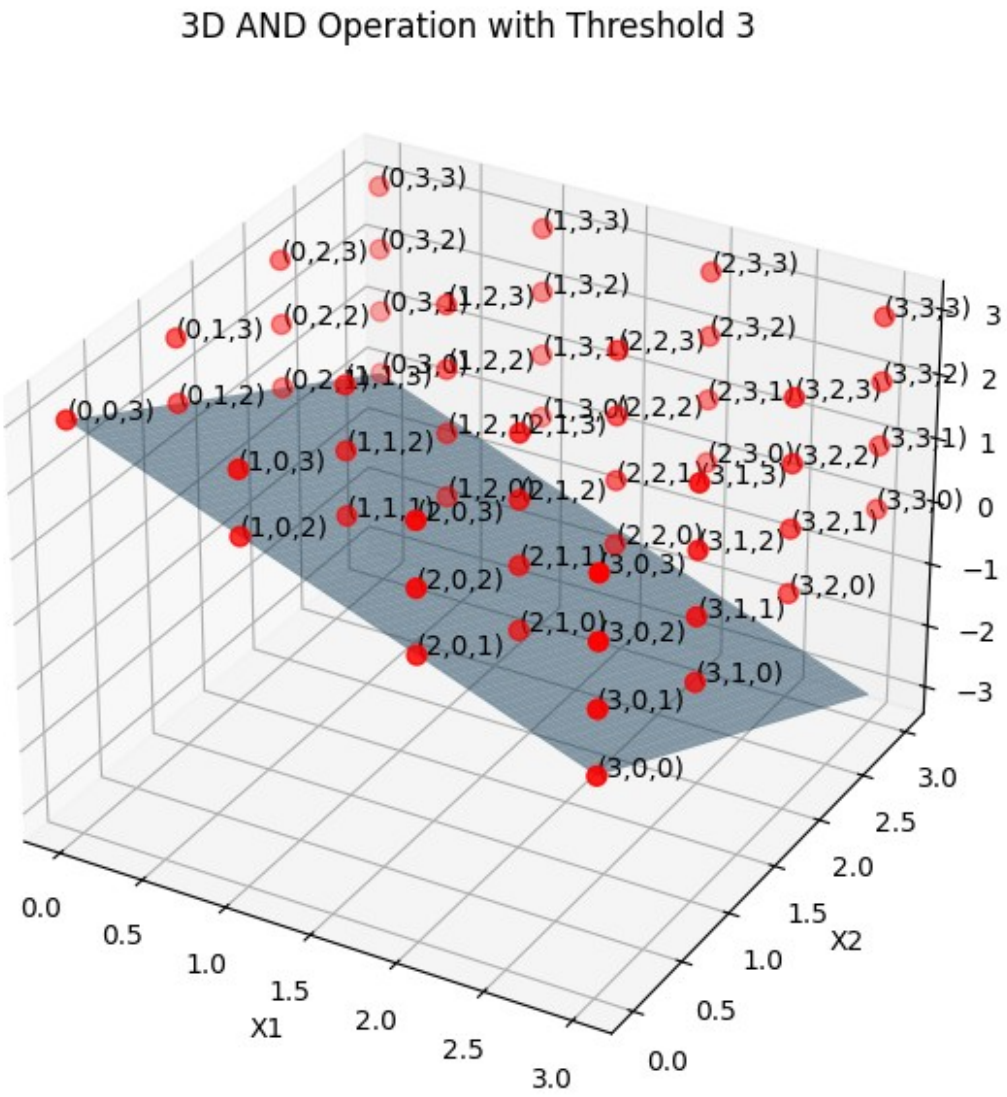
Implement OR with 2-variables and threshold 1.

Implement AND with 2-variables and threshold 2.

Implement OR with 3-variables and threshold 1.

Q1. Apply Mc-Culloch Pits model for the following Boolean operations: Plot the decision boundaries for 2 - dimensional and 3-dimensional. Implement AND with 3-variables and threshold 3.

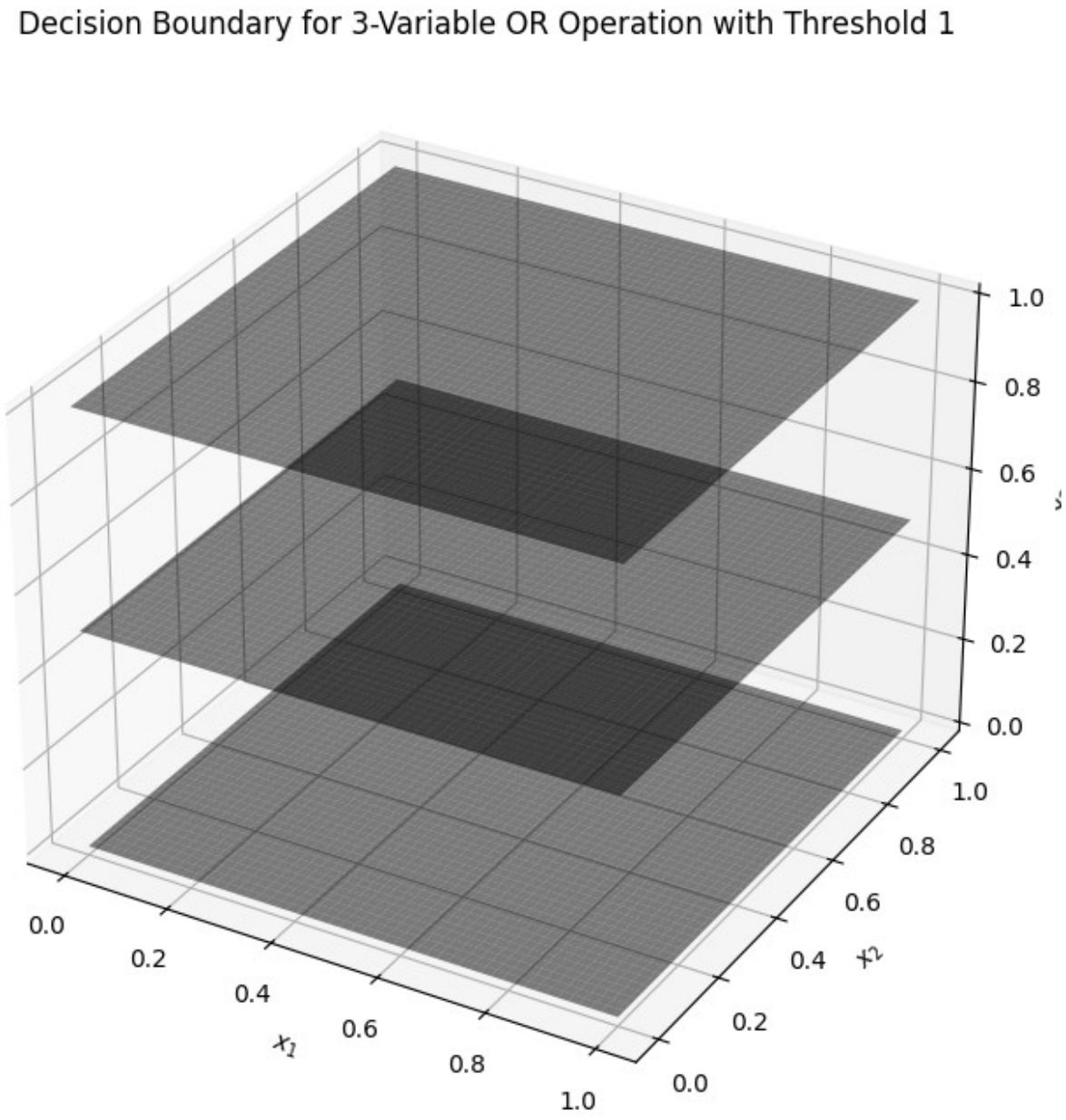
```
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
x1 = np.linspace(0, 3, 100)
x2 = np.linspace(0, 3, 100)
X1, X2 = np.meshgrid(x1, x2)
X3 = 3 - X1 - X2
ax.plot_surface(X1, X2, X3, alpha=0.5)
points = np.array([(i, j, k) for i in range(4) for j in range(4) for k
in range(4) if i + j + k >= 3])
ax.scatter(points[:, 0], points[:, 1], points[:, 2], color='r', s=50)
for point in points:
    ax.text(*point, f'({point[0]},{point[1]},{point[2]})',
color='black')
ax.set_xlabel('X1')
ax.set_ylabel('X2')
ax.set_zlabel('X3')
ax.set_title('3D AND Operation with Threshold 3')
plt.show()
```



Implement OR with 3-variables and threshold 1.

```
x = np.linspace(0, 1, 100)
X1, X2 = np.meshgrid(x, x)
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
x3_values = [0, 0.5, 1]
for x3 in x3_values:
    Z = (X1 + X2) >= (1 - x3)
    ax.plot_surface(X1, X2, x3 * np.ones_like(X1),
                    facecolors=plt.cm.gray(Z), alpha=0.5,
edgecolor='none')
ax.set_xlabel('$x_1$')
ax.set_ylabel('$x_2$')
ax.set_zlabel('$x_3$')
ax.set_title('Decision Boundary for 3-Variable OR Operation with
```

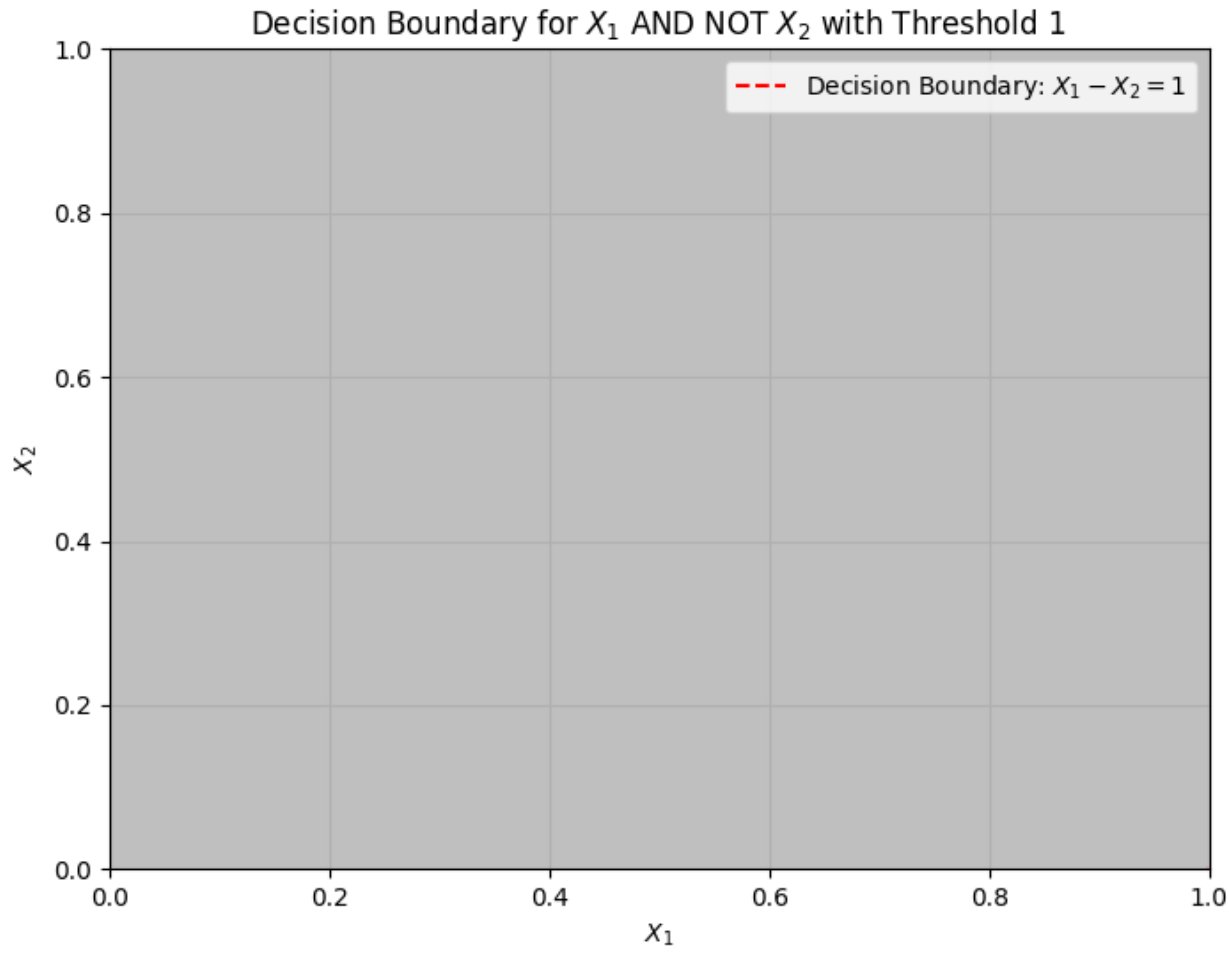
```
Threshold 1')
plt.show()
```



Implement X 1and !X2 with 2-variables and threshold 1.

```
x1 = np.linspace(0, 1, 100)
x2 = np.linspace(0, 1, 100)
X1, X2 = np.meshgrid(x1, x2)
weighted_sum = X1 - X2
decision_boundary = weighted_sum >= 1
plt.figure(figsize=(8, 6))
```

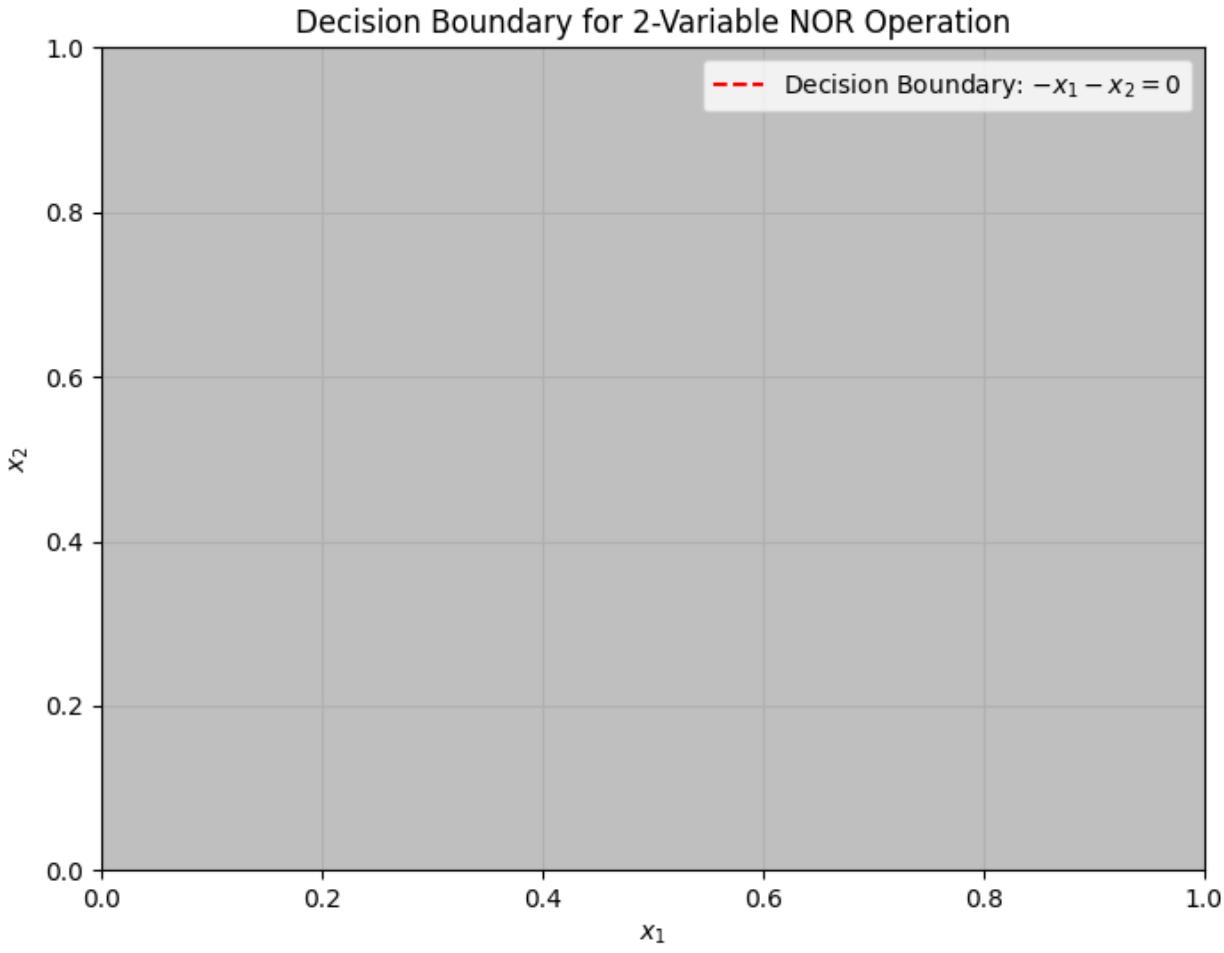
```
plt.contourf(X1, X2, decision_boundary, alpha=0.5, cmap='gray',
levels=[0, 1])
plt.plot(X1[0, :], X1[0, :] - 1, 'r--', label='Decision Boundary: $X_1
- X_2 = 1$')
plt.xlabel('$X_1$')
plt.ylabel('$X_2$')
plt.title('Decision Boundary for $X_1$ AND NOT $X_2$ with Threshold
1')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True)
plt.legend()
plt.show()
```



Implement NOR with 2-variables and threshold 0.

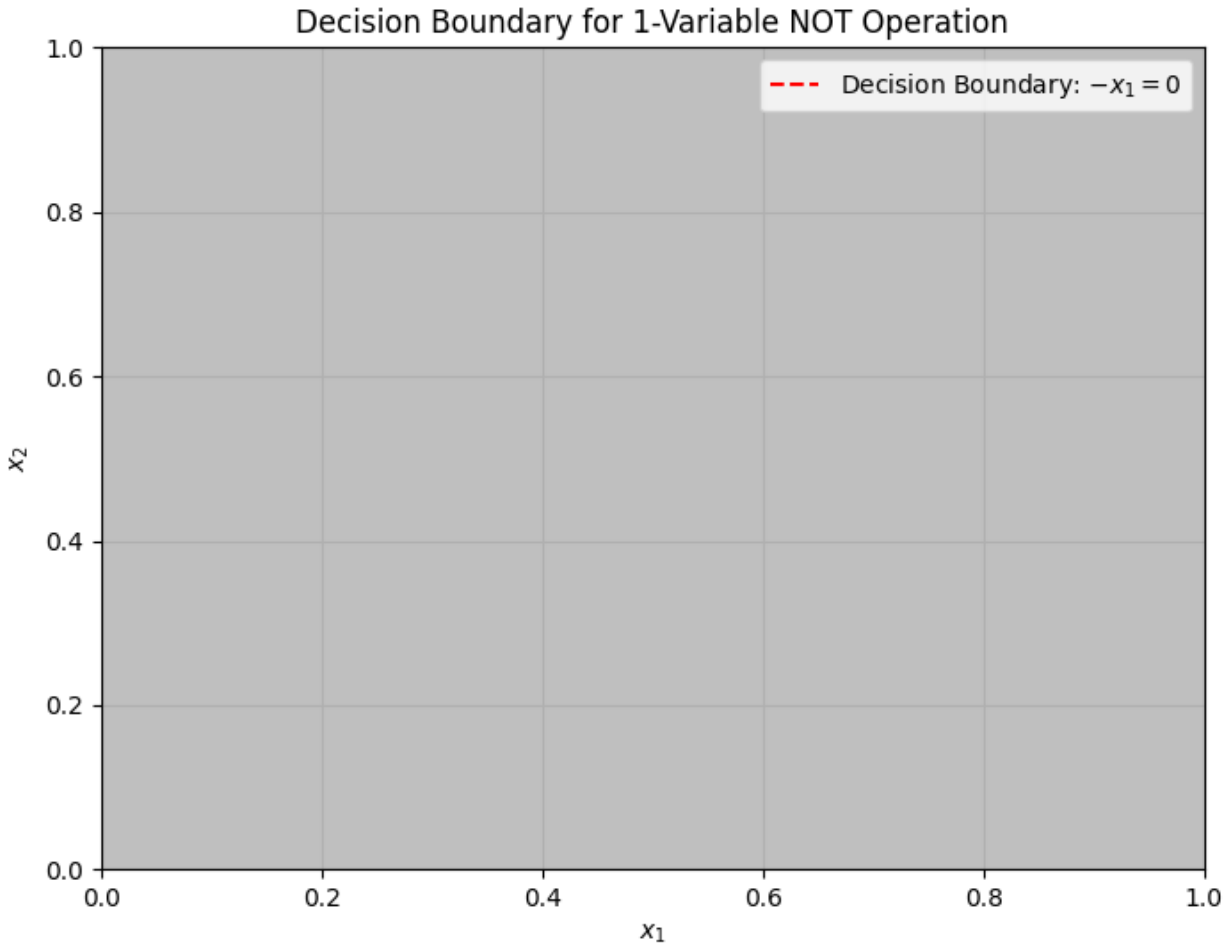
```
x1 = np.linspace(0, 1, 100)
x2 = np.linspace(0, 1, 100)
```

```
X1, X2 = np.meshgrid(x1, x2)
weighted_sum = -X1 - X2
decision_boundary = weighted_sum >= 0
plt.figure(figsize=(8, 6))
plt.contourf(X1, X2, decision_boundary, alpha=0.5, cmap='gray',
levels=[0, 1])
plt.plot(X1[0, :], -X1[0, :] + 0, 'r--', label='Decision Boundary: $-
x_1 - x_2 = 0$')
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('Decision Boundary for 2-Variable NOR Operation')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True)
plt.legend()
plt.show()
```



Implement NOT with 2-variables and threshold 0.

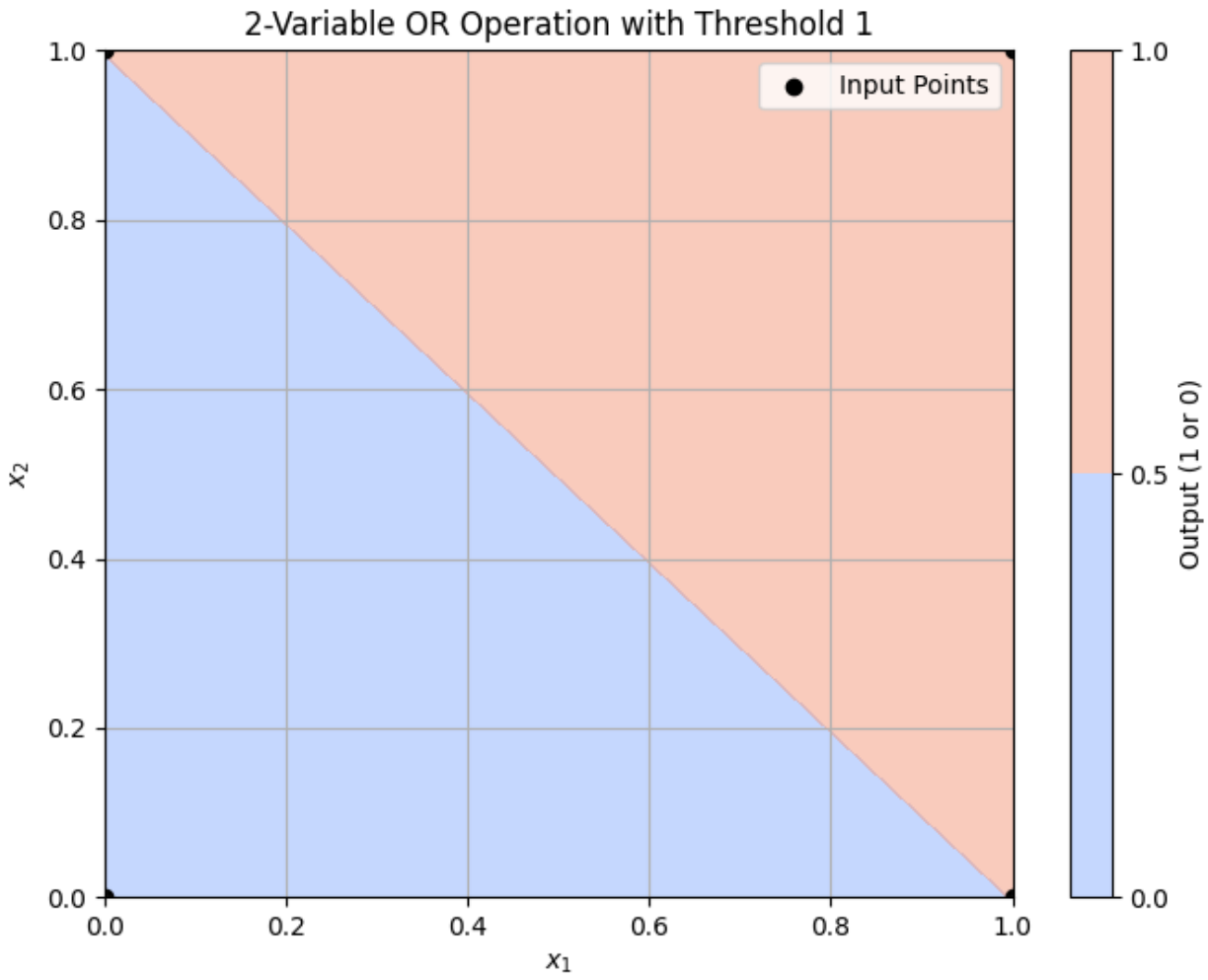
```
x1 = np.linspace(0, 1, 100)
x2 = np.linspace(0, 1, 100)
X1, X2 = np.meshgrid(x1, x2)
weighted_sum = -X1
decision_boundary = weighted_sum >= 0
plt.figure(figsize=(8, 6))
plt.contourf(X1, X2, decision_boundary, alpha=0.5, cmap='gray',
levels=[0, 1])
plt.plot(X1[0, :], -X1[0, :] + 0, 'r--', label='Decision Boundary: $-
x_1 = 0$')
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('Decision Boundary for 1-Variable NOT Operation')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True)
plt.legend()
plt.show()
```



Implement OR with 2-variables and threshold 1.

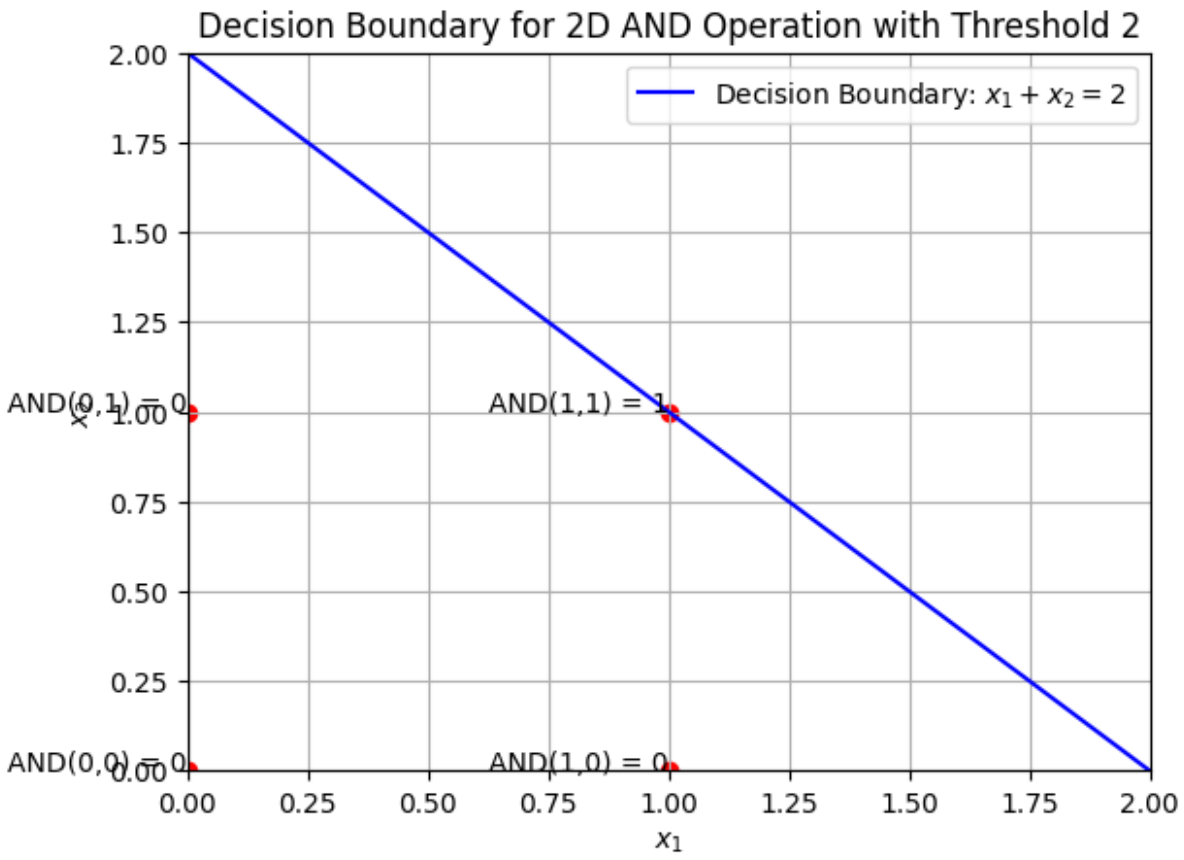
```
def or_gate(x1, x2):

    return 1 if (x1 + x2) >= 1 else 0

# Generate input values for x1 and x2
input_combinations = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

# Calculate the output for each combination of x1 and x2
outputs = np.array([or_gate(x1, x2) for x1, x2 in input_combinations])

# Display the results
for (x1, x2), output in zip(input_combinations, outputs):
    print(f"OR({x1}, {x2}) = {output}")

# Plotting the decision boundary
# Create a grid for x1 and x2
x1 = np.linspace(0, 1, 100)
x2 = np.linspace(0, 1, 100)
X1, X2 = np.meshgrid(x1, x2)

# Calculate the decision boundary
Z = (X1 + X2) >= 1

# Plotting
plt.figure(figsize=(8, 6))
plt.contourf(X1, X2, Z, alpha=0.5, cmap='coolwarm')
plt.colorbar(label='Output (1 or 0)')
plt.scatter(input_combinations[:, 0], input_combinations[:, 1],
color='black', label='Input Points')

# Set labels and title
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('2-Variable OR Operation with Threshold 1')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.axhline(0, color='black', lw=0.5, ls='--')
plt.axvline(0, color='black', lw=0.5, ls='--')
plt.legend()
plt.grid()

# Show the plot
plt.show()
```

```
OR(0, 0) = 0
OR(0, 1) = 1
OR(1, 0) = 1
OR(1, 1) = 1
```



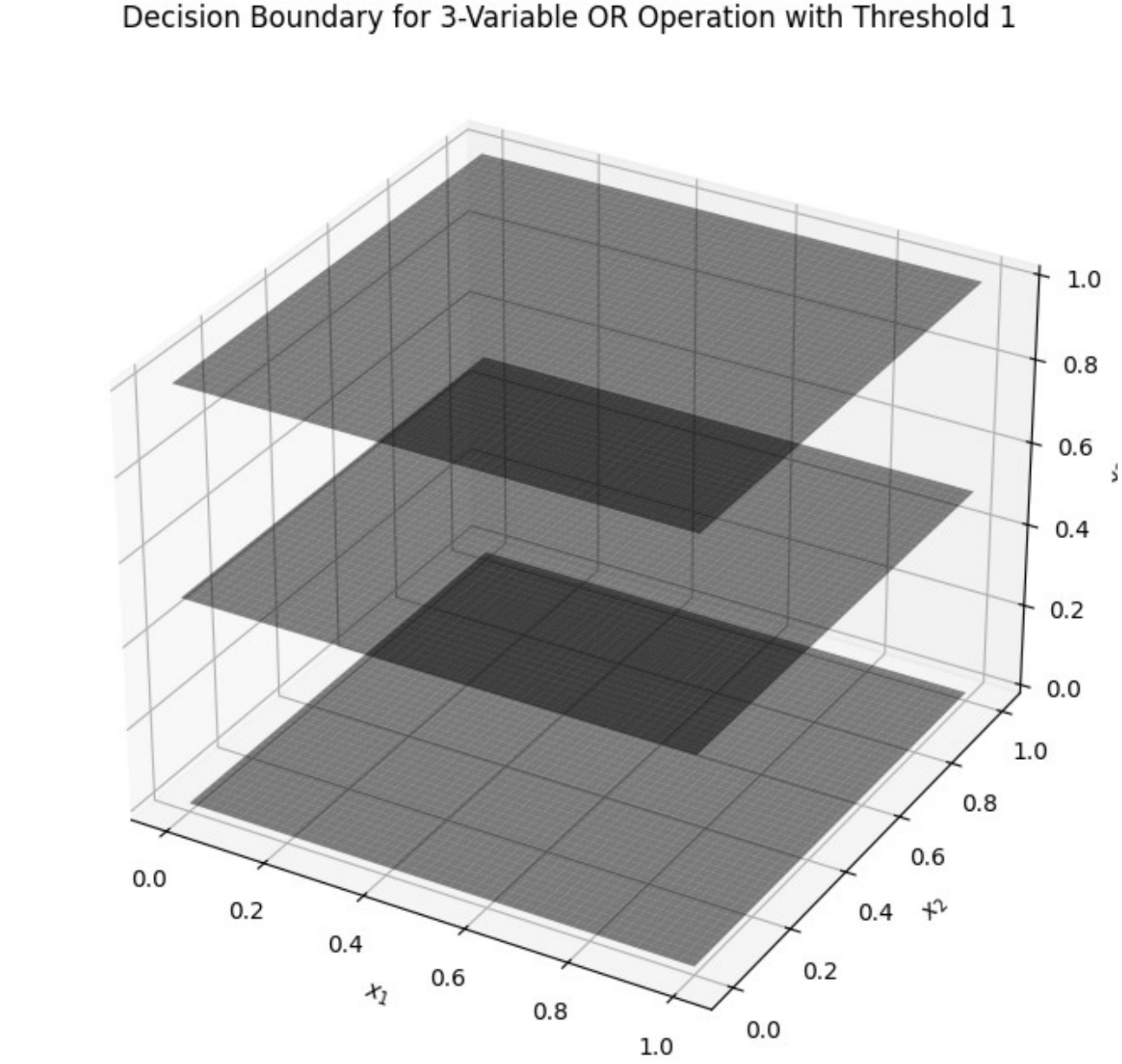
Implement AND with 2-variables and threshold 2.

```
x1 = np.linspace(0, 2, 100)
x2 = 2 - x1
plt.plot(x1, x2, label='Decision Boundary: $x_1 + x_2 = 2$',
color='blue')
plt.scatter([0, 0, 1, 1], [0, 1, 0, 1], color='red')
plt.text(0, 0, 'AND(0,0) = 0', fontsize=10, ha='right')
plt.text(1, 0, 'AND(1,0) = 0', fontsize=10, ha='right')
plt.text(0, 1, 'AND(0,1) = 0', fontsize=10, ha='right')
plt.text(1, 1, 'AND(1,1) = 1', fontsize=10, ha='right')
plt.xlim(0, 2)
plt.ylim(0, 2)
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('Decision Boundary for 2D AND Operation with Threshold 2')
plt.legend()
plt.grid(True)
plt.show()
```



Implement OR with 3-variables and threshold 1.

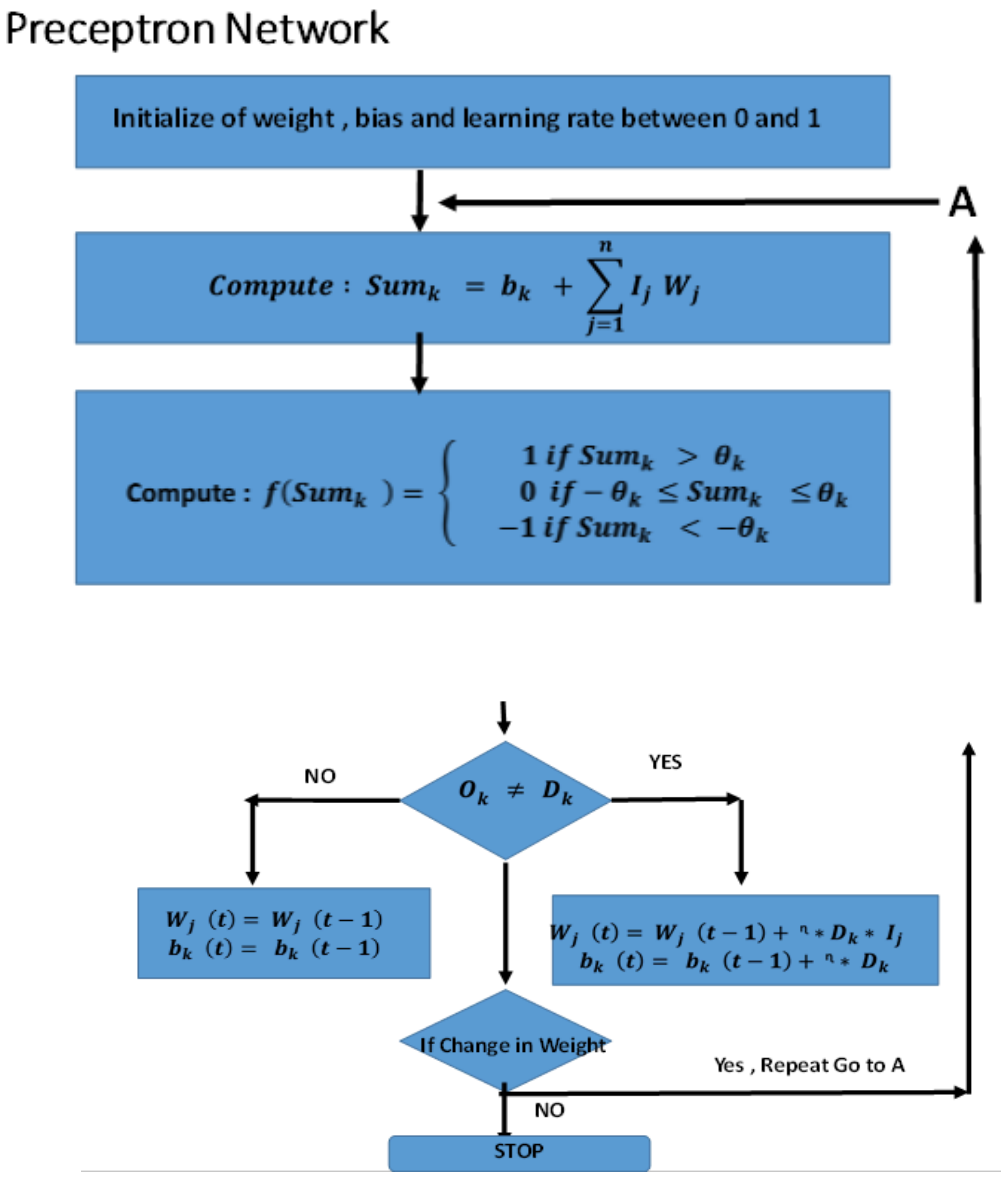
```
x = np.linspace(0, 1, 100)
X1, X2 = np.meshgrid(x, x)
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
x3_values = [0, 0.5, 1]
for x3 in x3_values:
    Z = (X1 + X2) >= (1 - x3)
    ax.plot_surface(X1, X2, x3 * np.ones_like(X1),
                    facecolors=plt.cm.gray(Z), alpha=0.5,
edgecolor='none')
ax.set_xlabel('$x_1$')
ax.set_ylabel('$x_2$')
ax.set_zlabel('$x_3$')
ax.set_title('Decision Boundary for 3-Variable OR Operation with
Threshold 1')
plt.show()
```


Decision Boundary for 3-Variable OR Operation with Threshold 1
0.0
0.2
0.4
0.6
0.8
1.0
x_1
x_2

Assignment 5

Q1. Implement the OR and AND problem using perceptron learning model.

Algorithm:



- transfer function used in the output neuron of the perceptron

- $f(Sum_k\ ) = \begin{cases} 1\ if\ Sum_k\ > \theta_k \\ 0\ \ if - \theta_k \leq Sum_k\ \leq \theta_k \\ -1\ if\ Sum_k\ < -\theta_k \end{cases}$

- θ_k =0.5 , W1=0, W2=1, n=1
- Then the weights will be updated

Implement AND and OR.

In each iteration outputs updated weights and t-y and output.

Q2. Implement the same with vector notation.

Where $y=g(W^TX)$ and $W^T=W^T(old)+\alpha(t-y).X^T$

Q3. Download the Fashion MNIST dataset from https://github.com/zalandoresearch/fashion-mnist and develop an image classification

Q1. Implement the OR and AND problem using perceptron learning model.

```
print("OR PROBLEM")
def g(x):
    if x > 0 :
        return 1
    else : return 0
inputs = [[0,0], [0,1], [1,0], [1,1]]
outputs = [0, 1, 1, 1]
weights = [0.5, 0, 1]
n = 0.25
old_weights = []
iterations = 0
while old_weights!=weights:
    iterations+=1
    old_weights = weights.copy()
    for i in range(4):
        inp, op = inputs[i], outputs[i]
        x, y = inp[0], inp[1]
        val = g(weights[0] + weights[1]*x + weights[2]*y)
        if (val!=op):
            weights[0] = weights[0] + n*(op - val)*1
            weights[1] = weights[1] + n*(op - val)*x
            weights[2] = weights[2] + n*(op - val)*y
        print(weights, '\t', val)
    print(old_weights,'\t', weights, '\n')
print("\nNumber of Iterations are : ", iterations)
```

```
OR PROBLEM
[0.25, 0.0, 1.0]        1
[0.25, 0.0, 1.0]        1
[0.25, 0.0, 1.0]        1
[0.25, 0.0, 1.0]        1
[0.5, 0, 1]       [0.25, 0.0, 1.0]

[0.0, 0.0, 1.0]   1
[0.0, 0.0, 1.0]   1
[0.25, 0.25, 1.0]       0
[0.25, 0.25, 1.0]       1
[0.25, 0.0, 1.0]        [0.25, 0.25, 1.0]

[0.0, 0.25, 1.0]        1
[0.0, 0.25, 1.0]        1
[0.0, 0.25, 1.0]        1
[0.0, 0.25, 1.0]        1
[0.25, 0.25, 1.0]       [0.0, 0.25, 1.0]

[0.0, 0.25, 1.0]        0
[0.0, 0.25, 1.0]        1
```

```
[0.0, 0.25, 1.0]       1
[0.0, 0.25, 1.0]       1
[0.0, 0.25, 1.0]       [0.0, 0.25, 1.0]


Number of Iterations are :  4
```

```
print("AND PROBLEM")
inputs = [[0,0], [0,1], [1,0], [1,1]]
outputs = [0, 0, 0, 1]
weights = [0.5, 0, 1]
n = 0.25
old_weights = []
iterations = 0
while old_weights!=weights:
    iterations+=1
    old_weights = weights.copy()
    for i in range(4):
        inp, op = inputs[i], outputs[i]
        x, y = inp[0], inp[1]
        val = g(weights[0] + weights[1]*x + weights[2]*y)
        if (val!=op):
            weights[0] = weights[0] + n*(op - val)*1
            weights[1] = weights[1] + n*(op - val)*x
            weights[2] = weights[2] + n*(op - val)*y
        print(weights, '\t', val)
    print(old_weights,'\t', weights, '\n')
print("\nNumber of Iterations are : ", iterations)
```

```
AND PROBLEM
[0.25, 0.0, 1.0]       1
[0.0, 0.0, 0.75]       1
[0.0, 0.0, 0.75]       0
[0.0, 0.0, 0.75]       1
[0.5, 0, 1]       [0.0, 0.0, 0.75]

[0.0, 0.0, 0.75]       0
[-0.25, 0.0, 0.5]      1
[-0.25, 0.0, 0.5]      0
[-0.25, 0.0, 0.5]      1
[0.0, 0.0, 0.75]       [-0.25, 0.0, 0.5]

[-0.25, 0.0, 0.5]      0
[-0.5, 0.0, 0.25]      1
[-0.5, 0.0, 0.25]      0
[-0.25, 0.25, 0.5]     0
[-0.25, 0.0, 0.5]      [-0.25, 0.25, 0.5]

[-0.25, 0.25, 0.5]     0
[-0.5, 0.25, 0.25]     1
```

```
[-0.5, 0.25, 0.25]     0
[-0.25, 0.5, 0.5]      0
[-0.25, 0.25, 0.5]     [-0.25, 0.5, 0.5]

[-0.25, 0.5, 0.5]      0
[-0.5, 0.5, 0.25]      1
[-0.5, 0.5, 0.25]      0
[-0.5, 0.5, 0.25]      1
[-0.25, 0.5, 0.5]      [-0.5, 0.5, 0.25]

[-0.5, 0.5, 0.25]      0
[-0.5, 0.5, 0.25]      0
[-0.5, 0.5, 0.25]      0
[-0.5, 0.5, 0.25]      1
[-0.5, 0.5, 0.25]      [-0.5, 0.5, 0.25]


Number of Iterations are :  6
```

Q2. Implement same with vector notation where: y=g(WTX) and WT=WT(old)+α(t-y).XT

```
import numpy as np
weights = np.array(((0.5, -0.5), (-0.5, 0), (0.4, -0.2), (-0.1, -
0.1)))
X = np.array(((1, ), (2, ), (1, ), (3, )))
output = (((1,), (0,)))
n = 0.3
old_weights = np.array(())
activate = np.vectorize(g)
iterations = 1
print(weights.T, '\n')
while old_weights.all()!=weights.all():
    iterations+=1
    old_weights = weights.copy()
    y = activate(np.dot(weights.T, X))
    weights = weights + (n*(output - y)*X.T).T
    print(weights.T, '\n')
print("Number of iterations : ", iterations)
```

```
[[ 0.5 -0.5  0.4 -0.1]
 [-0.5  0.  -0.2 -0.1]]

[[ 0.8  0.1  0.7  0.8]
 [-0.5  0.  -0.2 -0.1]]

Number of iterations :  2
```

Q3. Download the Fashion MNIST dataset from https://github.com/zalandoresearch/fashion-mnist and develop an image classification

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, datasets, models
(train_images, train_labels), (test_images, test_labels) =
datasets.fashion_mnist.load_data()
train_images = train_images / 255.0
test_images = test_images / 255.0
train_images = train_images.reshape((train_images.shape[0], 28, 28,
1))
test_images = test_images.reshape((test_images.shape[0], 28, 28, 1))
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28,
1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(train_images, train_labels, epochs=5,
validation_data=(test_images, test_labels))
history=model.fit(train_images, train_labels, epochs=5,
validation_data=(test_images, test_labels))
plt.plot((history.history['accuracy']))
plt.plot((history.history['val_accuracy']))
plt.plot((history.history['loss']))
plt.plot((history.history['val_loss']))
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['accuracy', 'val_accuracy','loss','val_loss'], loc='upper
left')
plt.show()
test_loss, test_accuracy = model.evaluate(test_images, test_labels)
print(f'\nTest accuracy: {test_accuracy:.4f}')
model.summary()
```

```
C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\site-
packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning:
Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 61s 29ms/step - accuracy: 0.7458 -
loss: 0.6877 - val_accuracy: 0.8545 - val_loss: 0.3944
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 46s 25ms/step - accuracy: 0.8770 -
loss: 0.3343 - val_accuracy: 0.8769 - val_loss: 0.3291
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 48s 26ms/step - accuracy: 0.8957 -
loss: 0.2811 - val_accuracy: 0.8922 - val_loss: 0.2931
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 47s 25ms/step - accuracy: 0.9088 -
loss: 0.2489 - val_accuracy: 0.8914 - val_loss: 0.2985
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 43s 23ms/step - accuracy: 0.9176 -
loss: 0.2213 - val_accuracy: 0.8939 - val_loss: 0.2996
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.9258 - loss:
0.2017 - val_accuracy: 0.8983 - val_loss: 0.2870
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 20s 6ms/step - accuracy: 0.9313 - loss:
0.1847 - val_accuracy: 0.9035 - val_loss: 0.2891
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.9393 - loss:
0.1665 - val_accuracy: 0.9055 - val_loss: 0.2755
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9438 - loss:
0.1477 - val_accuracy: 0.9062 - val_loss: 0.2757
Epoch 5/5
 336/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 7ms/step - accuracy: 0.9496 - loss:
0.1357

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
predictions = model.predict(test_images)
predicted_labels = np.argmax(predictions, axis=1)
accuracy = accuracy_score(test_labels, predicted_labels)
print(f'Test accuracy: {accuracy:.4f}')
print(classification_report(test_labels, predicted_labels))
conf_matrix = confusion_matrix(test_labels, predicted_labels)
print('Confusion Matrix:')
print(conf_matrix)

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
plt.figure(figsize=(10,10))
for i in range(25):
```

```
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])
plt.show()

def plot_image(i, predictions_array, true_label, img):
    true_label, img = true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(img, cmap=plt.cm.binary)
    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = 'blue'
    else:
        color = 'red'
    plt.xlabel("{} {:2.0f}% ({})".format(class_names[predicted_label],
                                100*np.max(predictions_array),
                                class_names[true_label]),
                                color=color)
def plot_value_array(i, predictions_array, true_label):
    true_label = true_label[i]
    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)

    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('blue')
num_rows = 5
num_cols = 3
num_images = num_rows*num_cols
plt.figure(figsize=(2*2*num_cols, 2*2*num_rows))
for i in range(num_images):
    plt.subplot(num_rows, 2*num_cols, 2*i+1)
    plot_image(i, predictions[i], test_labels, test_images)
    plt.subplot(num_rows, 2*num_cols, 2*i+2)
    plot_value_array(i, predictions[i], test_labels)
plt.tight_layout()
plt.show()
```

Assignment 6

\

Classification of brain MRI using hyper column technique with convolutional neural network and feature selection method

1. The dataset consists of open-access brain tumor MRI containing two classes of the tumor and normal.The dataset consists of 155 and 98 tumor and normal brain MRI, respectively. The dataset is heterogeneous MR images collected from 253 patients.

The images in the dataset has not high resolution and each image has a different resolution. The image format is JPEG.

2. Print the Normal class sample and tumor class sample.

3.Do as follows given in the model below,and compare the results.

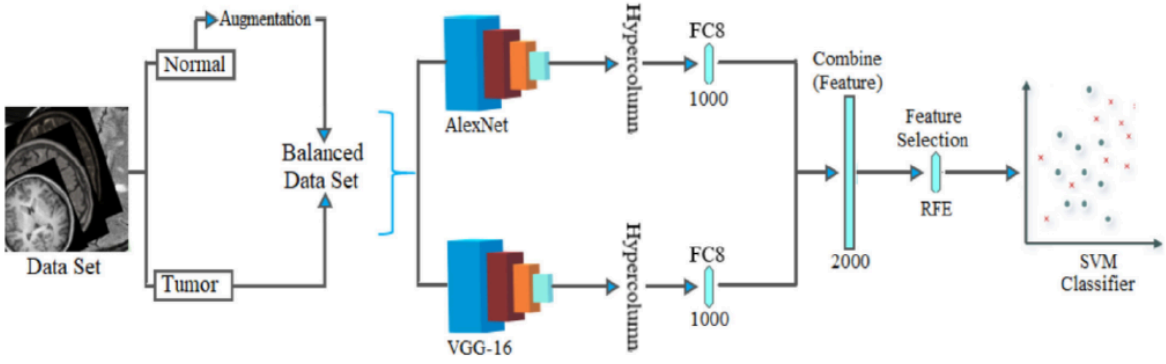


Fig. 8. The overall block diagram of the proposed model.

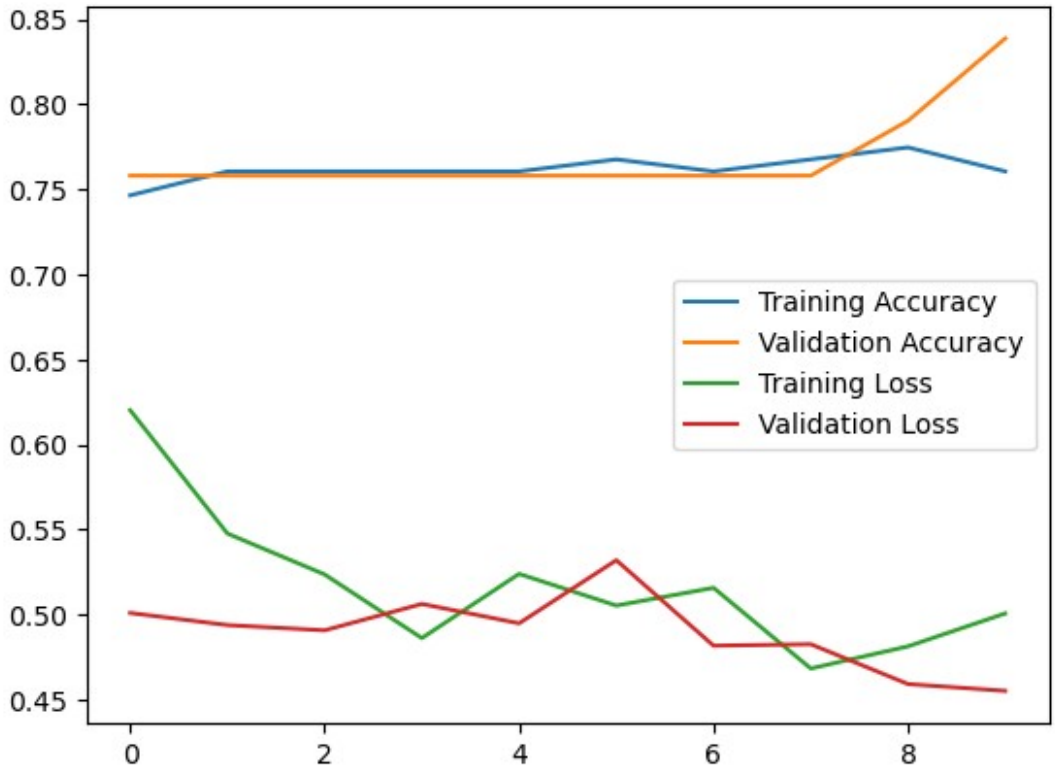
```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout
from tensorflow.keras.applications import VGG16

dataset_dir = 'images'
img_size = (180, 180)
batch_size = 32
tumor_images = []
no_tumor_images = []

for filename in os.listdir(dataset_dir):
    if filename.lower().startswith('y'):
        tumor_images.append(os.path.join(dataset_dir, filename))
    elif filename.lower().startswith('n'):
        no_tumor_images.append(os.path.join(dataset_dir, filename))

train_tumor_images, val_tumor_images =
tumor_images[:int(0.7*len(tumor_images))],
tumor_images[int(0.7*len(tumor_images)):]
train_no_tumor_images, val_no_tumor_images =
no_tumor_images[:int(0.7*len(no_tumor_images))],
no_tumor_images[int(0.7*len(no_tumor_images)):]

def load_image(file):
    img = tf.io.read_file(file)
    img = tf.image.decode_image(img, channels=3)
    img = tf.image.resize(img, img_size)
    img = img / 255.0
    return img

train_images = [load_image(file) for file in train_tumor_images +
train_no_tumor_images]
train_labels = [1] * len(train_tumor_images) + [0] *
len(train_no_tumor_images)
validation_images = [load_image(file) for file in val_tumor_images +
val_no_tumor_images]
validation_labels = [1] * len(val_tumor_images) + [0] *
len(val_no_tumor_images)

train_datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
```

```
    horizontal_flip=True,
    fill_mode='nearest'
)

validation_datagen = ImageDataGenerator()

train_dataset = train_datagen.flow(
    np.array(train_images),
    np.array(train_labels),
    batch_size=batch_size
)

validation_dataset = validation_datagen.flow(
    np.array(validation_images),
    np.array(validation_labels),
    batch_size=batch_size
)

# CNN
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(180, 180, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.2),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
history = model.fit(train_dataset, epochs=10,
validation_data=validation_dataset)
```

```
C:\Users\pranj\anaconda3\Lib\site-packages\keras\src\layers\
convolutional\base_conv.py:107: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Epoch 1/10

C:\Users\pranj\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:122: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
```

```
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
  self._warn_if_super_not_called()

5/5 ━━━━━━━━━━━━━━━━━━━━ 5s 526ms/step - accuracy: 0.7473 - loss:
0.6608 - val_accuracy: 0.7581 - val_loss: 0.5011
Epoch 2/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 478ms/step - accuracy: 0.7929 - loss:
0.5268 - val_accuracy: 0.7581 - val_loss: 0.4939
Epoch 3/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 600ms/step - accuracy: 0.7586 - loss:
0.5063 - val_accuracy: 0.7581 - val_loss: 0.4909
Epoch 4/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 541ms/step - accuracy: 0.7309 - loss:
0.5097 - val_accuracy: 0.7581 - val_loss: 0.5064
Epoch 5/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 523ms/step - accuracy: 0.8106 - loss:
0.4416 - val_accuracy: 0.7581 - val_loss: 0.4950
Epoch 6/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 494ms/step - accuracy: 0.7642 - loss:
0.4777 - val_accuracy: 0.7581 - val_loss: 0.5321
Epoch 7/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 499ms/step - accuracy: 0.7930 - loss:
0.5001 - val_accuracy: 0.7581 - val_loss: 0.4818
Epoch 8/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 554ms/step - accuracy: 0.7468 - loss:
0.5038 - val_accuracy: 0.7581 - val_loss: 0.4827
Epoch 9/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 635ms/step - accuracy: 0.7787 - loss:
0.4666 - val_accuracy: 0.7903 - val_loss: 0.4593
Epoch 10/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 531ms/step - accuracy: 0.7433 - loss:
0.5408 - val_accuracy: 0.8387 - val_loss: 0.4553
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.show()

image_path = os.path.join(os.getcwd(), 'images', 'Y10.jpg')
image_string = tf.io.read_file(image_path)
new_image = tf.image.decode_jpeg(image_string, channels=3)
new_image = tf.image.resize(new_image, img_size)
new_image = new_image / 255.0
new_image = new_image[tf.newaxis, ...]

prediction = model.predict(new_image)
print('Prediction:', prediction)
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step
Prediction: [[0.9548318]]
```

```
dataset_dir = 'images'
img_size = (180, 180)
batch_size = 32

tumor_images = []
no_tumor_images = []

for filename in os.listdir(dataset_dir):
    if filename.lower().startswith('y'):
        tumor_images.append(os.path.join(dataset_dir, filename))
    elif filename.lower().startswith('n'):
        no_tumor_images.append(os.path.join(dataset_dir, filename))

train_tumor_images = tumor_images[:int(0.8 * len(tumor_images))]
val_tumor_images = tumor_images[int(0.8 * len(tumor_images)):]
train_no_tumor_images = no_tumor_images[:int(0.8 *
len(no_tumor_images))]
val_no_tumor_images = no_tumor_images[int(0.8 *
len(no_tumor_images)):]

train_datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
```

```
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

validation_datagen = ImageDataGenerator()

train_dataset = train_datagen.flow(
    np.array(train_images),
    np.array(train_labels),
    batch_size=batch_size
)

validation_dataset = validation_datagen.flow(
    np.array(validation_images),
    np.array(validation_labels),
    batch_size=batch_size
)
```

AlexNet Model

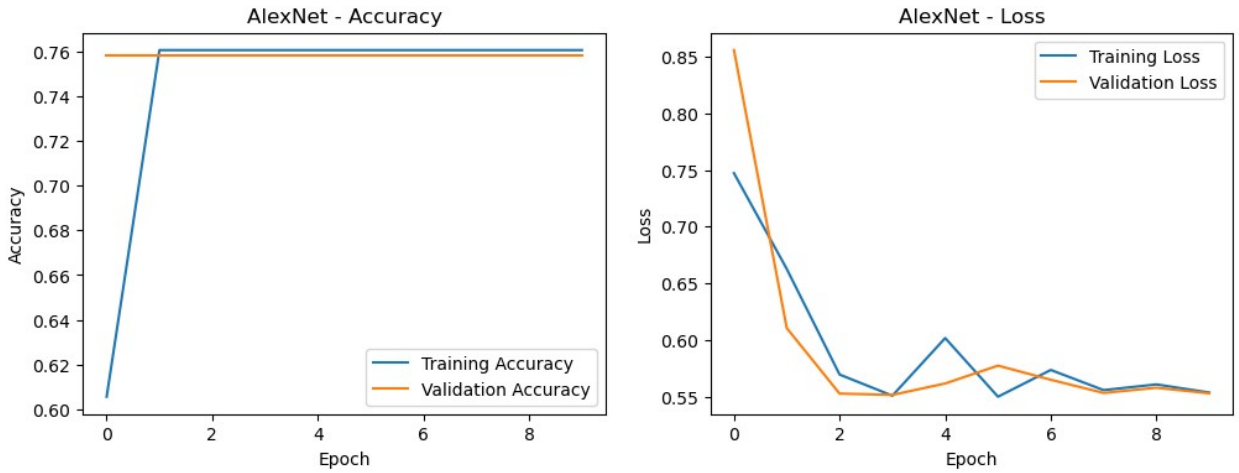
```
alexnet_model = Sequential([
    Conv2D(96, (11, 11), strides=4, activation='relu',
input_shape=(180, 180, 3)),
    MaxPooling2D((3, 3), strides=2),
    Conv2D(256, (5, 5), padding='same', activation='relu'),
    MaxPooling2D((3, 3), strides=2),
    Conv2D(384, (3, 3), padding='same', activation='relu'),
    Conv2D(384, (3, 3), padding='same', activation='relu'),
    Conv2D(256, (3, 3), padding='same', activation='relu'),
    MaxPooling2D((3, 3), strides=2),
    Flatten(),
    Dense(4096, activation='relu'),
    Dropout(0.5),
    Dense(4096, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

alexnet_model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
alexnet_history = alexnet_model.fit(train_dataset, epochs=10,
validation_data=validation_dataset)
```

```
C:\Users\pranj\anaconda3\Lib\site-packages\keras\src\layers\
convolutional\base_conv.py:107: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
```

```
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Epoch 1/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.5921 - loss: 0.7885
- val_accuracy: 0.7581 - val_loss: 0.8558
Epoch 2/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 6s 984ms/step - accuracy: 0.7456 - loss:
0.7222 - val_accuracy: 0.7581 - val_loss: 0.6105
Epoch 3/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.7683 - loss: 0.5766
- val_accuracy: 0.7581 - val_loss: 0.5527
Epoch 4/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 7s 1s/step - accuracy: 0.7480 - loss: 0.5603
- val_accuracy: 0.7581 - val_loss: 0.5516
Epoch 5/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 7s 1s/step - accuracy: 0.7668 - loss: 0.5449
- val_accuracy: 0.7581 - val_loss: 0.5616
Epoch 6/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 6s 1s/step - accuracy: 0.7586 - loss: 0.5425
- val_accuracy: 0.7581 - val_loss: 0.5774
Epoch 7/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 6s 1s/step - accuracy: 0.7454 - loss: 0.5838
- val_accuracy: 0.7581 - val_loss: 0.5649
Epoch 8/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.7286 - loss: 0.5866
- val_accuracy: 0.7581 - val_loss: 0.5533
Epoch 9/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.7835 - loss: 0.5300
- val_accuracy: 0.7581 - val_loss: 0.5579
Epoch 10/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 6s 1s/step - accuracy: 0.7938 - loss: 0.5099
- val_accuracy: 0.7581 - val_loss: 0.5530
```

```python
def plot_training_history(history, model_name):
    plt.figure(figsize=(12, 4))

    # Plot 1
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Training Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation
Accuracy')
    plt.title(f'{model_name} - Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    # Plot 2
    plt.subplot(1, 2, 2)
```

```
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title(f'{model_name} - Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.show()

plot_training_history(alexnet_history, 'AlexNet')
```



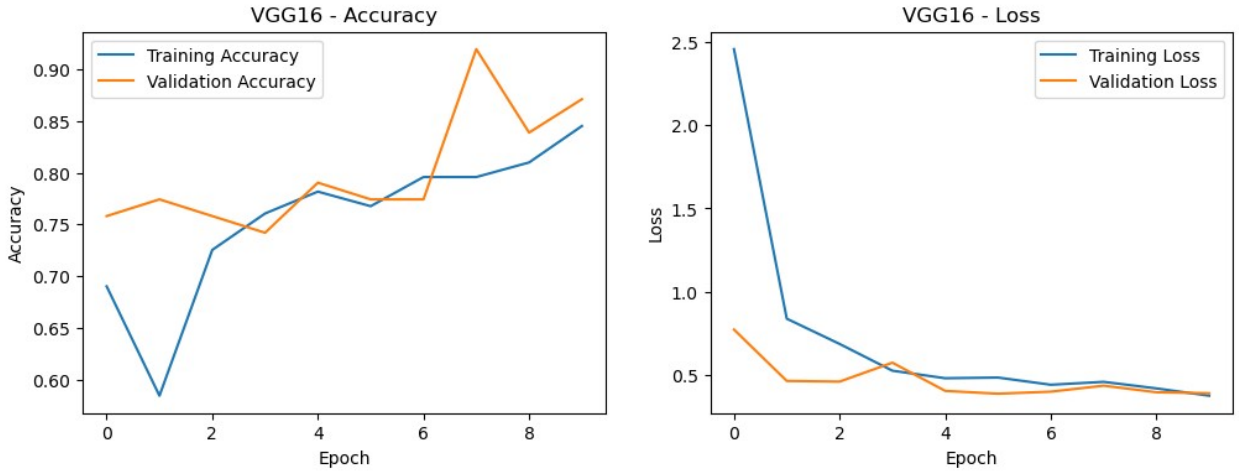
VGG16

```
vgg16_base = VGG16(weights='imagenet', include_top=False,
input_shape=(180, 180, 3))
for layer in vgg16_base.layers:
    layer.trainable = False

vgg16_model = Sequential([
    vgg16_base,
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

vgg16_model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
vgg16_history = vgg16_model.fit(train_dataset, epochs=10,
validation_data=validation_dataset)
```

```
Epoch 1/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 25s 5s/step - accuracy: 0.5929 - loss: 2.3551
- val_accuracy: 0.7581 - val_loss: 0.7729
Epoch 2/10
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 28s 6s/step - accuracy: 0.6697 - loss: 0.7186
- val_accuracy: 0.7742 - val_loss: 0.4641
Epoch 3/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 27s 5s/step - accuracy: 0.7140 - loss: 0.6636
- val_accuracy: 0.7581 - val_loss: 0.4600
Epoch 4/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 23s 5s/step - accuracy: 0.7784 - loss: 0.5030
- val_accuracy: 0.7419 - val_loss: 0.5739
Epoch 5/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 24s 5s/step - accuracy: 0.7592 - loss: 0.4913
- val_accuracy: 0.7903 - val_loss: 0.4043
Epoch 6/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 24s 5s/step - accuracy: 0.7179 - loss: 0.5517
- val_accuracy: 0.7742 - val_loss: 0.3874
Epoch 7/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 23s 5s/step - accuracy: 0.7935 - loss: 0.4438
- val_accuracy: 0.7742 - val_loss: 0.3997
Epoch 8/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 25s 5s/step - accuracy: 0.7863 - loss: 0.4781
- val_accuracy: 0.9194 - val_loss: 0.4357
Epoch 9/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 28s 6s/step - accuracy: 0.7980 - loss: 0.4262
- val_accuracy: 0.8387 - val_loss: 0.3962
Epoch 10/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 28s 6s/step - accuracy: 0.8362 - loss: 0.3654
- val_accuracy: 0.8710 - val_loss: 0.3902
```

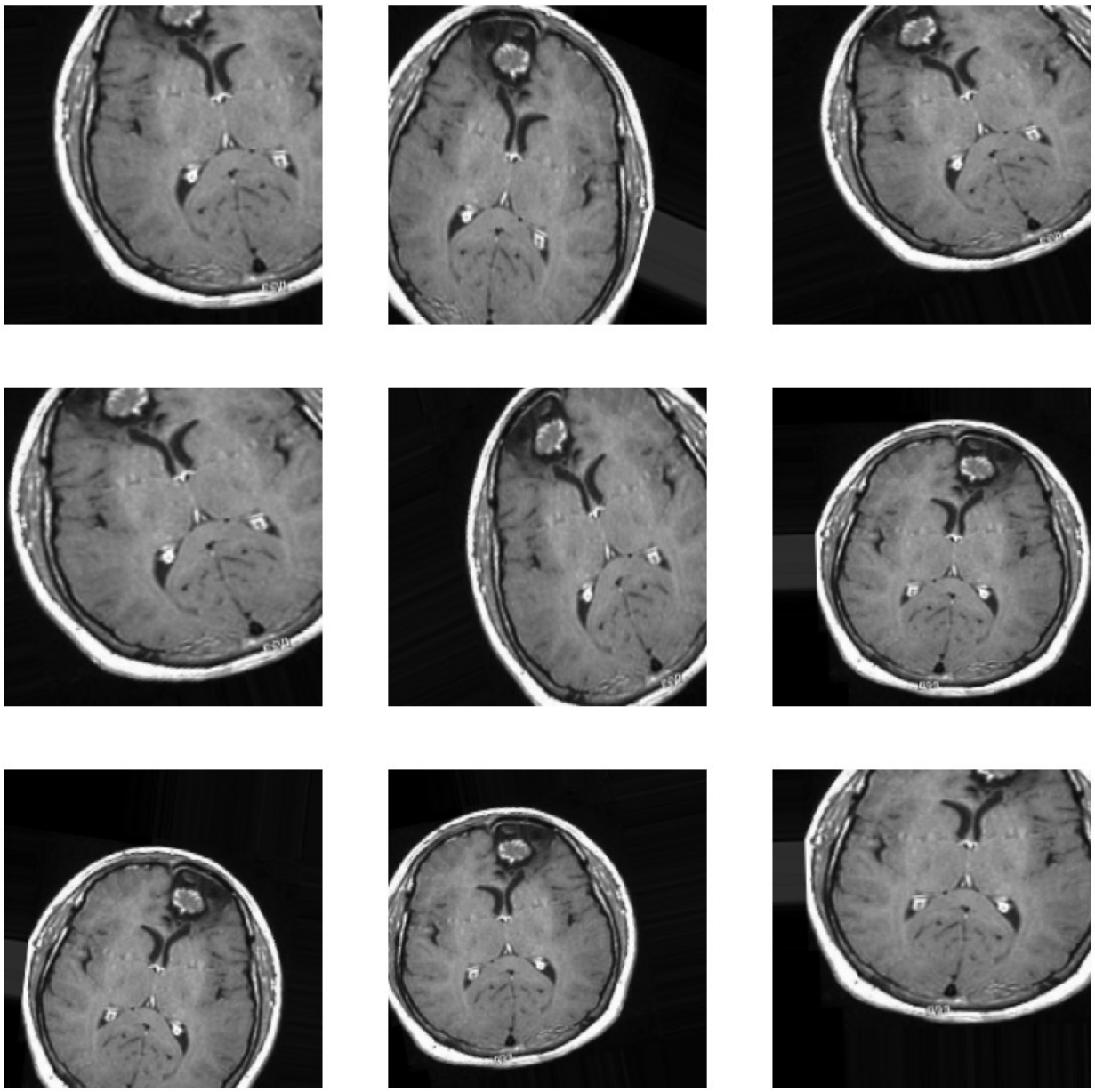
```
plot_training_history(vgg16_history, 'VGG16')
```



Augmentation

```
def display_augmented_images(datagen, image_path):
    if not os.path.exists(image_path):
        print(f"File not found: {image_path}")
        return
```

```
    image_string = tf.io.read_file(image_path)
    image = tf.image.decode_jpeg(image_string, channels=3)
    image = tf.image.resize(image, img_size)
    image = image / 255.0
    image = image[tf.newaxis, ...]

    augmented_images = datagen.flow(image, batch_size=1)

    plt.figure(figsize=(12, 12))
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        batch = next(augmented_images)
        augmented_image = batch[0]
        plt.imshow(augmented_image)
        plt.axis('off')

    plt.show()

image_to_display = 'Y12.jpg'

if image_to_display:
    display_augmented_images(train_datagen, os.path.join(dataset_dir,
image_to_display))
else:
    print("No tumor images found in the dataset.")
```



```
image_path = os.path.join(os.getcwd(), 'images', 'N11.jpg')
image_string = tf.io.read_file(image_path)
new_image = tf.image.decode_jpeg(image_string, channels=3)
new_image = tf.image.resize(new_image, img_size)
new_image = new_image / 255.0
new_image = new_image[tf.newaxis, ...]

vgg16_prediction = vgg16_model.predict(new_image)
alexnet_prediction = alexnet_model.predict(new_image)

print('VGG16 Prediction:', vgg16_prediction)
print('AlexNet Prediction:', alexnet_prediction)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 660ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step
VGG16 Prediction: [[0.65607154]]
AlexNet Prediction: [[0.75484836]]
```

Assignment 7

1. Implement Logistic Regression with Autoencoder on MNIST Dataset
2. Add some noise to images and then apply the autoencoder to denoise the dataset.

NAME - VISHAL MARANDI | ROLL - BTECH/10119/21 | DL ASSIGNMENT DATED 16th OCTOBER 2024

Implementing Logistic Regression with Autoencoder on MNIST DATASET

```
!pip install tensorflow
```

```
Requirement already satisfied: tensorflow in
/usr/local/lib/python3.10/dist-packages (2.17.0)
Requirement already satisfied: absl-py>=1.0.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (24.3.25)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1
in /usr/local/lib/python3.10/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: h5py>=3.10.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (3.11.0)
Requirement already satisfied: libclang>=13.0.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: ml-dtypes<0.5.0,>=0.3.1 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (0.4.1)
Requirement already satisfied: opt-einsum>=2.3.2 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (24.1)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!
=4.21.3,!=4.21.4,!=4.21.5,<5.0.0dev,>=3.20.3 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (3.20.3)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (2.32.3)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (71.0.4)
Requirement already satisfied: six>=1.12.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (2.5.0)
Requirement already satisfied: typing-extensions>=3.6.6 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (4.12.2)
Requirement already satisfied: wrapt>=1.11.0 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.16.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.64.1)
Requirement already satisfied: tensorboard<2.18,>=2.17 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (2.17.0)
Requirement already satisfied: keras>=3.2.0 in
```

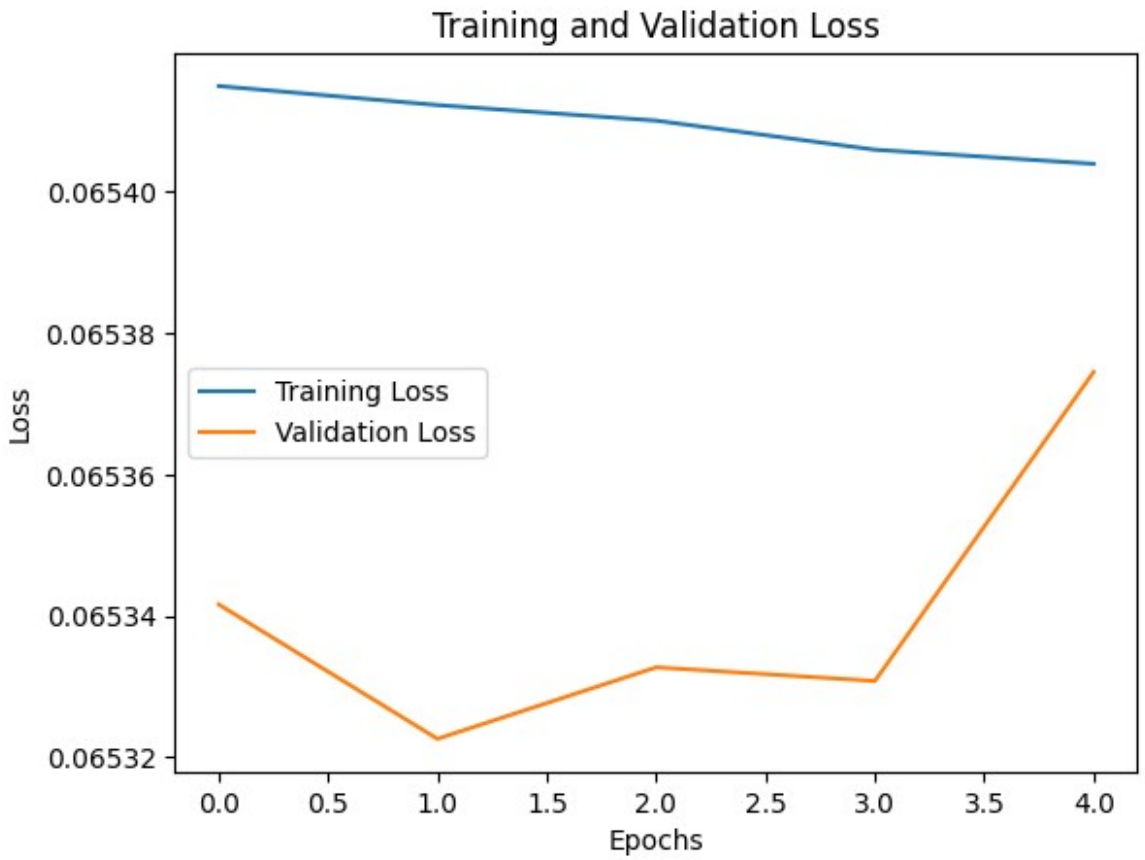
```
/usr/local/lib/python3.10/dist-packages (from tensorflow) (3.4.1)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (0.37.1)
Requirement already satisfied: numpy<2.0.0,>=1.23.5 in
/usr/local/lib/python3.10/dist-packages (from tensorflow) (1.26.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.10/dist-packages (from astunparse>=1.6.0-
>tensorflow) (0.44.0)
Requirement already satisfied: rich in /usr/local/lib/python3.10/dist-
packages (from keras>=3.2.0->tensorflow) (13.9.2)
Requirement already satisfied: namex in
/usr/local/lib/python3.10/dist-packages (from keras>=3.2.0-
>tensorflow) (0.0.8)
Requirement already satisfied: optree in
/usr/local/lib/python3.10/dist-packages (from keras>=3.2.0-
>tensorflow) (0.13.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (2024.8.30)
Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.10/dist-packages (from tensorboard<2.18,>=2.17-
>tensorflow) (3.7)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
in /usr/local/lib/python3.10/dist-packages (from
tensorboard<2.18,>=2.17->tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
/usr/local/lib/python3.10/dist-packages (from tensorboard<2.18,>=2.17-
>tensorflow) (3.0.4)
Requirement already satisfied: MarkupSafe>=2.1.1 in
/usr/local/lib/python3.10/dist-packages (from werkzeug>=1.0.1-
>tensorboard<2.18,>=2.17->tensorflow) (3.0.1)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0-
>tensorflow) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0-
>tensorflow) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.10/dist-packages (from markdown-it-py>=2.2.0-
>rich->keras>=3.2.0->tensorflow) (0.1.2)
```

```
#Importing Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Input,Dense,Flatten,Reshape
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

#Loading the data
(x_train,y_train),(x_test,y_test)=mnist.load_data()

#Preprocessing the data
x_train=x_train.astype('float32')/255
x_test=x_test.astype('float32')/255

#Flattening
x_train_flatten=x_train.reshape(-1,28*28)
x_test_flatten=x_test.reshape(-1,28*28)

# 3. Build Autoencoder Model
input_img = Input(shape=(28 * 28,))
encoded = Dense(128, activation='relu')(input_img)  # Change this to
128 features
decoded = Dense(28 * 28, activation='sigmoid')(encoded)
# Autoencoder model
autoencoder = Model(input_img, decoded)
# Encoder model to extract encoded features
encoder = Model(input_img, encoded)

# 4. Compile the Model
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
# 5. Train the Autoencoder
autoencoder.fit(x_train_flatten, x_train_flatten,
epochs=20,batch_size=256,shuffle=True,validation_data=(x_test_flatten,
x_test_flatten))
history=autoencoder.fit(x_train_flatten, x_train_flatten,
epochs=5,batch_size=256,shuffle=True,validation_data=(x_test_flatten,
x_test_flatten))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# 6. Use Encoded Features for Logistic Regression
```

```
x_train_encoded = encoder.predict(x_train_flatten)
x_test_encoded = encoder.predict(x_test_flatten)
logistic = LogisticRegression(max_iter=1000)
logistic.fit(x_train_encoded, y_train)
# Predictions
y_pred = logistic.predict(x_test_encoded)
# 7. Evaluate and Analyze
accuracy = accuracy_score(y_test, y_pred)
print(f'Logistic Regression accuracy on encoded features: {accuracy *
100:.2f}%')
```

```
Epoch 1/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 19ms/step - loss: 0.0656 - val_loss:
0.0654
Epoch 2/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 19ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 3/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 7s 30ms/step - loss: 0.0655 - val_loss:
0.0654
Epoch 4/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 7s 29ms/step - loss: 0.0655 - val_loss:
0.0654
Epoch 5/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 10s 29ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 6/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 7/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 11s 25ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 8/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 9/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 8s 35ms/step - loss: 0.0655 - val_loss:
0.0654
Epoch 10/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 11/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - loss: 0.0656 - val_loss:
0.0654
Epoch 12/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 13/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 8s 23ms/step - loss: 0.0653 - val_loss:
0.0654
```

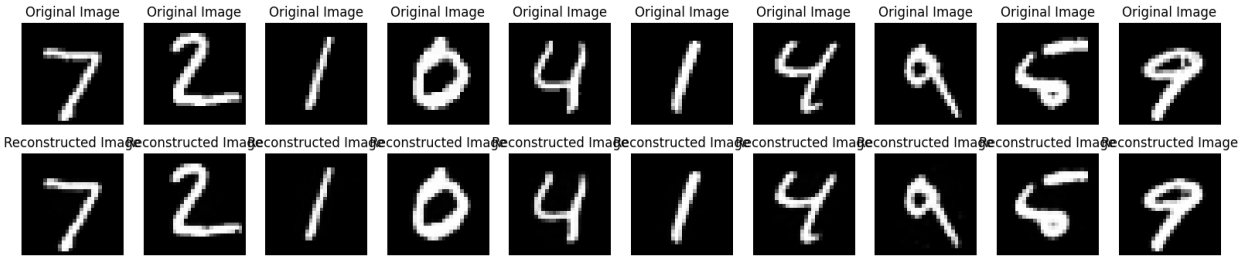
```
Epoch 14/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 7s 29ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 15/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - loss: 0.0654 - val_loss:
0.0653
Epoch 16/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 17ms/step - loss: 0.0654 - val_loss:
0.0653
Epoch 17/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 9s 38ms/step - loss: 0.0655 - val_loss:
0.0653
Epoch 18/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 7s 31ms/step - loss: 0.0654 - val_loss:
0.0654
Epoch 19/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - loss: 0.0653 - val_loss:
0.0653
Epoch 20/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - loss: 0.0653 - val_loss:
0.0653
Epoch 1/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0654 - val_loss:
0.0653
Epoch 2/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 8s 15ms/step - loss: 0.0655 - val_loss:
0.0653
Epoch 3/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 7s 24ms/step - loss: 0.0653 - val_loss:
0.0653
Epoch 4/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 10s 24ms/step - loss: 0.0654 - val_loss:
0.0653
Epoch 5/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 7s 27ms/step - loss: 0.0656 - val_loss:
0.0654
```




```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
Logistic Regression accuracy on encoded features: 92.24%

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/
_logistic.py:469: ConvergenceWarning: lbfgs failed to converge
(status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as
shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-
regression
  n_iter_i = _check_optimize_result(

decoded_imgs=autoencoder.predict(x_test_flatten)
x_test_reshaped=x_test_flatten.reshape(-1,28,28)
decoded_imgs=decoded_imgs.reshape(-1,28,28)
decoded_imgs_reshaped=decoded_imgs.reshape(-1,28,28)
n=10
```

```
plt.figure(figsize=(20,4))
for i in range(n):
  ax=plt.subplot(2,n,i+1)
  plt.imshow(x_test_reshaped[i],cmap='gray')
  plt.title("Original Image")
  plt.axis('off')
  ax=plt.subplot(2,n,i+1+n)
  plt.imshow(decoded_imgs_reshaped[i],cmap='gray')
  plt.title("Reconstructed Image")
  plt.axis('off')
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```

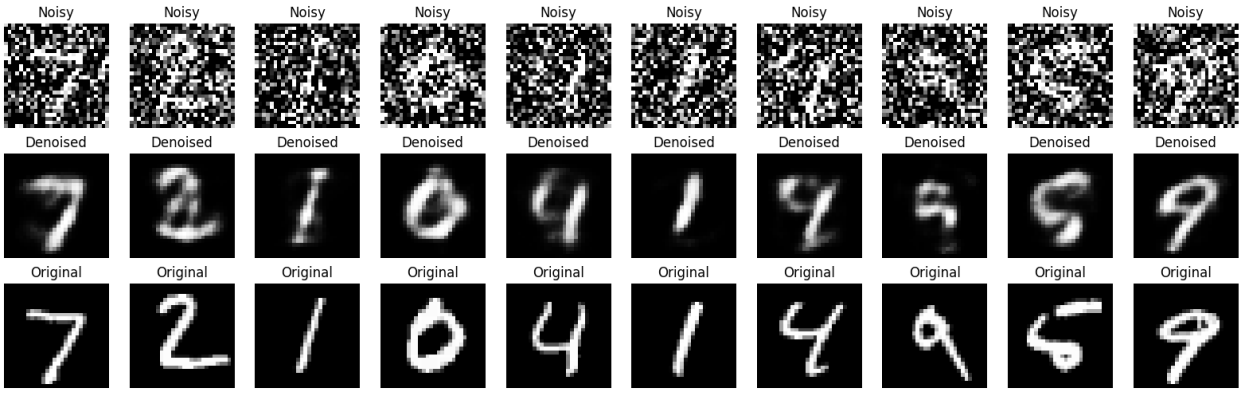



```
# 1. Import Libraries
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

# 2. Load and Preprocess the Dataset
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize the images to [0, 1]
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Flatten the images (to pass through the fully connected autoencoder)
x_train_flatten = x_train.reshape(-1, 28 * 28)
x_test_flatten = x_test.reshape(-1, 28 * 28)

# 3. Add noise to the dataset
noise_factor = 0.75  # Change this to increase or decrease noise level
x_train_noisy = x_train_flatten + noise_factor *
np.random.normal(loc=0.0, scale=1.0, size=x_train_flatten.shape)
x_test_noisy = x_test_flatten + noise_factor *
np.random.normal(loc=0.0, scale=1.0, size=x_test_flatten.shape)

# Clip the values to be between 0 and 1 (valid pixel range)
```

```
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# 4. Build Autoencoder Model
input_img = Input(shape=(28 * 28,))
encoded = Dense(128, activation='relu')(input_img)
decoded = Dense(28 * 28, activation='sigmoid')(encoded)

# Autoencoder model
autoencoder = Model(input_img, decoded)

# 5. Compile the Model
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# 6. Train the Autoencoder
history = autoencoder.fit(x_train_noisy, x_train_flatten,
                          epochs=5,
                          batch_size=256,
                          shuffle=True,
                          validation_data=(x_test_noisy,
x_test_flatten))

# 7. Predict Denoised Images
denoised_imgs = autoencoder.predict(x_test_noisy)

# Reshape the noisy and denoised images back to 28x28 for
visualization
x_test_noisy_reshaped = x_test_noisy.reshape(-1, 28, 28)
denoised_imgs_reshaped = denoised_imgs.reshape(-1, 28, 28)

# 8. Visualize Original Noisy and Denoised Images
n = 10  # number of images to display
plt.figure(figsize=(20, 6))
for i in range(n):
    # Display noisy images
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test_noisy_reshaped[i], cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Display denoised images
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(denoised_imgs_reshaped[i], cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

    # Display original images for comparison
    ax = plt.subplot(3, n, i + 1 + 2 * n)
    plt.imshow(x_test[i], cmap='gray')
    plt.title("Original")
```

```
    plt.axis('off')

plt.show()
```

```
Epoch 1/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.3215 - val_loss:
0.1912
Epoch 2/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.1845 - val_loss:
0.1669
Epoch 3/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.1648 - val_loss:
0.1568
Epoch 4/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step - loss: 0.1556 - val_loss:
0.1513
Epoch 5/5
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 20ms/step - loss: 0.1504 - val_loss:
0.1479
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```



Assignment 8

	Lab Assignment Deep Learning Lab
Q1(a)	You have to collect daily temperature (in degrees) of Mesra at 10 AM for last 60 days. Store data in temp.csv file.
(b)	Normalize the data points. Use Min-Max scaling
©	Create a sequence of data points based on last three days to predict the next day For example normalized_temps is [0.1, 0.2, 0.3, 0.4, 0.5, 0.6], and n_input_days is 3. For i = 0: X becomes [[0.1, 0.2, 0.3]] y becomes [0.4] For i = 1: X becomes [[0.1, 0.2, 0.3], [0.2, 0.3, 0.4]] y becomes [0.4, 0.5] similarly for others
(d)	You have to build LSTM model to predict the temperature of next day based on previous days temperature. Use MSE as the loss function and optimizer 'adam'. LSTMs require 3D input because they process data sequentially over time
(e)	Train your model for 50 epoch.
(f)	Use the model to predict values
	import numpy as np from keras.models import Sequential from keras.layers import LSTM, Dense
Q2	Predict the next number in a sequence using RNN. Use MSE as the loss function and optimizer 'adam'.
Q3	Predict the next character in a sequence using RNN given a string of characters. Use sparse_categorical_crossentropy as the loss function and optimizer 'adam'.

NAME - VISHAL MARANDI | ROLL NO. - BTECH/10119/21 | DL ASSIGNMENT

```
# Required Libraries
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Load the data (Q1a)
data = pd.read_csv('/content/temp.csv')
print("Original Data:")
print(data.head())
```

```
Original Data:
                       date  temperature_2m  relative_humidity_2m  \
0  2024-10-30 00:00:00+00:00       21.776499                  92.0
1  2024-10-30 01:00:00+00:00       21.826500                  91.0
2  2024-10-30 02:00:00+00:00       23.076500                  86.0
3  2024-10-30 03:00:00+00:00       24.726500                  81.0
4  2024-10-30 04:00:00+00:00       26.376500                  77.0

   apparent_temperature  precipitation_probability  precipitation
rain  \
0             25.369894                        5.0            0.0
0.0
1             25.363144                       15.0            0.0
0.0
2             26.651030                       13.0            0.0
0.0
3             28.552795                       13.0            0.0
0.0
4             30.727776                       28.0            0.0
0.0

   weather_code  pressure_msl  surface_pressure  cloud_cover
cloud_cover_mid  \
0           1.0        1010.6         940.59380         53.0
35.0
1           2.0        1011.7         941.62900         52.0
26.0
2           2.0        1012.5         942.65704         71.0
56.0
3           2.0        1013.4         943.86630         47.0
31.0
4           2.0        1013.4         944.23350         58.0
37.0

   visibility  wind_speed_120m  wind_gusts_10m  uv_index
```

```
0     24140.0        11.246759        8.640000      0.00
1     24140.0        11.609651        9.360000      0.10
2     24140.0        12.722830       10.440001      0.85
3     24140.0         5.815978       12.959999      1.70
4     24140.0         5.052841       11.879999      2.95
```

```
# Q1(b): Normalize the data using Min-Max scaling
scaler = MinMaxScaler()
temperature_data = data['temperature_2m'].values.reshape(-1, 1)
normalized_temps = scaler.fit_transform(temperature_data)

print("\nNormalized Temperature Data (first 5 values):")
print(normalized_temps[:5])
```

```
Normalized Temperature Data (first 5 values):
[[0.38805478]
 [0.3921345 ]
 [0.49412537]
 [0.62875332]
 [0.76338126]]
```

```
# Q1(c): Create sequences for the last three days (72 hours) to
predict next day
def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:(i + seq_length)])
        y.append(data[i + seq_length])
    return np.array(X), np.array(y)

seq_length = 72  # 3 days of hourly data
X, y = create_sequences(normalized_temps, seq_length)

print("\nSequence Shape:", X.shape)
print("Target Shape:", y.shape)
```

```
Sequence Shape: (312, 72, 1)
Target Shape: (312, 1)
```

```
# Q1(d): Build LSTM model
model = Sequential([
    LSTM(50, activation='relu', input_shape=(seq_length, 1),
return_sequences=True),
    LSTM(50, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
```

```
print("\nModel Summary:")
model.summary()
```

```
Model Summary:

/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/
rnn.py:204: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)

Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 72, 50)	10,400
lstm_3 (LSTM)	(None, 50)	20,200
dense_1 (Dense)	(None, 1)	51

```
 Total params: 30,651 (119.73 KB)

 Trainable params: 30,651 (119.73 KB)

 Non-trainable params: 0 (0.00 B)
```

```
# Q1(e): Train the model
# Reshape input data for LSTM [samples, timesteps, features]
X = X.reshape((X.shape[0], X.shape[1], 1))

history = model.fit(X, y,
                    epochs=50,
                    batch_size=32,
                    validation_split=0.2,
                    verbose=1)
```

```
Epoch 1/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 4s 112ms/step - loss: 0.3183 - val_loss:
```

```
0.1674
Epoch 2/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 64ms/step - loss: 0.2261 - val_loss:
0.1040
Epoch 3/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - loss: 0.1084 - val_loss:
0.1049
Epoch 4/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 82ms/step - loss: 0.0923 - val_loss:
0.0687
Epoch 5/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 94ms/step - loss: 0.0891 - val_loss:
0.0681
Epoch 6/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 90ms/step - loss: 0.0756 - val_loss:
0.0758
Epoch 7/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 91ms/step - loss: 0.0780 - val_loss:
0.0705
Epoch 8/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step - loss: 0.0666 - val_loss:
0.0687
Epoch 9/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - loss: 0.0698 - val_loss:
0.0711
Epoch 10/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - loss: 0.0670 - val_loss:
0.0670
Epoch 11/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - loss: 0.0677 - val_loss:
0.0697
Epoch 12/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 59ms/step - loss: 0.0715 - val_loss:
0.0690
Epoch 13/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - loss: 0.0681 - val_loss:
0.0666
Epoch 14/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 57ms/step - loss: 0.0687 - val_loss:
0.0693
Epoch 15/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 58ms/step - loss: 0.0666 - val_loss:
0.0646
Epoch 16/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step - loss: 0.0651 - val_loss:
0.0694
Epoch 17/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - loss: 0.0648 - val_loss:
0.0627
```

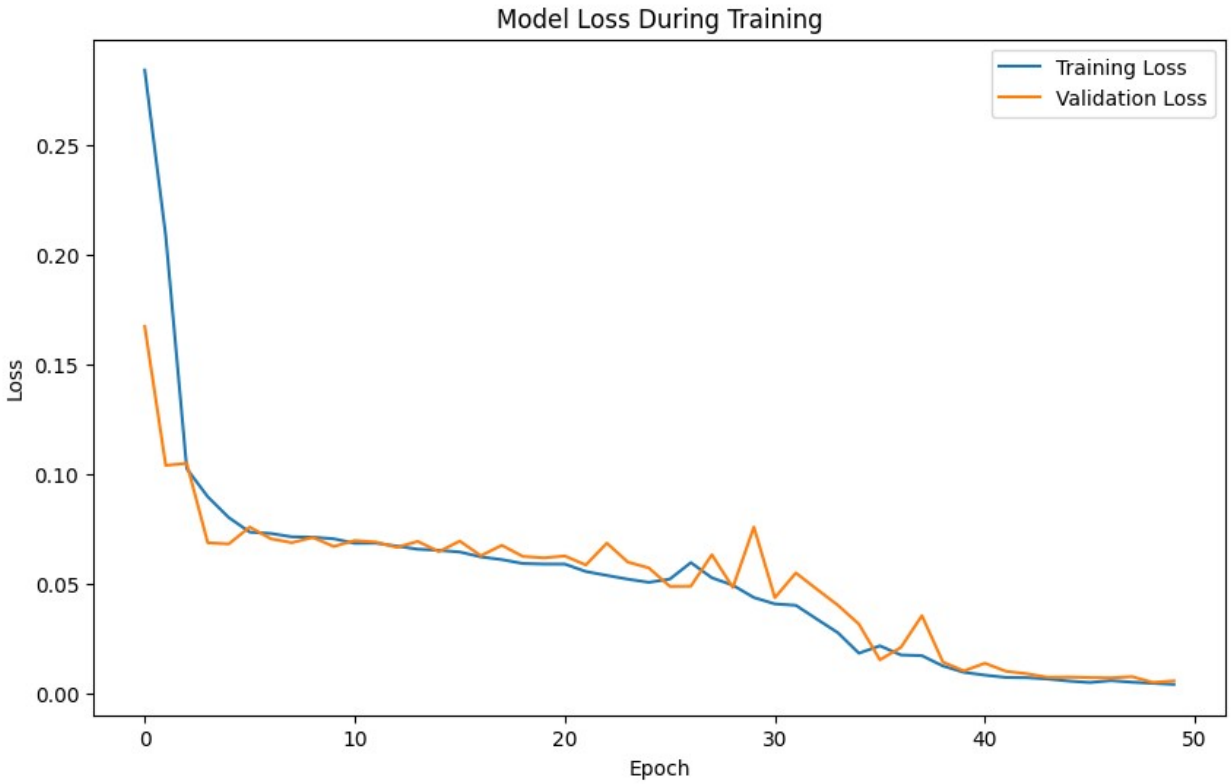
```
Epoch 18/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - loss: 0.0592 - val_loss:
0.0675
Epoch 19/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step - loss: 0.0604 - val_loss:
0.0624
Epoch 20/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 62ms/step - loss: 0.0580 - val_loss:
0.0618
Epoch 21/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - loss: 0.0550 - val_loss:
0.0626
Epoch 22/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - loss: 0.0539 - val_loss:
0.0585
Epoch 23/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 62ms/step - loss: 0.0538 - val_loss:
0.0685
Epoch 24/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 62ms/step - loss: 0.0536 - val_loss:
0.0598
Epoch 25/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 59ms/step - loss: 0.0510 - val_loss:
0.0572
Epoch 26/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 95ms/step - loss: 0.0487 - val_loss:
0.0488
Epoch 27/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 94ms/step - loss: 0.0533 - val_loss:
0.0488
Epoch 28/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 86ms/step - loss: 0.0534 - val_loss:
0.0632
Epoch 29/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 59ms/step - loss: 0.0476 - val_loss:
0.0483
Epoch 30/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - loss: 0.0471 - val_loss:
0.0758
Epoch 31/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step - loss: 0.0433 - val_loss:
0.0436
Epoch 32/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - loss: 0.0435 - val_loss:
0.0549
Epoch 33/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - loss: 0.0322 - val_loss:
0.0475
Epoch 34/50
```

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 65ms/step - loss: 0.0327 - val_loss:
0.0402
Epoch 35/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step - loss: 0.0202 - val_loss:
0.0316
Epoch 36/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - loss: 0.0186 - val_loss:
0.0153
Epoch 37/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step - loss: 0.0173 - val_loss:
0.0209
Epoch 38/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - loss: 0.0183 - val_loss:
0.0354
Epoch 39/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - loss: 0.0128 - val_loss:
0.0142
Epoch 40/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - loss: 0.0102 - val_loss:
0.0102
Epoch 41/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - loss: 0.0090 - val_loss:
0.0137
Epoch 42/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 62ms/step - loss: 0.0073 - val_loss:
0.0100
Epoch 43/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 61ms/step - loss: 0.0068 - val_loss:
0.0089
Epoch 44/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - loss: 0.0068 - val_loss:
0.0073
Epoch 45/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 82ms/step - loss: 0.0055 - val_loss:
0.0074
Epoch 46/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 96ms/step - loss: 0.0050 - val_loss:
0.0072
Epoch 47/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 101ms/step - loss: 0.0056 - val_loss:
0.0070
Epoch 48/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step - loss: 0.0051 - val_loss:
0.0076
Epoch 49/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - loss: 0.0043 - val_loss:
0.0049
Epoch 50/50
```

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 65ms/step - loss: 0.0039 - val_loss:
0.0056
```

```python
# Q1(f): Make predictions
# Prepare last sequence for prediction
last_sequence = normalized_temps[-seq_length:]
last_sequence = last_sequence.reshape((1, seq_length, 1))

# Make prediction
predicted_normalized = model.predict(last_sequence)

# Denormalize prediction
predicted_temp = scaler.inverse_transform(predicted_normalized)

print("\nPrediction Results:")
print(f"Predicted next temperature: {predicted_temp[0][0]:.2f}°C")

# Plot training history
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 436ms/step

Prediction Results:
Predicted next temperature: 19.46°C
```




```python
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler

# Define the sequence
sequence = np.array([1, 2, 4, 8, 16, 32, 64, 128, 256, 512])

# Normalize the data
scaler = MinMaxScaler()
normalized_sequence = scaler.fit_transform(sequence.reshape(-1, 1))

# Create sequences for training
def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:(i + seq_length)])
        y.append(data[i + seq_length])
    return np.array(X), np.array(y)

seq_length = 3
X, y = create_sequences(normalized_sequence, seq_length)

# Reshape input to be [samples, time steps, features]
X = X.reshape((X.shape[0], X.shape[1], 1))
```

```python
# Define the model
model = Sequential([
    LSTM(50, activation='relu', input_shape=(seq_length, 1),
return_sequences=True),
    LSTM(50, activation='relu'),
    Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Train the model
history = model.fit(X, y, epochs=100, batch_size=1, verbose=1)

# Make a prediction
last_sequence = normalized_sequence[-seq_length:]
last_sequence = last_sequence.reshape((1, seq_length, 1))
predicted_normalized = model.predict(last_sequence)

# Denormalize the prediction
predicted = scaler.inverse_transform(predicted_normalized)

print(f"Sequence: {sequence}")
print(f"Predicted next number: {predicted[0][0]:.2f}")

# Plot the training loss
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'])
plt.title('Model Loss During Training')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()
```

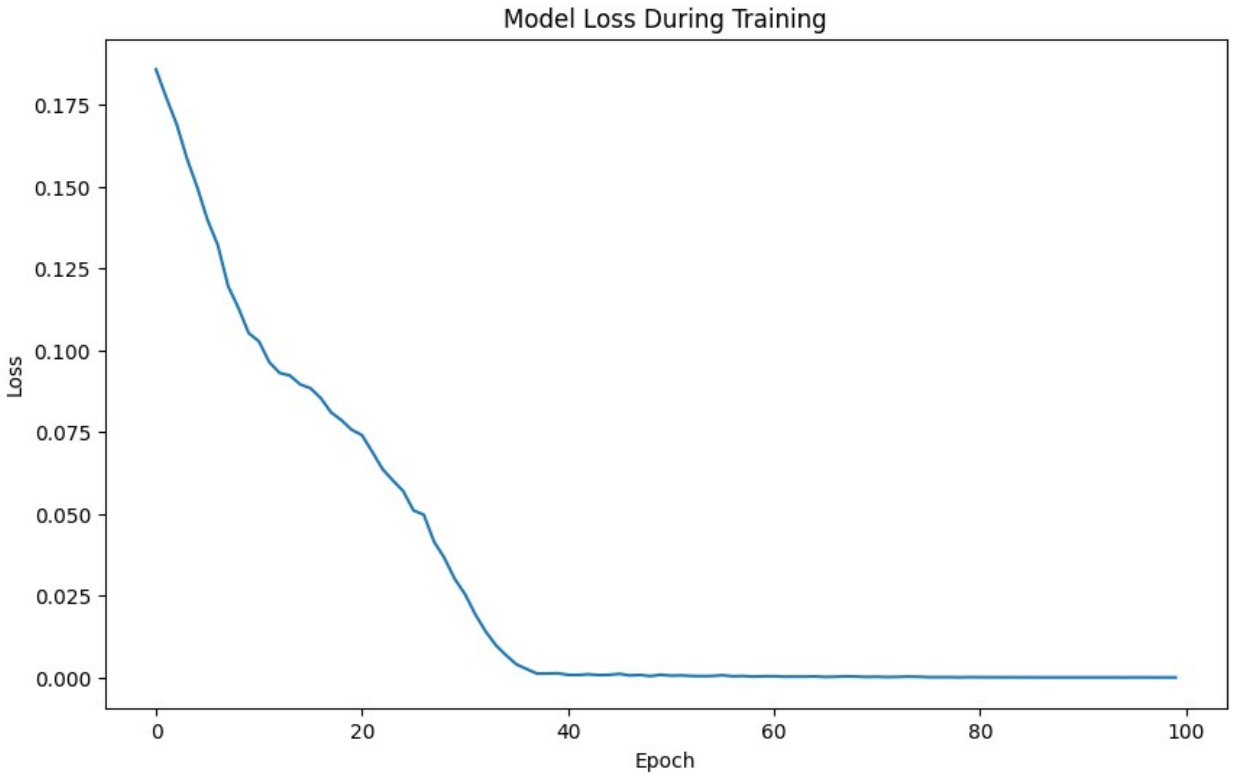
```
Epoch 1/100

/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/
rnn.py:204: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)

7/7 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1505
Epoch 2/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.1349
Epoch 3/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.3323
Epoch 4/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0996
Epoch 5/100
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.1356
Epoch 6/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1225
Epoch 7/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1377
Epoch 8/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0573
Epoch 9/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0725
Epoch 10/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0569
Epoch 11/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0872
Epoch 12/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0462
Epoch 13/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0439
Epoch 14/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0649
Epoch 15/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0899
Epoch 16/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1173
Epoch 17/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.1440
Epoch 18/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0646
Epoch 19/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0682
Epoch 20/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.1061
Epoch 21/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0370
Epoch 22/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0718
Epoch 23/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0367
Epoch 24/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0450
Epoch 25/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0317
Epoch 26/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0337
Epoch 27/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0856
Epoch 28/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0474
Epoch 29/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0362
```

```
Epoch 30/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0229
Epoch 31/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0292
Epoch 32/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0150
Epoch 33/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0112
Epoch 34/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0073
Epoch 35/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0064
Epoch 36/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0029
Epoch 37/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0022
Epoch 38/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 8.7448e-04
Epoch 39/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0014
Epoch 40/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 9.4513e-04
Epoch 41/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 5.3487e-04
Epoch 42/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 5.6396e-04
Epoch 43/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 9.1996e-04
Epoch 44/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0010
Epoch 45/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.5551e-04
Epoch 46/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0011
Epoch 47/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 8.5797e-04
Epoch 48/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.5944e-04
Epoch 49/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 3.3262e-04
Epoch 50/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 7.9507e-04
Epoch 51/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 5.7610e-04
Epoch 52/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 6.8836e-04
Epoch 53/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 4.1971e-04
Epoch 54/100
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 2.4971e-04
Epoch 55/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.3731e-04
Epoch 56/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 6.7571e-04
Epoch 57/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 3.7042e-04
Epoch 58/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 5.4504e-04
Epoch 59/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.0497e-04
Epoch 60/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.4908e-04
Epoch 61/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.2530e-04
Epoch 62/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 2.6994e-04
Epoch 63/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.7053e-04
Epoch 64/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 2.6442e-04
Epoch 65/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.4208e-04
Epoch 66/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 2.0784e-04
Epoch 67/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.8411e-04
Epoch 68/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.0888e-04
Epoch 69/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 3.0997e-04
Epoch 70/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 1.6669e-04
Epoch 71/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 2.2517e-04
Epoch 72/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 1.0623e-04
Epoch 73/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 1.4488e-04
Epoch 74/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 2.9294e-04
Epoch 75/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 2.6550e-04
Epoch 76/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 1.9894e-04
Epoch 77/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 1.2361e-04
Epoch 78/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 7.9886e-05
```

```
Epoch 79/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 5.9101e-05
Epoch 80/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.0901e-04
Epoch 81/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 6.1116e-05
Epoch 82/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 1.0688e-04
Epoch 83/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 9.0334e-05
Epoch 84/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 6.4809e-05
Epoch 85/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 8.9256e-05
Epoch 86/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 8.8408e-05
Epoch 87/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 4.4358e-05
Epoch 88/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 4.5070e-05
Epoch 89/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.2587e-05
Epoch 90/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 6.2594e-05
Epoch 91/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 5.5651e-05
Epoch 92/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 5.8087e-05
Epoch 93/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 3.9013e-05
Epoch 94/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 4.5478e-05
Epoch 95/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 3.8964e-05
Epoch 96/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 4.3546e-05
Epoch 97/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 5.8468e-05
Epoch 98/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 4.3040e-05
Epoch 99/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 5.8045e-05
Epoch 100/100
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.3155e-05
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 286ms/step
Sequence: [  1   2   4   8  16  32  64 128 256 512]
Predicted next number: 1004.71
```




```
import numpy as np
import tensorflow as tf

# Sample text data (first few lines of "Alice's Adventures in
Wonderland")
text = ("Alice was beginning to get very tired of sitting by her
sister on the bank, "
        "and of having nothing to do: once or twice she had peeped
into the book her sister "
        "was reading, but it had no pictures or conversations in it,
'and what is the use of "
        "a book,' thought Alice, 'without pictures or
conversations?'")

# Create a set of unique characters and a mapping from characters to
integers
chars = sorted(set(text))
char_to_int = {c: i for i, c in enumerate(chars)}
int_to_char = {i: c for i, c in enumerate(chars)}

# Prepare the input and output sequences
sequence_length = 5
X = []
y = []

for i in range(len(text) - sequence_length):
```

```
    seq_in = text[i:i + sequence_length]
    seq_out = text[i + sequence_length]
    X.append([char_to_int[char] for char in seq_in])
    y.append(char_to_int[seq_out])

X = np.array(X)
y = np.array(y)

# Reshape the input to be [samples, time steps, features]
X = X.reshape((X.shape[0], X.shape[1], 1))

# Normalize input
X = X / float(len(chars))

# Define the RNN model
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(100, input_shape=(X.shape[1], 1),
return_sequences=False),
    tf.keras.layers.Dense(len(chars), activation='softmax')
])
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/
rnn.py:204: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

```
# Compile the model
model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
# Fit the model
model.fit(X, y, epochs=100, batch_size=32)
```

```
Epoch 1/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.0841 - loss:
3.2235
Epoch 2/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1979 - loss:
2.9969
Epoch 3/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.1900 - loss:
2.9052
Epoch 4/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1912 - loss:
2.8030
Epoch 5/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2218 - loss:
2.7649
Epoch 6/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1860 - loss:
```

```
2.7317
Epoch 7/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2022 - loss:
2.7772
Epoch 8/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2007 - loss:
2.7453
Epoch 9/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1812 - loss:
2.7321
Epoch 10/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2121 - loss:
2.6904
Epoch 11/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2256 - loss:
2.6391
Epoch 12/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2449 - loss:
2.6409
Epoch 13/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1909 - loss:
2.7137
Epoch 14/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2002 - loss:
2.6657
Epoch 15/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1896 - loss:
2.6428
Epoch 16/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2075 - loss:
2.6467
Epoch 17/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2249 - loss:
2.6405
Epoch 18/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1802 - loss:
2.6873
Epoch 19/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2114 - loss:
2.6107
Epoch 20/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1793 - loss:
2.6806
Epoch 21/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2487 - loss:
2.5932
Epoch 22/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2025 - loss:
2.6357
```

```
Epoch 23/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2369 - loss:
2.5603
Epoch 24/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2092 - loss:
2.6333
Epoch 25/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2350 - loss:
2.6433
Epoch 26/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2341 - loss:
2.5453
Epoch 27/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2221 - loss:
2.5735
Epoch 28/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2496 - loss:
2.5225
Epoch 29/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2560 - loss:
2.5658
Epoch 30/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2385 - loss:
2.5390
Epoch 31/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2085 - loss:
2.5650
Epoch 32/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2350 - loss:
2.5842
Epoch 33/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.1927 - loss:
2.5346
Epoch 34/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2275 - loss:
2.5615
Epoch 35/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2549 - loss:
2.5191
Epoch 36/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2185 - loss:
2.6064
Epoch 37/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2552 - loss:
2.4991
Epoch 38/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2245 - loss:
2.5382
Epoch 39/100
```

```
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2098 - loss:
2.5640
Epoch 40/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2141 - loss:
2.5585
Epoch 41/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2023 - loss:
2.5602
Epoch 42/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.2491 - loss:
2.4850
Epoch 43/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2801 - loss:
2.4796
Epoch 44/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2717 - loss:
2.4524
Epoch 45/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.3032 - loss:
2.3967
Epoch 46/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2677 - loss:
2.4582
Epoch 47/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2843 - loss:
2.4398
Epoch 48/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2475 - loss:
2.5200
Epoch 49/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2938 - loss:
2.4003
Epoch 50/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2442 - loss:
2.3965
Epoch 51/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2323 - loss:
2.4644
Epoch 52/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2588 - loss:
2.4267
Epoch 53/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2728 - loss:
2.4201
Epoch 54/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2746 - loss:
2.4166
Epoch 55/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.2270 - loss:
```

```
2.4556
Epoch 56/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2566 - loss:
2.3732
Epoch 57/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2262 - loss:
2.4484
Epoch 58/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.3107 - loss:
2.3380
Epoch 59/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.2877 - loss:
2.3551
Epoch 60/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.3131 - loss:
2.3117
Epoch 61/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.2247 - loss:
2.4157
Epoch 62/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.3001 - loss:
2.2673
Epoch 63/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.3228 - loss:
2.2488
Epoch 64/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3110 - loss:
2.3056
Epoch 65/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2755 - loss:
2.3308
Epoch 66/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2775 - loss:
2.3136
Epoch 67/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3025 - loss:
2.2642
Epoch 68/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3318 - loss:
2.2911
Epoch 69/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2705 - loss:
2.3449
Epoch 70/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2813 - loss:
2.2902
Epoch 71/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3046 - loss:
2.2783
```

```
Epoch 72/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3004 - loss:
2.2713
Epoch 73/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3237 - loss:
2.2536
Epoch 74/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2962 - loss:
2.2556
Epoch 75/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3389 - loss:
2.1917
Epoch 76/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3212 - loss:
2.2385
Epoch 77/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2884 - loss:
2.2298
Epoch 78/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.2983 - loss:
2.2432
Epoch 79/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.3084 - loss:
2.1828
Epoch 80/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3265 - loss:
2.2021
Epoch 81/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2925 - loss:
2.2351
Epoch 82/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2950 - loss:
2.2043
Epoch 83/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3387 - loss:
2.1649
Epoch 84/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3124 - loss:
2.2189
Epoch 85/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2900 - loss:
2.1531
Epoch 86/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2761 - loss:
2.2035
Epoch 87/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3335 - loss:
2.1322
Epoch 88/100
```

```
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.2917 - loss:
2.1624
Epoch 89/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3396 - loss:
2.0920
Epoch 90/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3217 - loss:
2.1489
Epoch 91/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3135 - loss:
2.1676
Epoch 92/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3420 - loss:
2.1100
Epoch 93/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3350 - loss:
2.1146
Epoch 94/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3460 - loss:
2.0988
Epoch 95/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3963 - loss:
1.9904
Epoch 96/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3570 - loss:
2.0936
Epoch 97/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.3925 - loss:
2.0632
Epoch 98/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3687 - loss:
2.0485
Epoch 99/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3577 - loss:
2.0233
Epoch 100/100
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.3363 - loss:
2.0386

<keras.src.callbacks.history.History at 0x7bfc16ad59c0>
```

```python
# Function to predict the next character
def predict_next_char(model, input_seq):
    input_data = np.array([char_to_int[char] for char in input_seq])
    input_data = input_data.reshape((1, len(input_seq), 1)) /
float(len(chars))

    prediction = model.predict(input_data, verbose=0)
    index = np.argmax(prediction)
```

```
    return int_to_char[index]

# Example prediction
input_sequence = "the book"
next_char = predict_next_char(model, input_sequence)
print(f"The next character after '{input_sequence}' is '{next_char}'")
```

```
The next character after 'the book' is 'p'
```

Assignment-9

Q1. import numpy **as** np

import pandas **as** pd

import matplotlib.pyplot **as** plt

df = pd**.**read_csv('LSTM.csv', index_col='Date', parse_dates=**True**)

df**.**index**.**freq = 'MS'

Q2. Find head()

Q3. Plot prediction Vs Year.

Q4. Select first 156 rows for training and 156 onwards for test.

Q5. Normalize the data.

Q6. Add following code:

from keras.preprocessing.sequence **import** TimeseriesGenerator

 # define generator model

n_inputs = 3

n_features = 1

generator = TimeseriesGenerator(scaled_train, scaled_train, length=n_inputs, batch_size=1)

Q7. Find output:

Date	Production
1975-01-01	834
1975-02-01	782
1975-03-01	892
1975-04-01	903
1975-05-01	966

```
X,y = generator[1]
print(f'Given the array: \n {X.flatten()}')
print(f'Predict this y: \n {y}')
```

Q8. do the same thing but with 12 months of data

```
n_inputs = 12
generator = TimeseriesGenerator(scaled_train, scaled_train, length=n_inputs,
    batch_size=1)
```

Q9. Use LSTM with 100 inputs with ReLU and use adam and loss=mse.

Q10. Find model summary

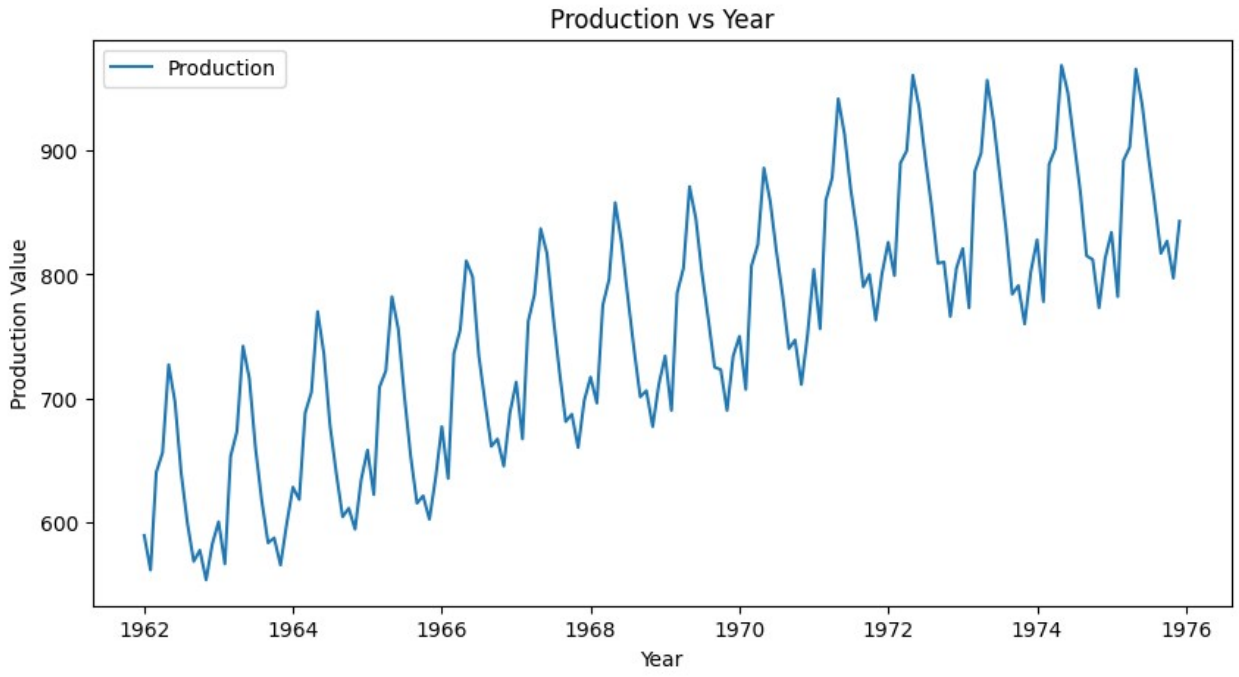
Q11. fit model with epocs=50.
Q12. Plot loss curve.
Q13. Find the following output
Q14. Plot production vs prediction.
Q15. calculate the error of the model.

Deep Learning Assignment - 9 | Name - Vishal Marandi |Roll no. BTECH/10119/21 | Section - 'A'

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv(r"C:\Users\Asus\Downloads\LSTM.csv",
index_col='Date', parse_dates=True)
df.index.freq = 'MS'

df.head()
```

Date	Production
1962-01-01	589
1962-02-01	561
1962-03-01	640
1962-04-01	656
1962-05-01	727

```
# Plot the production data vs. year
plt.figure(figsize=(10, 5))
plt.plot(df.index, df['Production'], label="Production")
plt.xlabel("Year")
plt.ylabel("Production Value")
plt.title("Production vs Year")
plt.legend()
plt.show()
```



```
train = df.iloc[:156]
test = df.iloc[156:]
print(train.size)
print(test.size)
```

```
156
12
```

```
# Q6: Normalize the data
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaled_train = scaler.fit_transform(train)
scaled_test = scaler.transform(test)
```

```
# Q7-8: Define Timeseries Generator for both 3-month and 12-month
inputs
from sklearn.preprocessing import MinMaxScaler
from keras.models import Sequential
from keras.layers import LSTM, Dense
from tensorflow.keras.preprocessing.sequence import
TimeseriesGenerator
from sklearn.metrics import mean_squared_error
# 3-month generator
n_inputs_3 = 3
n_inputs_12 = 12
generator_3 = TimeseriesGenerator(scaled_train, scaled_train,
length=n_inputs_3, batch_size=1)
generator_12 = TimeseriesGenerator(scaled_train, scaled_train,
length=n_inputs_12, batch_size=1)
# Sample output for 3-month generator
X_3, y_3 = generator_3[1]
print(f'Given the array (3 months): \n {X_3.flatten()}')
print(f'Predict this y: \n {y_3}')
# Sample output for 12-month generator
X, y = generator_12[1]
print(f'Given the array (12 months): \n {X.flatten()}')
print(f'Predict this y: \n {y}')
```

```
Given the array (3 months):
 [0.01923077 0.20913462 0.24759615]
Predict this y:
 [[0.41826923]]
Given the array (12 months):
 [0.01923077 0.20913462 0.24759615 0.41826923 0.34615385 0.20913462
 0.11057692 0.03605769 0.05769231 0.         0.06971154 0.11298077]
Predict this y:
 [[0.03125]]
```

```
# Q9-10: Define the LSTM model
model = Sequential([
```

```
    LSTM(100, activation='relu', input_shape=(n_inputs_12, 1)),  # 
input shape for 12-month generator
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
# Display model summary
print("Model Summary:")
model.summary()
```

```
Model Summary:
```

```
C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\site-
packages\keras\src\layers\rnn\rnn.py:204: UserWarning: Do not pass an 
`input_shape`/`input_dim` argument to a layer. When using Sequential 
models, prefer using an `Input(shape)` object as the first layer in 
the model instead.
  super().__init__(**kwargs)
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ lstm_1 (LSTM)                        │ (None, 100)                 │
40,800 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense_1 (Dense)                      │ (None, 1)                   │
101 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
┘
```

```
 Total params: 40,901 (159.77 KB)
```

```
 Trainable params: 40,901 (159.77 KB)
```

```
 Non-trainable params: 0 (0.00 B)
```

```
# Q11: Fit the model with 50 epochs
generator_12 = TimeseriesGenerator(scaled_train, scaled_train, 
length=n_inputs_12, batch_size=1)
history = model.fit(generator_12, epochs=50, verbose=1)
```

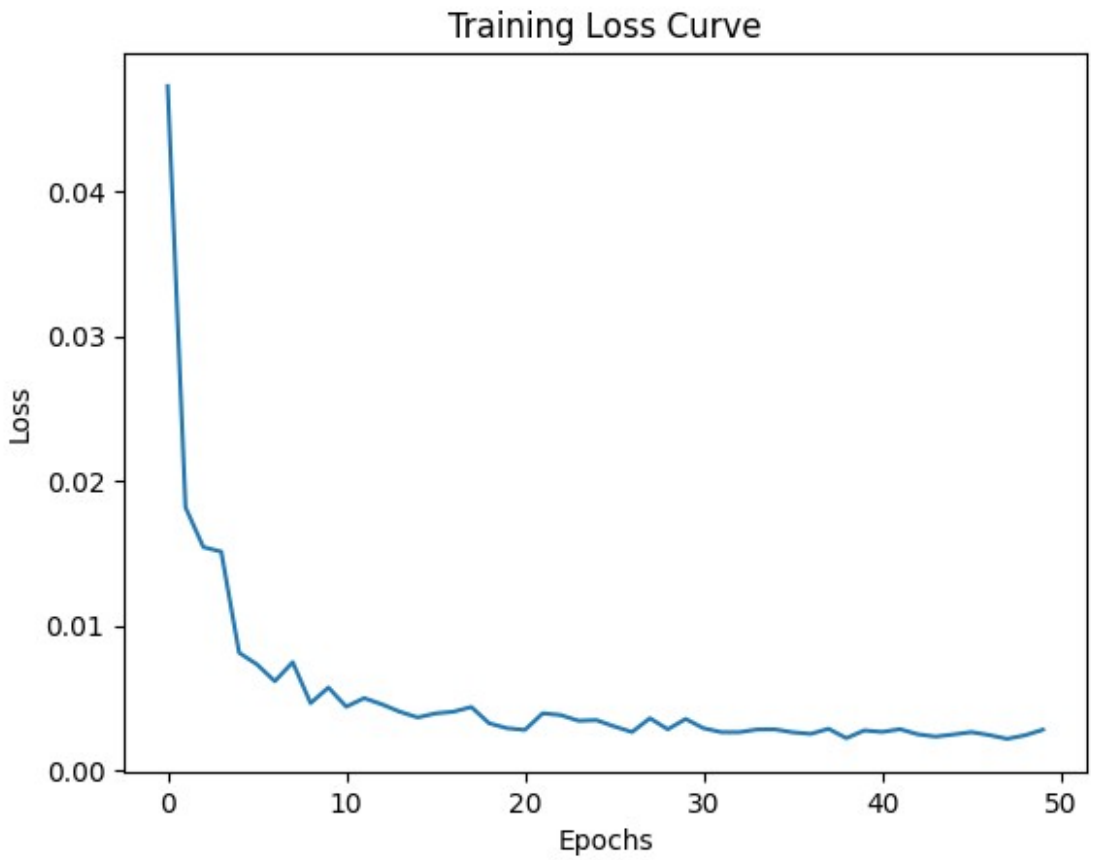
```
C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\site-
packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: 
UserWarning: Your `PyDataset` class should call 
`super().__init__(**kwargs)` in its constructor. `**kwargs` can 
include `workers`, `use_multiprocessing`, `max_queue_size`. Do not
```

```
pass these arguments to `fit()`, as they will be ignored.
  self._warn_if_super_not_called()

Epoch 1/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - loss: 0.0983
Epoch 2/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0178
Epoch 3/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - loss: 0.0127
Epoch 4/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0146
Epoch 5/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0084
Epoch 6/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - loss: 0.0066
Epoch 7/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - loss: 0.0052
Epoch 8/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - loss: 0.0087
Epoch 9/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - loss: 0.0041
Epoch 10/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0049
Epoch 11/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0046
Epoch 12/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 4s 23ms/step - loss: 0.0045
Epoch 13/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0048
Epoch 14/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 5s 18ms/step - loss: 0.0042
Epoch 15/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 6s 21ms/step - loss: 0.0028
Epoch 16/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0033
Epoch 17/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - loss: 0.0048
Epoch 18/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0031
Epoch 19/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0031
Epoch 20/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0036
Epoch 21/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0021
Epoch 22/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - loss: 0.0054
Epoch 23/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - loss: 0.0037
Epoch 24/50
```

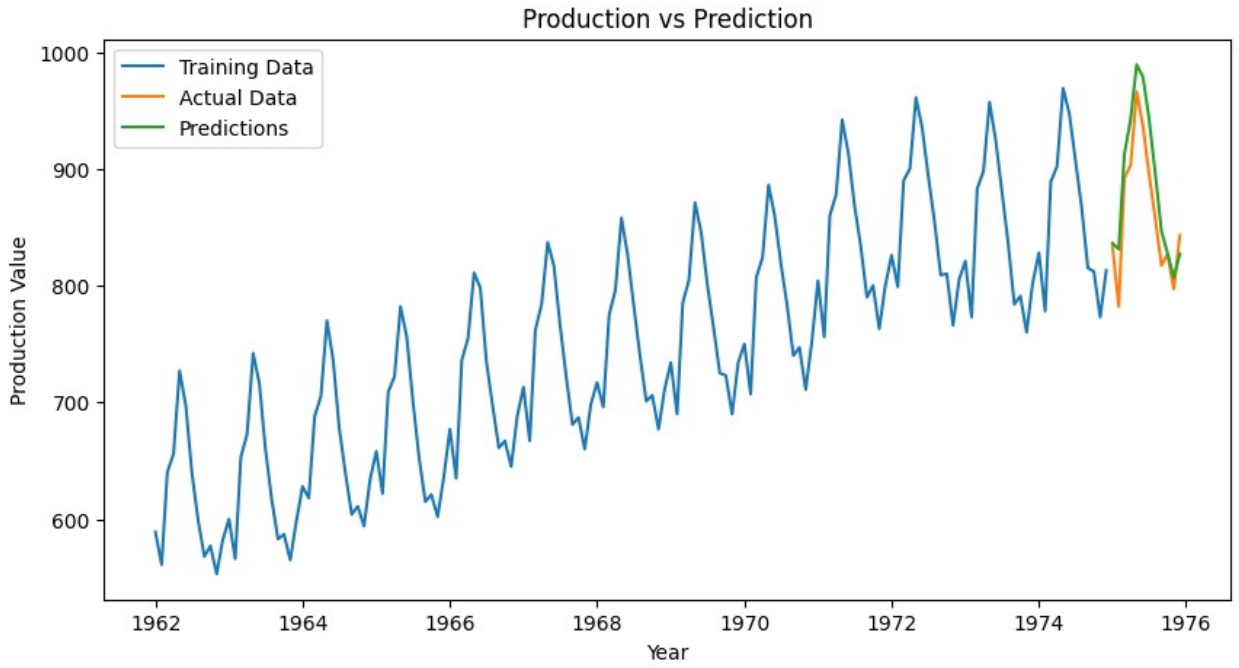
```
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0039
Epoch 25/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0031
Epoch 26/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - loss: 0.0028
Epoch 27/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - loss: 0.0027
Epoch 28/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - loss: 0.0043
Epoch 29/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 6s 20ms/step - loss: 0.0023
Epoch 30/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - loss: 0.0045
Epoch 31/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - loss: 0.0032
Epoch 32/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0019
Epoch 33/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - loss: 0.0023
Epoch 34/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0028
Epoch 35/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0044
Epoch 36/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0027
Epoch 37/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0024
Epoch 38/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0024
Epoch 39/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0020
Epoch 40/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.0030
Epoch 41/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0020
Epoch 42/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0026
Epoch 43/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0020
Epoch 44/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0022
Epoch 45/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0025
Epoch 46/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0022
Epoch 47/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0022
Epoch 48/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 0.0019
```

```
Epoch 49/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0022
Epoch 50/50
144/144 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0027
```

```python
# Q12: Plot the training loss curve
plt.plot(history.history['loss'])
plt.title("Training Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()

# Q13-14: Make predictions
predictions = []
batch = scaled_train[-n_inputs_12:]  # Last 12 months from training as
initial batch
current_batch = batch.reshape((1, n_inputs_12, 1))

for i in range(len(test)):
    pred = model.predict(current_batch)[0]
    predictions.append(pred)
    current_batch = np.append(current_batch[:, 1:, :], [[pred]],
axis=1)

# Inverse transform predictions
predictions = scaler.inverse_transform(predictions)

# Plot production vs prediction
plt.figure(figsize=(10, 5))
plt.plot(df.index[:156], train, label="Training Data")
plt.plot(df.index[156:], test, label="Actual Data")
plt.plot(df.index[156:], predictions, label="Predictions")
plt.xlabel("Year")
plt.ylabel("Production Value")
plt.title("Production vs Prediction")
plt.legend()
plt.show()

# Q15: Calculate and print Mean Squared Error (MSE)
mse = mean_squared_error(test, predictions)
print("Model Error (MSE):", mse)
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 834ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 109ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 113ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 99ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 133ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 112ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 144ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 102ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
```




```
Model Error (MSE): 969.8440586116493
```